

LangChain 中文教程（Python 版）

本教程由 LangChain 官方 Python 文档翻译整理而成

\newpage

1. LangChain 概述

原文链接: <https://docs.langchain.com/oss/python/langchain/overview>

LangChain 是一个开源框架，具有预构建的智能体架构和适用于任何模型或工具的集成——因此您可以构建能够随着生态系统快速演进而适应的智能体

LangChain 是开始构建由 LLM 驱动的智能体和应用程序的最简单方法。只需不到 10 行代码，您就可以连接到 OpenAI、Anthropic、Google 等。LangChain 提供了预构建的智能体架构和模型集成，帮助您快速入门，并将 LLM 无缝集成到您的智能体和应用程序中。

如果您想快速构建智能体和自主应用程序，我们建议您使用 LangChain。当您有更高级的需求，需要结合确定性和智能体工作流、大量自定义以及仔细控制的延迟时，请使用 LangGraph，我们的低级智能体编排框架和运行时。LangChain 智能体构建在 LangGraph 之上，以提供持久执行、流式传输、人在回路、持久化等功能。对于基本的 LangChain 智能体使用，您不需要了解 LangGraph。

创建智能体

```
# 安装 langchain 及其 Anthropic 集成
# pip install -qU langchain "langchain[anthropic]"
from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """获取指定城市的天气。"""
    return f"It's always sunny in {city}!"
```

```
agent = create_agent(  
    model="claude-sonnet-4-5-20250929",  
    tools=[get_weather],  
    system_prompt="You are a helpful assistant", # 你是一个乐于助人的助手  
)  
  
# 运行智能体  
agent.invoke(  
    {"messages": [{"role": "user", "content": "what is the weather in sf"}  
)
```

请参阅[安装说明](#)和[快速入门指南](#)，开始使用 LangChain 构建您自己的智能体和应用程序。

核心优势

标准化模型接口

不同的提供商有独特的 API 来与模型交互，包括响应格式。LangChain 标准化了您与模型的交互方式，因此您可以无缝切换提供商并避免锁定。

[了解更多](#)

易于使用、高度灵活的智能体

LangChain 的智能体抽象设计易于入门，让您可以用不到 10 行代码构建一个简单的智能体。但它也提供了足够的灵活性，让您能够进行所有您想要的上下文工程。

[了解更多](#)

构建在 LangGraph 之上

LangChain 的智能体构建在 LangGraph 之上。这使我们能够利用 LangGraph 的持久执行、人在回路支持、持久化等功能。

[了解更多](#)

使用 LangSmith 进行调试

通过可视化工具深入了解复杂的智能体行为，这些工具可以跟踪执行路径、捕获状态转换并提供详细的运行时指标。

[了解更多](#)

本文档由 LangChain 官方文档翻译而来

\newpage

2. 安装 LangChain

原文链接: <https://docs.langchain.com/oss/python/langchain/install>

要安装 LangChain 包：

使用 `pip`

```
pip install -U langchain
# 需要 Python 3.10 及以上版本
```

LangChain 为数百个 LLM 和上千种其他服务提供了集成能力。这些集成都位于各自独立的提供方包中。

安装常用模型集成

```
# 安装 OpenAI 集成
pip install -U langchain-openai

# 安装 Anthropic 集成
pip install -U langchain-anthropic
```

完整的可用集成列表请参见文档中的 **Integrations** 选项卡。

现在你已经安装好了 LangChain，可以继续查看[快速入门指南](#)开始上手使用。

本文基于 LangChain 官方安装文档翻译整理，原文见 [Install LangChain](https://docs.langchain.com/oss/python/langchain/quickstart)。

\newpage

3. 快速入门

原文链接: <https://docs.langchain.com/oss/python/langchain/quickstart>

本快速入门将带您在几分钟内从简单设置到构建一个功能完整的 AI 智能体。

LangChain 文档 MCP 服务器 如果您正在使用 AI 编程助手或 IDE（例如 Claude Code 或 Cursor），您应该安装 LangChain 文档 MCP 服务器以充分利用它。这确保您的智能体能够访问最新的 LangChain 文档和示例。

要求

对于这些示例，您需要：

- * 安装 LangChain 包
- * 设置 Claude (Anthropic) 账户并获取 API 密钥
- * 在终端中设置 `ANTHROPIC_API_KEY` 环境变量

虽然这些示例使用 Claude，但您可以通过更改代码中的模型名称并设置相应的 API 密钥来使用任何支持的模型。

构建基础智能体

首先创建一个简单的智能体，它可以回答问题并调用工具。该智能体将使用 Claude Sonnet 4.5 作为其语言模型，一个基本的天气函数作为工具，以及一个简单的提示来指导其行为。

```
from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """获取指定城市的天气。"""
    return f"It's always sunny in {city}!"

agent = create_agent(
```

```
model="claude-sonnet-4-5-20250929",
tools=[get_weather],
system_prompt="You are a helpful assistant", # 你是一个乐于助人的助手
)

# 运行智能体
agent.invoke(
    {"messages": [{"role": "user", "content": "what is the weather in sf"}]}
)
```

要了解如何使用 LangSmith 跟踪您的智能体，请参阅 [LangSmith 文档](#)。

构建真实世界的智能体

接下来，构建一个实用的天气预报智能体，展示关键的生产概念： 1. **详细的系统提示** 以获得更好的智能体行为 2. **创建工具** 与外部数据集成 3. **模型配置** 以获得一致的响应 4. **结构化输出** 以获得可预测的结果 5. **对话记忆** 用于类似聊天的交互 6. **创建并运行智能体** 创建一个功能完整的智能体

让我们逐步完成每个步骤：

1. 定义系统提示

系统提示定义了智能体的角色和行为。保持其具体和可操作：

```
SYSTEM_PROMPT = """You are an expert weather forecaster, who speaks in p

You have access to two tools:

- get_weather_for_location: use this to get the weather for a specific l
- get_user_location: use this to get the user's location

If a user asks you for the weather, make sure you know the location. If y
```

2. 创建工具

工具允许模型通过调用您定义的函数与外部系统交互。工具可以依赖于运行时上下文，也可以与智能体记忆交互。

请注意下面的 `get_user_location` 工具如何使用运行时上下文：

```
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime

@tool
def get_weather_for_location(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"

@dataclass
class Context:
    """Custom runtime context schema."""
    user_id: str

@tool
def get_user_location(runtime: ToolRuntime[Context]) -> str:
    """Retrieve user information based on user ID."""
    user_id = runtime.context.user_id
    return "Florida" if user_id == "1" else "SF"
```

工具应该被很好地记录：它们的名称、描述和参数名称成为模型提示的一部分。

LangChain 的 `@tool` 装饰器添加元数据，并通过 `ToolRuntime` 参数启用运行时注入。

3. 配置模型

为您的用例设置具有正确参数的语言模型：

```
from langchain.chat_models import init_chat_model

model = init_chat_model(
```

```
"claude-sonnet-4-5-20250929",
temperature=0.5,
timeout=10,
max_tokens=1000
)
```

根据选择的模型和提供商，初始化参数可能有所不同；请参阅它们的参考页面以获取详细信息。

4. 定义响应格式

如果需要智能体响应匹配特定模式，可以选择定义结构化响应格式。

```
from dataclasses import dataclass

# 这里我们使用 dataclass，也同样支持 Pydantic 模型。
@dataclass
class ResponseFormat:
    """智能体响应的结构化模式。"""
    # 搞笑双关回复（必填）
    punny_response: str
    # 如果有的话，关于天气的有趣信息
    weather_conditions: str | None = None
```

5. 添加记忆

向智能体添加记忆以在交互之间维护状态。这允许智能体记住先前的对话和上下文。

```
from langgraph.checkpoint.memory import InMemorySaver

# 使用内存检查点保存对话状态
checkpointer = InMemorySaver()
```

在生产环境中，使用持久化检查点，将数据保存到数据库。有关更多详细信息，请参阅[添加和管理记忆](#)。

6. 创建并运行智能体

现在将所有组件组装到智能体中并运行它！

```
from langchain.agents.structured_output import ToolStrategy

agent = create_agent(
    model=model,
    system_prompt=SYSTEM_PROMPT,
    tools=[get_user_location, get_weather_for_location],
    context_schema=Context,
    response_format=ToolStrategy(ResponseFormat),
    checkpointer=checkpointer
)

# `thread_id` 是一次对话的唯一标识符
config = {"configurable": {"thread_id": "1"}}

response = agent.invoke(
    {"messages": [{"role": "user", "content": "what is the weather outside?"}],
    config=config,
    context=Context(user_id="1")
)

print(response['structured_response'])
# ResponseFormat(
#     punny_response="Florida is still having a 'sun-derful' day! The sun is shining bright and clear.",
#     weather_conditions="It's always sunny in Florida!"
# )

# 注意：使用相同的 `thread_id` 可以继续这段对话。
response = agent.invoke(
    {"messages": [{"role": "user", "content": "thank you!"}]},
    config=config,
    context=Context(user_id="1")
)
```



```
print(response['structured_response'])
# ResponseFormat(
#     punny_response="You're 'thund-erfully' welcome! It's always a 'bre
#     weather_conditions=None
# )
```

完整示例代码

```
from dataclasses import dataclass

from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain.tools import tool, ToolRuntime
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.structured_output import ToolStrategy

# 定义系统提示
SYSTEM_PROMPT = """You are an expert weather forecaster, who speaks in p

You have access to two tools:

- get_weather_for_location: use this to get the weather for a specific l
- get_user_location: use this to get the user's location

If a user asks you for the weather, make sure you know the location. If y

# 定义上下文结构
@dataclass
class Context:
    """自定义运行时上下文结构。"""
    user_id: str

# 定义工具
@tool
```

```

def get_weather_for_location(city: str) -> str:
    """获取指定城市的天气。"""
    return f"It's always sunny in {city}!"

@tool
def get_user_location(runtime: ToolRuntime[Context]) -> str:
    """基于用户 ID 获取用户位置信息。"""
    user_id = runtime.context.user_id
    return "Florida" if user_id == "1" else "SF"

# 配置模型
model = init_chat_model(
    "claude-sonnet-4-5-20250929",
    temperature=0
)

# 定义响应格式
@dataclass
class ResponseFormat:
    """智能体响应的结构化模式。"""
    # 搞笑双关回复（必填）
    punny_response: str
    # 如果有的话，关于天气的有趣信息
    weather_conditions: str | None = None

# 设置记忆
checkpointer = InMemorySaver()

# 创建智能体
agent = create_agent(
    model=model,
    system_prompt=SYSTEM_PROMPT,
    tools=[get_user_location, get_weather_for_location],
    context_schema=Context,
    response_format=ToolStrategy(ResponseFormat),
    checkpointer=checkpointer
)

```

```

# 运行智能体
# `thread_id` 是一次对话的唯一标识符
config = {"configurable": {"thread_id": "1"}}

response = agent.invoke(
    {"messages": [{"role": "user", "content": "what is the weather outside"}],
    config=config,
    context=Context(user_id="1")
)

print(response['structured_response'])
# ResponseFormat(
#     punny_response="Florida is still having a 'sun-derful' day! The sun is shining bright and the weather conditions are perfect for a day in the state capital! The weather conditions are always sunny in Florida!"
# )

# 注意：使用相同的 `thread_id` 可以继续这段对话。
response = agent.invoke(
    {"messages": [{"role": "user", "content": "thank you!"}]},
    config=config,
    context=Context(user_id="1")
)

print(response['structured_response'])
# ResponseFormat(
#     punny_response="You're 'thund-erfully' welcome! It's always a 'breath-taking' experience in the state capital! The weather conditions are always sunny in Florida!"
#     weather_conditions=None
# )

```

要了解如何使用 LangSmith 跟踪您的智能体，请参阅 [LangSmith 文档](#)。

恭喜！您现在拥有一个可以执行以下操作的 AI 智能体：

- * 理解上下文 并记住对话
- * 智能使用多个工具
- * 以一致的格式提供结构化响应
- * 通过上下文处理用户特定信息
- * 在交互之间维护对话状态

本文档由 LangChain 官方文档翻译而来

\newpage

4. 更新日志

原文链接: <https://docs.langchain.com/oss/python/releases/changelog>

我们 Python 包的更新和改进日志

订阅: 我们的更新日志包含一个 RSS 源, 可以与 Slack、电子邮件、Discord 机器人 (如 Readybot 或 RSS Feeds to Discord Bot) 以及其他订阅工具集成。

2025 年 12 月 15 日

langchain v1.2.0

- `create_agent`: 通过工具上的新 `extras` 属性简化了对特定提供商工具参数和定义的支持。示例:
- 特定提供商的配置, 如 Anthropic 的程序化工具调用和工具搜索。
- 内置工具, 由 Anthropic、OpenAI 和其他提供商支持, 在客户端执行。
- 支持智能体 `response_format` 中的严格模式遵循 (参见 `ProviderStrategy` 文档)。

2025 年 12 月 8 日

langchain-google-genai v4.0.0

我们重写了 Google GenAI 集成, 以使用 Google 的统一生成式 AI SDK, 该 SDK 在同一接口下提供对 Gemini API 和 Vertex AI Platform 的访问。这包括最小的破坏性更改以及 `langchain-google-vertexai` 中已弃用的包。

有关详细信息, 请参阅[完整的发布说明和迁移指南](#)。

2025 年 11 月 25 日

langchain v1.1.0

- 模型配置文件：聊天模型现在通过 `.profile` 属性公开支持的功能和能力。这些数据来自 `models.dev`，这是一个提供模型能力数据的开源项目。
- 摘要中间件：更新以支持使用模型配置文件进行上下文感知摘要的灵活触发点。
- 结构化输出：`ProviderStrategy` 支持（原生结构化输出）现在可以从模型配置文件中推断。
- `create_agent` 的 `SystemMessage`：支持直接将 `SystemMessage` 实例传递给 `create_agent` 的 `system_prompt` 参数，启用高级功能，如缓存控制和结构化内容块。
- 模型重试中间件：新的中间件，用于自动重试失败的模型调用，具有可配置的指数退避。
- 内容审核中间件：OpenAI 内容审核中间件，用于检测和处理智能体交互中的不安全内容。支持检查用户输入、模型输出和工具结果。

2025 年 10 月 20 日

v1.0.0

langchain

- [发布说明](#)
- [迁移指南](#)

langgraph

- [发布说明](#)
- [迁移指南](#)

如果您遇到任何问题或有反馈，请提交问题，以便我们改进。要查看 v0.x 文档，请访问 [归档内容和 API 参考](#)。

本文档由 LangChain 官方文档翻译而来

\newpage

5. LangChain 的理念 (Philosophy)

原文链接: <https://docs.langchain.com/oss/python/langchain/philosophy>

LangChain 的存在目标, 是在 **尽可能方便开发者开始使用 LLM 构建应用** 的同时, 依然保持足够的灵活性和可用于生产环境的可靠性。

LangChain 的设计由几条核心信念驱动:

- LLM 是一种强大且全新的技术。
- 当 LLM 与外部数据源结合使用时, 会变得更强大。
- LLM 将重塑未来应用的形态, 未来的应用会越来越具“智能体 (agentic)”特征。
- 我们目前仍处在这种变革的早期阶段。
- 虽然做一个“原型级”的智能体应用很容易, 但要做出足够可靠、可真正投入生产的智能体依然非常困难。

基于此, LangChain 有两个核心关注点:

1. 让开发者可以方便地使用“最好的模型”。

不同提供商暴露出的 API 各不相同, 包括模型参数、消息格式等。标准化这些模型输入与输出是 LangChain 的核心目标之一, 这样开发者就可以轻松切换到最新、最先进的模型, 而不被某个厂商锁定。

1. 让开发者更容易用模型来编排复杂流程, 并与其他数据和计算进行交互。

模型不应仅仅用于 文本生成, 它还应该被用来编排更复杂的流程, 这些流程会与其他数据源交互。LangChain 让你可以轻松定义供 LLM 动态调用的工具, 同时也帮助解析和访问非结构化数据。

历史沿革 (History)

由于这一领域变化极快, LangChain 本身也在持续演进。下面是一个简要时间线, 展示了 LangChain 如何随着“用 LLM 构建应用”这一概念的演变而变化:

2022-10-24 — v0.0.1

在 ChatGPT 推出前一个月，LangChain 以 Python 包的形式发布。它最初包含两个主要组件：

- LLM 抽象层
- “Chains”：为常见用例预先设定的一系列计算步骤。例如：RAG（检索增强生成）包含“先检索，再生成”两个步骤。

“LangChain”这个名字来源于“Language”（语言/语言模型）和“Chains”（链式调用）。

2022-12

第一批通用智能体被加入 LangChain。

这些通用智能体基于 ReAct 论文（ReAct = Reasoning and Acting，推理与行动）。它们使用 LLM 生成代表工具调用的 JSON，然后解析这段 JSON 来决定实际要调用哪些工具。

2023-01 — Chat Completion API

OpenAI 发布“Chat Completion”API。

在此之前，模型只接受字符串并返回一个字符串；在 ChatCompletions API 中，模型演进为接受“消息列表”并返回一条消息。其他模型提供商很快跟进，LangChain 也随之更新以支持基于消息列表的交互方式。

2023-01 — JavaScript 版本

LangChain 发布 JavaScript 版本。

LLM 和智能体正在改变应用的构建方式，而 JavaScript 则是应用开发者最常用的语言之一。

2023-02 — LangChain Inc. 成立

LangChain Inc. 公司围绕开源的 LangChain 项目成立。

其主要目标是“让智能体无处不在”。团队意识到，LangChain 虽然是关键一环（它让使用 LLM 入门变得简单），但要做到这一点，还需要更多配套组件。

2023-03 — 函数调用 (Function Calling)

OpenAI 在 API 中发布 “function calling” 功能。

这使得 API 可以显式生成表示工具调用的 payload。其他模型提供商也陆续支持，LangChain 更新后，把这种方式作为调用工具的首选方案（取代之之前解析 JSON 的方式）。

2023-06 — LangSmith 发布

LangSmith 以闭源平台形式发布，由 LangChain Inc. 提供，用于可观测性与评估 (evals)。

构建智能体的最大难题是“可靠性”，LangSmith 正是为解决这一需求而生，它提供了可观测性和评估能力。LangChain 也更新以便无缝集成 LangSmith。

2024-01 — v0.1.0

LangChain 发布 0.1.0，这是其首个非 0.0.x 版本。

随着行业从“原型阶段”走向“生产阶段”，LangChain 也更加重视稳定性。

2024-02 — LangGraph 发布

LangGraph 以开源库形式发布。

最初的 LangChain 有两个重点：LLM 抽象层，以及用于快速上手常见应用的高级接口；但它缺少一个“低层级的编排层”，让开发者可以精细控制智能体的执行流程。

LangGraph 正是为此而生。

在构建 LangGraph 时，团队基于构建 LangChain 所获得的经验，引入了他们发现必需的功能：流式传输、持久执行、短期记忆、人在回路等。

2024-06 — 超过 700 个集成

LangChain 拥有超过 700 个集成。

这些集成从核心 LangChain 包中拆分出来，核心集成迁移到各自独立的包，其他集成则迁移到 `langchain-community`。

2024-10 — LangGraph 成为首选编排方式

对于任何超出“单次 LLM 调用”的 AI 应用，LangGraph 成为推荐的构建方式。

当开发者尝试提升应用的可靠性时，他们需要比高层接口更细粒度的控制。LangGraph

提供了这种低层级的灵活性。

在 LangChain 中，大多数 chains 和 agents 被标记为已弃用，并提供了迁移到 LangGraph 的指南。

但在 LangGraph 中仍保留了一个高层抽象：**智能体抽象**。它基于低层的 LangGraph 构建，并且与 LangChain 中最初的 ReAct 智能体接口保持一致。

2025-04 — 模型 API 走向多模态

模型开始支持文件、图像、视频等多种输入。

我们据此更新了 `langchain-core` 的消息格式，以便开发者可以用统一的方式指定这些多模态输入。

2025-10-20 — v1.0.0

LangChain 发布 1.0 版本，带来两项重大变化：

1. 对 `langchain` 中所有 chains 和 agents 完全重构。

所有原有的 chains 和 agents 被统一为一个高层抽象：**基于 LangGraph 构建的智能体抽象**。

这个抽象最初在 LangGraph 中创建，如今被迁移回 LangChain。

对于仍然使用旧版 LangChain chains/agents 且暂时不想升级的用户（仍然推荐升级），可以通过安装 `langchain-classic` 继续使用旧版。

2. 标准化消息内容格式。

模型 API 从“返回简单字符串内容的消息”逐步演进为返回更复杂的输出类型——推理块、引用信息、服务端工具调用等。

LangChain 也相应演进其消息格式，以在不同提供商之间对这些复杂结构进行标准化。

本文档由 LangChain 官方文档翻译而来，原文见 [Philosophy](#)。

\newpage

6. Agents（智能体）

原文链接: <https://docs.langchain.com/oss/python/langchain/agents>

Agents 将语言模型和工具（Tools）结合起来，用来创建可以 **推理目标、选择并调用工具、迭代执行直至完成任务** 的系统。 `create_agent` 提供了一个可用于生产环境的智能体实现。

一个智能体会反复循环，直到满足停止条件——要么模型给出最终输出，要么达到设定的迭代次数上限。

在 LangChain 中， `create_agent` 会基于 LangGraph 构建一个 **图（graph）** 形式的 **智能体运行时**。图由节点（步骤）和边（连接）组成，用于定义智能体如何处理信息。智能体会执行图中的不同节点，例如：

- 调用模型的 **模型节点**
- 执行某个工具的 **工具节点**
- 做前后处理或控制流的 **中间件节点**

核心组件

智能体由三部分核心组件组成：

- **模型（Model）**：负责理解、推理与生成。
- **工具（Tools）**：执行外部动作，如调用 API、查询数据库、检索文档等。
- **中间件（Middleware）与内存（Memory）**：扩展行为、控制流程、持久化状态。

下面分别介绍。

模型（Model）

LangChain 支持多种方式配置模型，包括 **静态模型** 与 **动态模型**。

静态模型（Static model）

静态模型在创建智能体时就配置好，并在执行期间保持不变。这是最常见也是最直接的用法。

你可以直接传入一个模型标识符字符串， `create_agent` 会自动为你初始化模型：

```
from langchain.agents import create_agent

# 使用模型标识符字符串创建智能体
agent = create_agent("openai:gpt-5", tools=tools)
```

对于常见 provider，模型前缀可以被自动推断，例如：

- "gpt-5" 会被推断为 "openai:gpt-5"

如果你需要对模型的配置（如温度、超时时间、最大 token 数等）做更精细控制，可以先通过对应 provider 包初始化模型实例，然后将这个实例传给 `create_agent`：

```
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent

# 配置更细致的模型参数
model = ChatOpenAI(
    model="gpt-4o",
    temperature=0.2,
    max_tokens=512,
    timeout=10,
)

agent = create_agent(
    model=model,
    tools=tools,
)
```

动态模型（Dynamic model）

动态模型指的是：在运行时根据当前状态或上下文选择不同的模型。这对实现复杂路由逻辑、成本优化或能力分级非常有用。

要使用动态模型，可以通过中间件来“包裹”模型调用。例如，根据对话长度选择不同的模型：

```

from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse

basic_model = ChatOpenAI(model="gpt-4o-mini")
advanced_model = ChatOpenAI(model="gpt-4o")

@wrap_model_call
def dynamic_model_selection(request: ModelRequest, handler) -> ModelResponse:
    """根据对话消息数量选择不同模型。"""
    message_count = len(request.state["messages"])

    if message_count > 10:
        # 对话较长时使用更强的模型
        model = advanced_model
    else:
        # 默认使用轻量模型以节省成本
        model = basic_model

    # 使用覆盖后的模型继续处理请求
    return handler(request.override(model=model))

agent = create_agent(
    model=basic_model,                # 默认模型
    tools=tools,
    middleware=[dynamic_model_selection],
)

```

如果你使用 **结构化输出 (structured output)**，并且需要动态模型选择，务必确保通过中间件传入的模型不是已经与工具强绑定的模型，否则可能影响工具调用和输出格式。

工具 (Tools)

工具是智能体与外部世界交互的接口，让模型可以“执行动作”，而不仅是“生成文本”。

与简单地在模型上绑定工具相比，智能体在工具调用方面提供了更多能力：

- 能按需要调用 **多个工具** 并串联它们
- 能在合适的情况下 **并行调用工具**
- 能基于前一步的结果 **动态决定下一个工具**
- 提供工具调用的 **重试与错误处理逻辑**
- 在多次工具调用之间 **保持状态持久性**

定义工具

你可以将普通的 Python 函数或协程封装为工具。使用 `@tool` 装饰器可以自定义工具的名称、描述和参数模式等属性。

```
from langchain.tools import tool
from langchain.agents import create_agent

@tool
def search(query: str) -> str:
    """搜索信息。"""
    return f"Results for: {query}"

@tool
def get_weather(location: str) -> str:
    """获取指定地点的天气信息。"""
    return f"Weather in {location}: Sunny, 72°F"

agent = create_agent(
    model,
    tools=[search, get_weather],
)
```

工具的名称、文档字符串（docstring）和参数名称都会出现在模型提示中，因此尽量写得清晰具体。

如果你传入一个空的工具列表，智能体就只包含一个 LLM 节点，不具备工具调用能力。

工具错误处理

你可以使用中间件来统一处理工具执行过程中出现的错误。通过 `@wrap_tool_call` 装饰器可以拦截每次工具调用，并自定义错误返回：

```
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage

@wrap_tool_call
def handle_tool_errors(request, handler):
    """自定义工具错误处理逻辑。"""
    try:
        return handler(request)
    except Exception as e:
        # 将错误信息包装为 ToolMessage 返回给模型
        return ToolMessage(
            content=f"Tool error: 请检查输入参数并重试。({str(e)}) ",
            tool_call_id=request.tool_call["id"],
        )

agent = create_agent(
    model="gpt-4o",
    tools=[search, get_weather],
    middleware=[handle_tool_errors],
)
```

当工具执行失败时，智能体会收到一个包含错误描述的 `ToolMessage`，模型可以据此决定下一步（例如提示用户修正输入、尝试其他工具等）。

系统提示 (System prompt)

你可以通过 `system_prompt` 参数来控制智能体的“角色”和“行为风格”，它可以是一个简单字符串，也可以是一个 `SystemMessage`。

```
from langchain.agents import create_agent

agent = create_agent(
    model,
    tools,
    system_prompt="You are a helpful assistant. Be concise and accurate."
)
```

当没有显式提供 `system_prompt` 时，智能体会仅根据对话消息来推断任务。对于某些 provider（例如 Anthropic），使用 `SystemMessage` 可以更精细地控制提示结构（如启用缓存、分块内容等）。

动态系统提示 (Dynamic system prompt)

有时你需要 根据运行时上下文或智能体状态动态调整系统提示，比如：

- 根据用户级别（新手 / 专家）给出不同详尽程度的回答
- 根据业务场景切换不同的助手机器人 persona

这可以通过中间件装饰器 `@dynamic_prompt` 实现：

```
from typing import TypedDict

from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest

class Context(TypedDict):
    """上下文结构定义。"""
    user_role: str # 用户角色, 比如 "expert" 或 "beginner"
```

```
@dynamic_prompt
def user_role_prompt(request: ModelRequest) -> str:
    """根据用户角色动态生成系统提示。"""
    user_role = request.runtime.context.get("user_role", "user")
    base_prompt = "You are a helpful assistant."

    if user_role == "expert":
        return f"{base_prompt} Provide detailed technical responses."
    elif user_role == "beginner":
        return f"{base_prompt} Explain concepts simply and avoid jargon."

    return base_prompt

agent = create_agent(
    model="gpt-4o",
    tools=[search],
    middleware=[user_role_prompt],
    context_schema=Context,
)

# 系统提示会根据 context 中的 user_role 字段进行动态设置
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Explain machine learning"}],
    context={"user_role": "expert"},
)
```

Structured Output（结构化输出）

在许多应用中，希望智能体返回 **结构化数据**（例如 JSON / 对象），而不是自由文本。LangChain 为此提供了两种策略：

- **ToolStrategy**：通过“工具”来实现结构化输出，适用于 provider 不支持原生结构化输出或行为不够稳定的情况。
- **ProviderStrategy**：利用 provider 自身的原生结构化输出能力。

示例（使用 **ToolStrategy** 将结构化响应描述为一个 schema）：

```
from dataclasses import dataclass

from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

@dataclass
class ContactInfo:
    """联系人信息结构。"""
    name: str
    email: str | None = None
    phone: str | None = None

agent = create_agent(
    model="gpt-4o",
    tools=tools,
    response_format=ToolStrategy(ContactInfo),
)
```

自 **langchain 1.0** 起，不再支持仅传入 schema（例如 `response_format=ContactInfo`）的简写写法，必须显式使用 **ToolStrategy** 或 **ProviderStrategy**。

内存 (Memory)

智能体会自动通过 **消息状态** 维护对话历史，这本身就提供了一种“短期记忆”。除此之外，你还可以配置自定义状态结构，以在对话中存储额外信息。

自定义状态 schema 推荐使用 `TypedDict` 来定义。例如：

```
from typing import TypedDict

class AgentState(TypedDict):
    """智能体自定义状态结构。"""
    messages: list
    # 比如累计消耗的 token 数
    total_tokens: int
```

在 `langchain 1.0` 之后，自定义 state 不再支持 Pydantic 模型或 dataclass，只支持 `TypedDict`。

你可以通过中间件或通过 `state_schema` 在创建智能体时显式指定这个状态结构，并在执行过程中对其进行读写，用于实现：

- 自定义记忆字段
- 统计与监控（如 token 用量）
- 特定业务相关的状态持久化

流式 (Streaming)

智能体可以以流式方式返回结果，让你在长流程中 **实时看到中间进度**，而不是等到整个流程结束。

```
for chunk in agent.stream(
    {
        "messages": [
            {"role": "user", "content": "Search for AI news and summarize"}
        ]
    }
```

```
    },
    stream_mode="values",
):
    latest_message = chunk["messages"][-1]

    if latest_message.content:
        # 输出模型当前生成的内容
        print(f"Agent: {latest_message.content}")
    elif latest_message.tool_calls:
        # 输出当前即将调用的工具名
        print(f"Calling tools: {[tc['name'] for tc in latest_message.tool_calls}]")
```

这非常适合构建带有“即时反馈”的应用，例如聊天助手、Dashboard、CLI 工具等。

中间件（Middleware）

中间件是智能体的“扩展点”，用于在执行的不同阶段插入自定义逻辑。常见用途包括：

- 在模型调用前预处理状态（例如裁剪历史消息、注入额外上下文）
- 在模型响应后进行修改或验证（例如安全策略、输出校验、Guardrails）
- 自定义工具错误处理（如上文 `handle_tool_errors` 示例）
- 基于当前状态和上下文进行动态模型选择
- 记录日志、埋点、监控和分析

通过中间件，你可以在 **不修改核心智能体逻辑** 的前提下，对其行为进行强大的扩展和细粒度控制。

本文档由 LangChain 官方文档翻译而来，原文见 [Agents](#)。

\newpage

7. Models（模型）

原文链接: <https://docs.langchain.com/oss/python/langchain/models>

LLM（大语言模型）是强大的 AI 工具，能够像人类一样理解和生成文本。它们功能多样，足以编写内容、翻译语言、总结和回答问题，而无需为每个任务进行专门训练。除了文本生成，许多模型还支持：

- * **工具调用** – 调用外部工具（如数据库查询或 API 调用）并在响应中使用结果。
- * **结构化输出** – 模型的响应被约束为遵循定义的格式。
- * **多模态** – 处理和返回文本以外的数据，如图像、音频和视频。
- * **推理** – 模型执行多步推理以得出结论。

模型是智能体的推理引擎。它们驱动智能体的决策过程，决定调用哪些工具、如何解释结果以及何时提供最终答案。您选择的模型的质量和直接影响智能体的基线可靠性和性能。不同的模型擅长不同的任务——有些更擅长遵循复杂指令，有些擅长结构化推理，有些支持更大的上下文窗口以处理更多信息。LangChain 的标准模型接口让您访问许多不同的提供商集成，这使得实验和切换模型以找到最适合您用例的模型变得容易。

有关特定提供商的集成信息和功能，请参阅提供商的聊天模型页面。

基本用法

模型可以通过两种方式使用：

1. **与智能体一起使用** – 在创建智能体时可以动态指定模型。
2. **独立使用** – 模型可以直接调用（在智能体循环之外），用于文本生成、分类或提取等任务，无需智能体框架。

相同的模型接口在这两种情况下都有效，这使您可以灵活地从简单开始，并根据需要扩展到更复杂的基于智能体的工作流。

初始化模型

在 LangChain 中使用独立模型的最简单方法是使用 `init_chat_model` 从您选择的聊天模型提供商初始化一个模型（以下示例）：

OpenAI

👉 阅读 [OpenAI 聊天模型集成文档](#)

```
pip install -U "langchain[openai]"
```

使用 `init_chat_model`：

```
import os
from langchain.chat_models import init_chat_model

os.environ["OPENAI_API_KEY"] = "sk-..."

model = init_chat_model("gpt-4.1")
```

使用模型类：

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4.1")
```

Anthropic

👉 阅读 [Anthropic 聊天模型集成文档](#)

```
pip install -U "langchain[anthropic]"
```

使用 init_chat_model：

```
import os
from langchain.chat_models import init_chat_model

os.environ["ANTHROPIC_API_KEY"] = "sk-..."

model = init_chat_model("claude-sonnet-4-5-20250929")
```

使用模型类：

```
from langchain_anthropic import ChatAnthropic

model = ChatAnthropic(model="claude-sonnet-4-5-20250929")
```

Azure

👉 阅读 [Azure 聊天模型集成文档](#)

```
pip install -U "langchain[openai]"
```

使用 `init_chat_model`:

```
import os
from langchain.chat_models import init_chat_model

os.environ["AZURE_OPENAI_API_KEY"] = "..."
os.environ["AZURE_OPENAI_ENDPOINT"] = "..."
os.environ["OPENAI_API_VERSION"] = "2025-03-01-preview"

model = init_chat_model(
    "azure_openai:gpt-4.1",
    azure_deployment=os.environ["AZURE_OPENAI_DEPLOYMENT_NAME"],
)
```

使用模型类:

```
from langchain_openai import AzureChatOpenAI

model = AzureChatOpenAI(
    azure_deployment=os.environ["AZURE_OPENAI_DEPLOYMENT_NAME"],
    model="gpt-4.1",
)
```

Google Gemini

👉 阅读 [Google GenAI 聊天模型集成文档](#)

```
pip install -U "langchain[google-genai]"
```

使用 `init_chat_model`:

```
import os
from langchain.chat_models import init_chat_model

os.environ["GOOGLE_API_KEY"] = "..."

model = init_chat_model("google_genai:gemini-2.5-flash-lite")
```

使用模型类:

```
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash-lite")
```

AWS Bedrock

👉 阅读 [AWS Bedrock 聊天模型集成文档](#)

```
pip install -U "langchain[aws]"
```

使用 `init_chat_model`:

```
from langchain.chat_models import init_chat_model

# 按照这里的步骤配置您的凭证：
# https://docs.aws.amazon.com/bedrock/latest/userguide/getting-started.h

model = init_chat_model(
    "anthropic.claude-3-5-sonnet-20240620-v1:0",
    model_provider="bedrock_converse",
)
```

使用模型类:

```
from langchain_aws import ChatBedrock

model = ChatBedrock(
    model_id="anthropic.claude-3-5-sonnet-20240620-v1:0",
    model_provider="bedrock_converse",
)
```

HuggingFace

👉 阅读 [HuggingFace 聊天模型集成文档](#)

```
pip install -U "langchain[huggingface]"
```

使用 `init_chat_model`:

```
import os
from langchain.chat_models import init_chat_model

os.environ["HUGGINGFACEHUB_API_TOKEN"] = "hf_..."

model = init_chat_model(
    "microsoft/Phi-3-mini-4k-instruct",
    model_provider="huggingface",
    temperature=0.7,
    max_tokens=1024,
)
```

使用模型类:

```
from langchain_huggingface import ChatHuggingFace

model = ChatHuggingFace(
    model_id="microsoft/Phi-3-mini-4k-instruct",
    temperature=0.7,
```



```
max_tokens=1024,  
)
```

调用模型：

```
response = model.invoke("Why do parrots talk?")
```

有关更多详细信息，包括如何传递模型参数的信息，请参阅 `init_chat_model`。

支持的模型

LangChain 支持所有主要的模型提供商，包括 OpenAI、Anthropic、Google、Azure、AWS Bedrock 等。每个提供商都提供具有不同功能的各种模型。有关 LangChain 中支持的模型的完整列表，请参阅[集成页面](#)。

关键方法

Invoke（调用） – 模型接收消息作为输入，并在生成完整响应后输出消息。

Stream（流式） – 调用模型，但在生成时实时流式传输输出。

Batch（批处理） – 批量向模型发送多个请求以提高处理效率。

除了聊天模型，LangChain 还提供对其他相关技术的支持，如嵌入模型和向量存储。有关详细信息，请参阅[集成页面](#)。

参数

聊天模型接受可用于配置其行为的参数。支持的完整参数集因模型和提供商而异，但标准参数包括：

model

类型: string

必需: 是

您要使用的特定模型的名称或标识符。您也可以使用 ':' 格式在单个参数中指定模型及其提供商，例如 'openai:o1'。

api_key

类型: string

与模型提供商进行身份验证所需的密钥。这通常在您注册访问模型时颁发。通常通过设置环境变量来访问。

temperature

类型: number

控制模型输出的随机性。较高的数字使响应更具创造性；较低的数字使响应更具确定性。

max_tokens

类型: number

限制响应中的总令牌数，有效控制输出的长度。

timeout

类型: number

在取消请求之前等待模型响应的最长时间（以秒为单位）。

max_retries

类型: number

如果请求由于网络错误或其他临时问题而失败，系统将尝试重新发送请求的最大次数。

调用

Invoke（调用）

`invoke` 方法接受消息作为输入并返回完整的响应。这是最常用的方法：

```
from langchain.chat_models import init_chat_model
```

```
model = init_chat_model("gpt-4o-mini")

# 使用字符串（自动转换为 HumanMessage）
response = model.invoke("What is the capital of France?")
print(response.content)
# "The capital of France is Paris."

# 使用消息对象
from langchain_core.messages import HumanMessage, SystemMessage

messages = [
    SystemMessage(content="You are a helpful assistant."),
    HumanMessage(content="What is 2+2?")
]
response = model.invoke(messages)
print(response.content)
# "2+2 equals 4."
```

Stream（流式）

`stream` 方法允许您实时接收响应块，这对于长响应或需要显示进度的应用程序很有用：

```
for chunk in model.stream("Tell me a story about a robot"):
    print(chunk.content, end="", flush=True)
```

Batch（批处理）

`batch` 方法允许您同时处理多个请求，这对于提高吞吐量很有用：

```
messages = [
    "What is the capital of France?",
    "What is the capital of Spain?",
    "What is the capital of Italy?",
]
```

```
responses = model.batch(messages)
for response in responses:
    print(response.content)
```

工具调用

许多现代 LLM 支持工具调用，允许模型请求执行外部函数。LangChain 通过 `bind_tools` 方法支持此功能：

```
from langchain.tools import tool
from langchain.chat_models import init_chat_model

@tool
def get_weather(location: str) -> str:
    """获取指定位置的天气信息。"""
    return f"{location} 的天气：晴天, 72°F"

model = init_chat_model("gpt-4o")
model_with_tools = model.bind_tools([get_weather])

response = model_with_tools.invoke("What's the weather in San Francisco?")
print(response.tool_calls)
# [{'name': 'get_weather', 'args': {'location': 'San Francisco'}, 'id':
```

结构化输出

结构化输出允许您将模型的响应约束为遵循特定模式。LangChain 通过 `with_structured_output` 方法支持此功能：

```
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model

class WeatherInfo(BaseModel):
```

```
"""天气信息"""
location: str = Field(description="城市名称")
temperature: float = Field(description="温度（华氏度）")
condition: str = Field(description="天气状况")

model = init_chat_model("gpt-4o")
structured_model = model.with_structured_output(WeatherInfo)

response = structured_model.invoke("What's the weather in San Francisco?")
print(response)
# WeatherInfo(location='San Francisco', temperature=72.0, condition='Sun
```

高级主题

模型配置文件

模型配置文件提供有关模型功能和能力的信息。您可以通过 `.profile` 属性访问这些信息：

```
model = init_chat_model("gpt-4o")
print(model.profile)
# 显示模型支持的功能，如工具调用、结构化输出等
```

多模态

许多现代模型支持多模态输入，可以处理图像、音频和视频。LangChain 通过消息内容块支持此功能：

```
from langchain_core.messages import HumanMessage
from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-4o")

message = HumanMessage(
```

```
        content=[
            {"type": "text", "text": "这张图片中有什么?"},
            {
                "type": "image_url",
                "image_url": {"url": "https://example.com/image.jpg"}
            }
        ]
    )

    response = model.invoke(message)
```

推理

某些模型支持多步推理，允许模型在执行复杂任务时进行更深入的思考：

```
model = init_chat_model("o1-preview") # OpenAI 的推理模型

response = model.invoke("解决这个数学问题：如果一个圆的半径是 5，它的面积是多少？")
print(response.content)
```

本地模型

LangChain 支持运行本地模型，这对于隐私或成本考虑很有用：

```
from langchain.chat_models import init_chat_model

# 使用本地模型（需要相应的本地模型服务器）
model = init_chat_model(
    "local:llama2",
    base_url="http://localhost:8000/v1"
)
```

提示缓存

某些提供商支持提示缓存，可以降低重复请求的延迟和成本：

```
from langchain_core.messages import SystemMessage
from langchain.chat_models import init_chat_model

model = init_chat_model("claude-sonnet-4-5-20250929")

# 使用缓存控制的系统消息
system_message = SystemMessage(
    content="You are a helpful assistant.",
    additional_kwargs={"cache_control": {"type": "ephemeral"}}
)

response = model.invoke([
    system_message,
    {"role": "user", "content": "Hello!"}
])
```

服务器端工具使用

某些提供商支持服务器端工具执行，允许模型在服务器端直接调用工具：

```
# 这取决于提供商的具体实现
# 某些模型可能支持服务器端工具执行
```

速率限制

您可以通过配置参数控制请求速率：

```
model = init_chat_model(
    "gpt-4o",
    max_retries=3,
    timeout=30
)
```

基础 URL 或代理

对于本地部署或需要通过代理访问的模型，您可以指定基础 URL：

```
model = init_chat_model(
    "gpt-4o",
    base_url="https://api.openai.com/v1", # 或您的代理 URL
)
```

对数概率

某些模型支持返回对数概率，这对于分析模型的不确定性很有用：

```
model = init_chat_model("gpt-4o", logprobs=True)

response = model.invoke("Why do parrots talk?")
print(response.response_metadata["logprobs"])
```

令牌使用

许多模型提供商会返回令牌使用信息作为调用响应的一部分。当可用时，此信息将包含在相应模型生成的 `AIMessage` 对象上。有关更多详细信息，请参阅[消息指南](#)。

某些提供商 API，特别是 OpenAI 和 Azure OpenAI 聊天完成，要求用户选择在流式上下文中接收令牌使用数据。有关详细信息，请参阅集成指南的流式使用元数据部分。

您可以使用回调或上下文管理器跟踪应用程序中跨模型的聚合令牌计数，如下所示：

回调处理器：

```
from langchain.chat_models import init_chat_model
from langchain_core.callbacks import UsageMetadataCallbackHandler

model_1 = init_chat_model(model="gpt-4o-mini")
model_2 = init_chat_model(model="claude-haiku-4-5-20251001")

callback = UsageMetadataCallbackHandler()
```



```
result_1 = model_1.invoke("Hello", config={"callbacks": [callback]})
result_2 = model_2.invoke("Hello", config={"callbacks": [callback]})
print(callback.usage_metadata)
```

上下文管理器：

```
from langchain.chat_models import init_chat_model
from langchain_core.callbacks import get_usage_metadata_callback

model_1 = init_chat_model(model="gpt-4o-mini")
model_2 = init_chat_model(model="claude-haiku-4-5-20251001")

with get_usage_metadata_callback() as cb:
    model_1.invoke("Hello")
    model_2.invoke("Hello")
    print(cb.usage_metadata)
```

调用配置

调用模型时，您可以使用 `RunnableConfig` 字典通过 `config` 参数传递其他配置。这提供了对执行行为、回调和元数据跟踪的运行时控制。常见的配置选项包括：

使用配置调用：

```
response = model.invoke(
    "Tell me a joke",
    config={
        "run_name": "joke_generation",      # 此运行的自定义名称
        "tags": ["humor", "demo"],         # 用于分类的标签
        "metadata": {"user_id": "123"},     # 自定义元数据
        "callbacks": [my_callback_handler], # 回调处理器
    }
)
```

这些配置值在以下情况下特别有用：

- * 使用 LangSmith 跟踪进行调试
- * 实现自定义日志记录或监控
- * 在生产中控制资源使用
- * 跨复杂管道跟踪调用

关键配置属性：

- `run_name` (string) – 在日志和跟踪中标识此特定调用。不被子调用继承。
- `tags` (string[]) – 由所有子调用继承的标签，用于在调试工具中进行过滤和组织。
- `metadata` (object) – 用于跟踪其他上下文的自定义键值对，由所有子调用继承。
- `max_concurrency` (number) – 在使用 `batch()` 或 `batch_as_completed()` 时控制最大并行调用数。
- `callbacks` (array) – 用于在执行期间监控和响应事件的处理器。
- `recursion_limit` (number) – 链的最大递归深度，以防止复杂管道中的无限循环。

有关所有支持的属性，请参阅完整的 `RunnableConfig` 参考。

可配置模型

您还可以通过指定 `configurable_fields` 创建运行时可配置的模型。如果您不指定模型值，则默认情况下 `'model'` 和 `'model_provider'` 将是可配置的。

```
from langchain.chat_models import init_chat_model

configurable_model = init_chat_model(temperature=0)

configurable_model.invoke(
    "what's your name",
    config={"configurable": {"model": "gpt-5-nano"}}, # 使用 GPT-5-Nano 模型
)
configurable_model.invoke(
    "what's your name",
    config={"configurable": {"model": "claude-sonnet-4-5-20250929"}}, # 使用 Claude Sonnet 4.5 模型
)
```

具有默认值的可配置模型：

我们可以创建一个具有默认模型值的可配置模型，指定哪些参数是可配置的，并为可配置参数添加前缀：

```
first_model = init_chat_model(
    model="gpt-4.1-mini",
```

```

    temperature=0,
    configurable_fields=("model", "model_provider", "temperature", "max_
    config_prefix="first",  # 当您有一个包含多个模型的链时很有用
)

first_model.invoke("what's your name")

```

```

first_model.invoke(
    "what's your name",
    config={
        "configurable": {
            "first_model": "claude-sonnet-4-5-20250929",
            "first_temperature": 0.5,
            "first_max_tokens": 100,
        }
    },
)

```

有关 `configurable_fields` 和 `config_prefix` 的更多详细信息，请参阅 `init_chat_model` 参考。

声明式使用可配置模型：

我们可以在可配置模型上调用声明式操作，如 `bind_tools`、`with_structured_output`、`with_configurable` 等，并以与常规实例化的聊天模型对象相同的方式链接可配置模型：

```

from pydantic import BaseModel, Field

class GetWeather(BaseModel):
    """获取指定位置的当前天气"""
    location: str = Field(..., description="城市和州，例如 San Francisco, CA")

class GetPopulation(BaseModel):
    """获取指定位置的当前人口"""
    location: str = Field(..., description="城市和州，例如 San Francisco, CA")

```

```
model = init_chat_model(temperature=0)
model_with_tools = model.bind_tools([GetWeather, GetPopulation])

model_with_tools.invoke(
    "what's bigger in 2024 LA or NYC",
    config={"configurable": {"model": "gpt-4.1-mini"}}
).tool_calls
```

本文档由 LangChain 官方文档翻译而来

\newpage

8. 工具 (Tools)

原文链接: <https://docs.langchain.com/oss/python/langchain/tools>

Tools 扩展了智能体 (Agents) 的能力——允许它们获取实时数据、执行代码、查询外部数据库，以及触发现实世界中的操作。本质上，工具就是带有明确输入/输出 schema 的可调用函数，这些 schema 会传递给聊天模型。模型再根据会话上下文，决定何时调用工具，以及用什么参数调用工具。

要了解模型在底层如何处理工具调用，请参考文档中的 **Tool calling** 小节。

创建工具 (Create tools)

基本工具定义 (Basic tool definition)

创建工具最简单的方式是使用 `@tool` 装饰器。默认情况下，函数的 docstring 会成为工具的描述，帮助模型理解在什么场景下应当使用该工具：

```
from langchain.tools import tool

@tool
def search_database(query: str, limit: int = 10) -> str:
```

```
"""在客户数据库中根据查询条件检索记录。
```

```
Args:
```

```
    query: 要搜索的关键词
```

```
    limit: 返回结果的最大数量
```

```
"""
```

```
return f"Found {limit} results for '{query}'"
```

类型注解是必需的，因为它们用来定义工具输入的 schema。docstring 应当简洁明确，以便模型正确理解工具的用途和使用方式。

部分聊天模型（例如 OpenAI、Anthropic、Gemini 等）还提供了一些**内置服务器端工具**（如网页搜索、代码执行环境等），这些工具在提供商服务器上执行。你可以在对应提供商的集成文档中查看如何启用和使用这些工具。

自定义工具属性 (Customize tool properties)

自定义工具名称 (Custom tool name)

默认情况下，工具名来自函数名。如果你希望给模型一个更清晰或更稳定的名称，可以显式指定：

```
from langchain.tools import tool

@tool("web_search") # 自定义工具名称
def search(query: str) -> str:
    """在网络上搜索信息。"""
    return f"Results for: {query}"

print(search.name) # web_search
```

自定义工具描述 (Custom tool description)

你也可以覆盖自动生成的描述，以便更精细地引导模型：

```
from langchain.tools import tool

@tool(
    "calculator",
    description="执行四则运算和简单数学计算。遇到任何数学相关问题都应优先使用此工具。"
)
def calc(expression: str) -> str:
    """计算给定表达式的结果。"""
    return str(eval(expression))
```

高级 Schema 定义 (Advanced schema definition)

对于较复杂的输入，你可以使用 Pydantic / BaseModel 来定义参数结构：

```
from pydantic import BaseModel, Field
from typing import Literal
from langchain.tools import tool

class WeatherInput(BaseModel):
    """用于天气查询的输入结构。"""
    location: str = Field(description="城市名称或坐标")
    units: Literal["celsius", "fahrenheit"] = Field(
        default="celsius",
        description="温度单位偏好",
    )
    include_forecast: bool = Field(
        default=False,
        description="是否包含未来 5 天的天气预报",
    )

@tool(args_schema=WeatherInput)
def get_weather(
    location: str,
    units: str = "celsius",
```

```
        include_forecast: bool = False,
    ) -> str:
        """获取当前天气以及可选的未来预报。"""
        temp = 22 if units == "celsius" else 72
        result = f"Current weather in {location}: {temp} degrees {units[0].upper()}"
        if include_forecast:
            result += "\nNext 5 days: Sunny"
        return result
```

通过这种方式，模型可以看到更丰富的参数描述和约束，从而更好地构造调用参数。

保留参数名 (Reserved argument names)

以下参数名被 LangChain 内部保留，**不能**作为工具函数的参数名使用，否则会在运行时报错：

- `config`：内部用于传递 `RunnableConfig`
- `runtime`：用于注入 `ToolRuntime`，让工具访问状态、上下文、store 等

如需访问运行时信息，应通过 `ToolRuntime` 参数，而不是自己使用这些保留名称。

访问上下文 (Accessing context)

如果工具能够访问智能体的状态、运行时上下文和长期记忆，它就能做出更具上下文感知的决策、生成更个性化的回复，并在多轮对话间“记住”信息。

`ToolRuntime` 提供了一种在工具执行时注入依赖（如数据库连接、用户 ID、配置对象等）的方式，使工具更加易于测试和复用。

`ToolRuntime` 中常用的能力包括：

- `state`：执行过程中的可变状态（如消息列表、计数器、自定义字段等）
- `context`：不可变的配置（如用户 ID、会话信息、应用级配置）
- `store`：跨会话的持久存储（长期记忆）
- `stream_writer`：在工具执行中向外流式输出进度信息或中间结果
- `config`：本次调用使用的 `RunnableConfig`
- `tool_call_id`：当前工具调用的唯一标识

使用 ToolRuntime 访问状态

在工具签名中加入 `runtime: ToolRuntime` 参数即可访问所有运行时信息；该参数对模型不可见，不会出现在工具 schema 里。

在工具内部访问消息状态

```
from langchain.tools import tool, ToolRuntime

@tool
def summarize_conversation(runtime: ToolRuntime) -> str:
    """总结当前对话的整体情况。"""
    messages = runtime.state["messages"]

    human_msgs = sum(1 for m in messages if m.__class__.__name__ == "HumanMessage")
    ai_msgs = sum(1 for m in messages if m.__class__.__name__ == "AIMessage")
    tool_msgs = sum(1 for m in messages if m.__class__.__name__ == "ToolMessage")

    return (
        f"对话中共有 {human_msgs} 条用户消息, "
        f"{ai_msgs} 条 AI 响应, {tool_msgs} 条工具消息。"
    )
```

访问自定义状态字段

```
from langchain.tools import tool, ToolRuntime

@tool
def get_user_preference(pref_name: str, runtime: ToolRuntime) -> str:
    """获取用户在对话状态中存储的偏好设置。"""
    preferences = runtime.state.get("user_preferences", {})
    return preferences.get(pref_name, "Not set")
```

内存 / Store（持久存储）

通过 `runtime.store` 可以在对话之间访问和更新长期数据（如用户档案、历史记录等）。

```
from typing import Any
from langgraph.store.memory import InMemoryStore
from langchain.tools import tool, ToolRuntime

@tool
def get_user_info(user_id: str, runtime: ToolRuntime) -> str:
    """根据用户 ID 查询用户信息。"""
    store = runtime.store
    user_info = store.get(("users",), user_id)
    return str(user_info.value) if user_info else "Unknown user"

@tool
def save_user_info(user_id: str, user_info: dict[str, Any], runtime: ToolRuntime) -> str:
    """保存或更新用户信息。"""
    store = runtime.store
    store.put(("users",), user_id, user_info)
    return "Successfully saved user info."
```

在生产环境中，你通常会使用基于数据库或外部存储的 store 实现，而不仅仅是 `InMemoryStore`。

流写器（Stream writer）

通过 `runtime.stream_writer`，工具可以在执行过程中向外流式发送自定义更新信息，用于实时向用户展示进度。这必须在 LangGraph 的执行上下文中使用。

```
from langchain.tools import tool, ToolRuntime

@tool
def get_weather(city: str, runtime: ToolRuntime) -> str:
    """获取指定城市的天气。"""
    writer = runtime.stream_writer

    # 在执行过程中流式输出进度信息
    writer(f"正在查询城市 {city} 的天气数据.....")
    writer(f"已成功获取城市 {city} 的天气数据。")

    return f"It's always sunny in {city}!"
```

本文档由 LangChain 官方文档翻译而来，原文见 [Tools](#)。

\newpage

9. 短期记忆 (Short-term memory)

原文链接: <https://docs.langchain.com/oss/python/langchain/short-term-memory>

概述

记忆是一个记住先前交互信息的系统。对于 AI 智能体来说，记忆至关重要，因为它让智能体能够记住先前的交互、从反馈中学习并适应用户偏好。随着智能体处理更复杂的任务和大量用户交互，这种能力对于效率和用户满意度都变得至关重要。短期记忆让您的应用程序能够在单个线程或对话中记住先前的交互。

线程在会话中组织多个交互，类似于电子邮件将消息分组到单个对话中的方式。

对话历史是短期记忆最常见的形式。长对话对当今的 LLM 构成了挑战；完整的历史可能无法放入 LLM 的上下文窗口，导致上下文丢失或错误。即使您的模型支持完整的上下文长度，大多数 LLM 在长上下文上的表现仍然很差。它们会被过时或偏离主题的内容"分散

注意力", 同时还会遭受更慢的响应时间和更高的成本。聊天模型使用消息接受上下文, 包括指令 (系统消息) 和输入 (人类消息)。在聊天应用程序中, 消息在人类输入和模型响应之间交替, 导致消息列表随时间增长。由于上下文窗口有限, 许多应用程序可以通过使用技术来删除或"忘记"过时的信息而受益。

用法

要为智能体添加短期记忆 (线程级持久化), 您需要在创建智能体时指定一个 `checkpointer`。

LangChain 的智能体将短期记忆作为智能体状态的一部分进行管理。通过将这些存储在图的状态中, 智能体可以访问给定对话的完整上下文, 同时保持不同线程之间的分离。状态使用检查点保存到数据库 (或内存) 中, 以便可以随时恢复线程。短期记忆在智能体被调用或步骤 (如工具调用) 完成时更新, 状态在每个步骤开始时读取。

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    "gpt-5",
    tools=[get_user_info],
    checkpointer=InMemorySaver(), # 使用内存检查点保存对话状态
)

agent.invoke(
    {"messages": [{"role": "user", "content": "Hi! My name is Bob."}]},
    {"configurable": {"thread_id": "1"}}, # 使用 thread_id 标识对话线程
)
```

生产环境

在生产环境中, 使用由数据库支持的检查点:

```
pip install langgraph-checkpoint-postgres
```

```

from langchain.agents import create_agent
from langgraph.checkpoint.postgres import PostgresSaver

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?sslmode=
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup() # 在 PostgreSQL 中自动创建表
    agent = create_agent(
        "gpt-5",
        tools=[get_user_info],
        checkpointer=checkpointer, # 使用 PostgreSQL 检查点
    )

```

自定义智能体记忆

默认情况下，智能体使用 `AgentState` 来管理短期记忆，特别是通过 `messages` 键管理对话历史。您可以扩展 `AgentState` 以添加其他字段。自定义状态模式通过 `state_schema` 参数传递给 `create_agent`。

```

from langchain.agents import create_agent, AgentState
from langgraph.checkpoint.memory import InMemorySaver

class CustomAgentState(AgentState): # 扩展 AgentState 添加自定义字段
    user_id: str
    preferences: dict

agent = create_agent(
    "gpt-5",
    tools=[get_user_info],
    state_schema=CustomAgentState, # 使用自定义状态模式
    checkpointer=InMemorySaver(),
)

# 可以在 invoke 中传递自定义状态
result = agent.invoke(
    {

```

```
        "messages": [{"role": "user", "content": "Hello"}],
        "user_id": "user_123",  # 自定义状态字段
        "preferences": {"theme": "dark"}  # 自定义状态字段
    },
    {"configurable": {"thread_id": "1"}}
)
```

常见模式

启用短期记忆后，长对话可能会超过 LLM 的上下文窗口。常见的解决方案包括：

- **修剪消息** – 删除前 N 条或后 N 条消息（在调用 LLM 之前）
- **删除消息** – 从 LangGraph 状态中永久删除消息
- **总结消息** – 总结历史中的早期消息并用摘要替换它们
- **自定义策略** – 自定义策略（例如消息过滤等）

这允许智能体跟踪对话而不会超过 LLM 的上下文窗口。

修剪消息

大多数 LLM 都有最大支持的上下文窗口（以令牌为单位）。决定何时截断消息的一种方法是计算消息历史中的令牌数，并在接近该限制时截断。如果您使用 LangChain，可以使用修剪消息实用程序并指定要从列表中保留的令牌数，以及用于处理边界的

`strategy`（例如，保留最后 `max_tokens`）。要在智能体中修剪消息历史，请使用 `@before_model` 中间件装饰器：

```
from langchain.messages import RemoveMessage
from langgraph.graph.message import REMOVE_ALL_MESSAGES
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import before_model
from langgraph.runtime import Runtime
from langchain_core.runnables import RunnableConfig
from typing import Any

@before_model
```

```

def trim_messages(state: AgentState, runtime: Runtime) -> dict[str, Any]:
    """仅保留最后几条消息以适合上下文窗口。"""
    messages = state["messages"]

    if len(messages) <= 3:
        return None # 无需更改

    first_msg = messages[0]
    recent_messages = messages[-3:] if len(messages) % 2 == 0 else messages[-2:]
    new_messages = [first_msg] + recent_messages

    return {
        "messages": [
            RemoveMessage(id=REMOVE_ALL_MESSAGES),
            *new_messages
        ]
    }

```

```

agent = create_agent(
    your_model_here,
    tools=your_tools_here,
    middleware=[trim_messages],
    checkpointinter=InMemorySaver(),
)

```

```

config: RunnableConfig = {"configurable": {"thread_id": "1"}}

```

```

agent.invoke({"messages": "hi, my name is bob"}, config)
agent.invoke({"messages": "write a short poem about cats"}, config)
agent.invoke({"messages": "now do the same but for dogs"}, config)
final_response = agent.invoke({"messages": "what's my name?"}, config)

```

```

final_response["messages"][-1].pretty_print()
"""

```

```

===== Ai Message =====

```

```

Your name is Bob. You told me that earlier.

```

```
If you'd like me to call you a nickname or use a different name, just say  
"""
```

删除消息

您可以从图状态中删除消息以管理消息历史。当您想要删除特定消息或清除整个消息历史时，这很有用。要从图状态中删除消息，您可以使用 `RemoveMessage`。要使 `RemoveMessage` 工作，您需要使用带有 `add_messages` reducer 的状态键。默认的 `AgentState` 提供了这一点。要删除特定消息：

```
from langchain.messages import RemoveMessage

def delete_messages(state):
    messages = state["messages"]
    if len(messages) > 2:
        # 删除除最后两条消息外的所有消息
        messages_to_remove = messages[:-2]
        return {
            "messages": [RemoveMessage(id=msg.id) for msg in messages_to_remove]
        }
    return None
```

总结消息

总结消息是管理长对话历史的另一种方法。您可以使用 `SummarizationMiddleware` 来自动总结早期消息并用摘要替换它们：

```
from langchain.agents.middleware import SummarizationMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent

agent = create_agent(
    "gpt-5-nano",
    tools=[],
    middleware=[
```

```

        SummarizationMiddleware(
            summarizer_model="gpt-5-nano",
            max_tokens=1000, # 保留最后 1000 个令牌的消息
        )
    ],
    checkpointer=InMemorySaver(),
)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}
agent.invoke({"messages": "hi, my name is bob"}, config)
agent.invoke({"messages": "write a short poem about cats"}, config)
agent.invoke({"messages": "now do the same but for dogs"}, config)
final_response = agent.invoke({"messages": "what's my name?"}, config)

final_response["messages"][-1].pretty_print()
"""
===== Ai Message =====

Your name is Bob!
"""

```

有关更多配置选项，请参阅 `SummarizationMiddleware`。

访问记忆

您可以通过多种方式访问和修改智能体的短期记忆（状态）：

工具

在工具中读取短期记忆

使用 `runtime` 参数（类型为 `ToolRuntime`）在工具中访问短期记忆（状态）。
`runtime` 参数在工具签名中隐藏（因此模型看不到它），但工具可以通过它访问状态。


```

from langchain.agents import create_agent, AgentState
from langchain.tools import tool, ToolRuntime

class CustomState(AgentState):
    user_id: str

@tool
def get_user_info(
    runtime: ToolRuntime
) -> str:
    """查找用户信息。"""
    user_id = runtime.state["user_id"]
    return "User is John Smith" if user_id == "user_123" else "Unknown u

agent = create_agent(
    model="gpt-5-nano",
    tools=[get_user_info],
    state_schema=CustomState,
)

result = agent.invoke({
    "messages": "look up user information",
    "user_id": "user_123"
})
print(result["messages"][-1].content)
# > User is John Smith.

```

从工具写入短期记忆

要在执行期间修改智能体的短期记忆（状态），您可以直接从工具返回状态更新。这对于持久化中间结果或使信息可供后续工具或提示访问很有用。

```

from langchain.tools import tool, ToolRuntime
from langchain_core.runnables import RunnableConfig
from langchain.messages import ToolMessage
from langchain.agents import create_agent, AgentState

```

```

from langgraph.types import Command
from pydantic import BaseModel

class CustomState(AgentState): # 自定义状态模式
    user_name: str

class CustomContext(BaseModel):
    user_id: str

@tool
def update_user_info(
    runtime: ToolRuntime[CustomContext, CustomState],
) -> Command:
    """查找并更新用户信息。"""
    user_id = runtime.context.user_id
    name = "John Smith" if user_id == "user_123" else "Unknown user"
    return Command(update={ # 返回 Command 以更新状态
        "user_name": name,
        # 更新消息历史
        "messages": [
            ToolMessage(
                "Successfully looked up user information",
                tool_call_id=runtime.tool_call_id
            )
        ]
    })

@tool
def greet(
    runtime: ToolRuntime[CustomContext, CustomState]
) -> str | Command:
    """一旦找到用户信息，使用此工具问候用户。"""
    user_name = runtime.state.get("user_name", None)
    if user_name is None:
        return Command(update={
            "messages": [
                ToolMessage(

```

```

        "Please call the 'update_user_info' tool it will get
        tool_call_id=runtime.tool_call_id
    )
]
})
return f"Hello {user_name}!"

agent = create_agent(
    model="gpt-5-nano",
    tools=[update_user_info, greet],
    state_schema=CustomState, # 使用自定义状态模式
    context_schema=CustomContext,
)

agent.invoke(
    {"messages": [{"role": "user", "content": "greet the user"}]},
    context=CustomContext(user_id="user_123"),
)

```

提示

在中间件中访问短期记忆（状态）以基于对话历史或自定义状态字段创建动态提示。

```

from langchain.agents import create_agent
from typing import TypedDict
from langchain.agents.middleware import dynamic_prompt, ModelRequest

class CustomContext(TypedDict):
    user_name: str

def get_weather(city: str) -> str:
    """获取城市的天气。"""
    return f"The weather in {city} is always sunny!"

@dynamic_prompt
def dynamic_system_prompt(request: ModelRequest) -> str:

```

```

"""根据上下文动态生成系统提示。"""
user_name = request.runtime.context["user_name"]
system_prompt = f"You are a helpful assistant. Address the user as {"
return system_prompt

agent = create_agent(
    model="gpt-5-nano",
    tools=[get_weather],
    middleware=[dynamic_system_prompt],
    context_schema=CustomContext,
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What is the weather in SF"}],
    context=CustomContext(user_name="John Smith"),
)
for msg in result["messages"]:
    msg.pretty_print()

```

输出:

```

===== Human Message =====

What is the weather in SF?

===== Ai Message =====

Tool Calls:
  get_weather (call_WFQl0Gn4b2yoJrv7cih342FG)
Call ID: call_WFQl0Gn4b2yoJrv7cih342FG
Args:
  city: San Francisco

===== Tool Message =====

Name: get_weather

The weather in San Francisco is always sunny!

===== Ai Message =====

```

Hi John Smith, the weather in San Francisco is always sunny!

Before model

在 `@before_model` 中间件中访问短期记忆（状态）以在处理模型调用之前处理消息。

```
from langchain.messages import RemoveMessage
from langgraph.graph.message import REMOVE_ALL_MESSAGES
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import before_model
from langchain_core.runnables import RunnableConfig
from langgraph.runtime import Runtime
from typing import Any

@before_model
def trim_messages(state: AgentState, runtime: Runtime) -> dict[str, Any]:
    """仅保留最后几条消息以适合上下文窗口。"""
    messages = state["messages"]

    if len(messages) <= 3:
        return None # 无需更改

    first_msg = messages[0]
    recent_messages = messages[-3:] if len(messages) % 2 == 0 else messages[-2:]
    new_messages = [first_msg] + recent_messages

    return {
        "messages": [
            RemoveMessage(id=REMOVE_ALL_MESSAGES),
            *new_messages
        ]
    }
```

```

agent = create_agent(
    "gpt-5-nano",
    tools=[],
    middleware=[trim_messages],
    checkpointer=InMemorySaver()
)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}

agent.invoke({"messages": "hi, my name is bob"}, config)
agent.invoke({"messages": "write a short poem about cats"}, config)
agent.invoke({"messages": "now do the same but for dogs"}, config)
final_response = agent.invoke({"messages": "what's my name?"}, config)

final_response["messages"][-1].pretty_print()
"""
===== Ai Message =====

Your name is Bob. You told me that earlier.
If you'd like me to call you a nickname or use a different name, just say so.
"""

```

After model

在 `@after_model` 中间件中访问短期记忆（状态）以在处理模型调用之后处理消息。

```

from langchain.messages import RemoveMessage
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import after_model
from langgraph.runtime import Runtime

@after_model
def validate_response(state: AgentState, runtime: Runtime) -> dict | None:
    """删除包含敏感词的消息。"""
    STOP_WORDS = ["password", "secret"]

```

```
last_message = state["messages"][-1]
if any(word in last_message.content for word in STOP_WORDS):
    return {"messages": [RemoveMessage(id=last_message.id)]}
return None

agent = create_agent(
    model="gpt-5-nano",
    tools=[],
    middleware=[validate_response],
    checkpoint=InMemorySaver(),
)
```

本文档由 LangChain 官方文档翻译而来

\newpage

10. 流式处理概述（Streaming Overview）

原文链接: <https://docs.langchain.com/oss/python/langchain/streaming/overview>

实时流式传输智能体运行的更新

LangChain 实现了一个流式处理系统来展示实时更新。流式处理对于增强基于 LLM 构建的应用程序的响应性至关重要。通过逐步显示输出，甚至在完整响应准备好之前，流式处理显著改善了用户体验（UX），特别是在处理 LLM 的延迟时。

概述

LangChain 的流式处理系统让您可以将智能体运行的实时反馈展示到您的应用程序中。LangChain 流式处理可以实现的功能：

- * **流式传输智能体进度** — 在每个智能体步骤后获取状态更新。
- * **流式传输 LLM 令牌** — 在生成时流式传输语言模型令牌。
- * **流式传输自定义更新** — 发出用户定义的信号（例如，"已获取 10/100 条记录"）。
- * **流式传输多种模式** — 从 `updates`（智能体进度）、`messages`（LLM 令牌 + 元数据）或 `custom`（任意用户数据）中选择。

请参阅下面的常见模式部分以获取其他端到端示例。

支持的流模式

将一个或多个以下流模式作为列表传递给 `stream` 或 `astream` 方法：

模式	描述
updates	在每个智能体步骤后流式传输状态更新。如果在同一步骤中进行了多个更新（例如，运行了多个节点），这些更新会分别流式传输。
messages	从调用 LLM 的任何图节点流式传输（令牌，元数据）元组。
custom	使用流写入器从图节点内部流式传输自定义数据。

智能体进度

要流式传输智能体进度，请使用 `stream_mode="updates"` 的 `stream` 或 `astream` 方法。这会在每个智能体步骤后发出一个事件。例如，如果您有一个调用一次工具的智能体，您应该看到以下更新：

- * **LLM 节点**：带有工具调用请求的 `AIMessage`
- * **工具节点**：带有执行结果的 `ToolMessage`
- * **LLM 节点**：最终的 AI 响应

流式传输智能体进度：

```
from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """获取指定城市的天气。"""
    return f"It's always sunny in {city}!"

agent = create_agent(
    model="gpt-5-nano",
    tools=[get_weather],
)

for chunk in agent.stream( # 流式传输智能体运行
```



```

{"messages": [{"role": "user", "content": "What is the weather in SF"}],
stream_mode="updates",
):
    for step, data in chunk.items():
        print(f"step: {step}")
        print(f"content: {data['messages'][-1].content_blocks}")

```

输出:

```

step: model
content: [{'type': 'tool_call', 'name': 'get_weather', 'args': {'city': 'San Francisco'}}]

step: tools
content: [{'type': 'text', 'text': "It's always sunny in San Francisco!"}]

step: model
content: [{'type': 'text', 'text': "It's always sunny in San Francisco!"}]

```

LLM 令牌

要在 LLM 生成令牌时流式传输令牌，请使用 `stream_mode="messages"`。下面您可以看到智能体流式传输工具调用和最终响应的输出。

流式传输 LLM 令牌:

```

from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """获取指定城市的天气。"""
    return f"It's always sunny in {city}!"

agent = create_agent(
    model="gpt-5-nano",
    tools=[get_weather],
)

```

```
for token, metadata in agent.stream( # 流式传输令牌和元数据
    {"messages": [{"role": "user", "content": "What is the weather in SF"}],
    stream_mode="messages",
):
    print(f"node: {metadata['langgraph_node']}")
    print(f"content: {token.content_blocks}")
    print("\n")
```

输出：

node: model	content: [{ 'type': 'tool_call_chunk', 'id': 'call_vbCyBcP8VuneUzyYlSBZZs'
node: model	content: [{ 'type': 'tool_call_chunk', 'id': None, 'name': None, 'args':
node: model	content: [{ 'type': 'tool_call_chunk', 'id': None, 'name': None, 'args':
node: model	content: [{ 'type': 'tool_call_chunk', 'id': None, 'name': None, 'args':
node: model	content: [{ 'type': 'tool_call_chunk', 'id': None, 'name': None, 'args':
node: model	content: [{ 'type': 'tool_call_chunk', 'id': None, 'name': None, 'args':
node: model	content: []
node: tools	

```
content: [{ 'type': 'text', 'text': "It's always sunny in San Francisco!" }]
```

```
node: model
```

```
content: []
```

```
node: model
```

```
content: [{ 'type': 'text', 'text': 'Here' }]
```

```
node: model
```

```
content: [{ 'type': 'text', 'text': "'s' }]
```

```
node: model
```

```
content: [{ 'type': 'text', 'text': ' what' }]
```

```
node: model
```

```
content: [{ 'type': 'text', 'text': ' I' }]
```

```
node: model
```

```
content: [{ 'type': 'text', 'text': ' got' }]
```

```
node: model
```

```
content: [{ 'type': 'text', 'text': ':' }]
```

```
node: model
```

```
content: [{ 'type': 'text', 'text': ' "' }]
```

```
node: model
```

```
content: [{ 'type': 'text', 'text': "It's" }]
```

```
node: model
```

```
content: [{ 'type': 'text', 'text': ' always' }]
```

```
node: model
```

```
content: [{ 'type': 'text', 'text': ' sunny' }]
```

```
node: model
```

```
content: [{ 'type': 'text', 'text': ' in' }]
```

```
node: model
content: [{'type': 'text', 'text': ' San'}]

node: model
content: [{'type': 'text', 'text': ' Francisco'}]

node: model
content: [{'type': 'text', 'text': '!"\n\n'}]
```

自定义更新

要在工具执行时流式传输更新，您可以使用 `get_stream_writer` 。

流式传输自定义更新：

```
from langchain.agents import create_agent
from langchain.tools import tool, ToolRuntime, get_stream_writer

@tool
def process_data(data: str, runtime: ToolRuntime) -> str:
    """处理数据并流式传输进度更新。"""
    writer = get_stream_writer(runtime)

    # 流式传输进度更新
    writer("开始处理数据...")
    writer("已处理 50% 的数据...")
    writer("处理完成！")

    return f"处理后的数据：{data}"

agent = create_agent(
    model="gpt-5-nano",
    tools=[process_data],
)
```

```
for update in agent.stream(
    {"messages": [{"role": "user", "content": "处理这些数据：example"}]},
    stream_mode="custom",
):
    print(update)
```

流式传输多种模式

您可以同时流式传输多种模式。只需将多个模式作为列表传递：

```
for stream_type, stream_mode, data in agent.stream(
    {"messages": [{"role": "user", "content": "What is the weather in SF"}]},
    stream_mode=["updates", "messages", "custom"], # 同时流式传输多种模式
):
    if stream_mode == "updates":
        # 处理智能体进度更新
        print(f"更新：{data}")
    elif stream_mode == "messages":
        # 处理 LLM 令牌
        token, metadata = data
        print(f"令牌：{token.content}")
    elif stream_mode == "custom":
        # 处理自定义更新
        print(f"自定义：{data}")
```

常见模式

流式传输工具调用

当智能体调用工具时，您可以在工具执行期间流式传输进度更新：

```
from langchain.agents import create_agent
from langchain.tools import tool, ToolRuntime
```

```

@tool
def long_running_task(task_name: str, runtime: ToolRuntime) -> str:
    """执行长时间运行的任务并流式传输进度。"""
    from langchain.tools import get_stream_writer

    writer = get_stream_writer(runtime)
    writer(f"开始执行任务: {task_name}")

    # 模拟长时间运行的任务
    import time
    for i in range(5):
        time.sleep(0.5)
        writer(f"进度: {i+1}/5")

    writer(f"任务 {task_name} 完成!")
    return f"任务 {task_name} 已完成"

agent = create_agent(
    model="gpt-5-nano",
    tools=[long_running_task],
)

for update in agent.stream(
    {"messages": [{"role": "user", "content": "执行任务: 数据分析"}]},
    stream_mode="custom",
):
    print(update)

```

访问已完成的消息

在流式传输过程中，您可能想要访问已完成的消息。您可以使用

`stream_mode="updates"` 来获取每个步骤的完整消息：

```
from langchain.agents import create_agent
```

```

agent = create_agent(
    model="gpt-5-nano",
    tools=[get_weather],
)

for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "What is the weather in SF"}],
    stream_mode="updates",
):
    for step, data in chunk.items():
        if step == "model":
            last_message = data["messages"][-1]
            if hasattr(last_message, "tool_calls") and last_message.tool_calls:
                print(f"工具调用: {last_message.tool_calls}")
        elif step == "tools":
            last_message = data["messages"][-1]
            print(f"工具响应: {last_message.content}")

```

流式传输与人在回路

当使用人在回路功能时，您仍然可以流式传输智能体进度。这允许您在等待人工输入时显示实时更新：

```

from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model="gpt-5-nano",
    tools=[get_weather],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "1"}}

for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "What is the weather in SF"}],

```

```

        config=config,
        stream_mode="updates",
    ):
        for step, data in chunk.items():
            print(f"步骤: {step}")
            # 处理更新...

```

从子智能体流式传输

当智能体内部有多个 LLM 时，通常需要消除消息来源的歧义。为此，在创建智能体时为每个智能体传递一个名称。当以 "messages" 模式流式传输时，此名称可通过元数据中的 `lc_agent_name` 键获得。下面，我们更新流式传输工具调用示例：1. 我们将工具替换为在内部调用智能体的 `call_weather_agent` 工具 2. 我们为每个智能体添加一个 `name` 3. 我们在创建流时指定 `subgraphs=True` 4. 我们的流处理与之前相同，但添加了使用 `create_agent` 的 `name` 参数跟踪哪个智能体处于活动状态的逻辑

当您在智能体上设置 `name` 时，该名称也会附加到该智能体生成的任何 `AIMessage`。

首先我们构建智能体：

```

from typing import Any
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain.messages import AIMessage, AnyMessage

def get_weather(city: str) -> str:
    """获取指定城市的天气。"""
    return f"It's always sunny in {city}!"

weather_model = init_chat_model("openai:gpt-5.2")
weather_agent = create_agent(
    model=weather_model,
    tools=[get_weather],
    name="weather_agent", # 为天气智能体设置名称
)

```



```
def call_weather_agent(query: str) -> str:
    """查询天气智能体。"""
    result = weather_agent.invoke({
        "messages": [{"role": "user", "content": query}]
    })
    return result["messages"][-1].text

supervisor_model = init_chat_model("openai:gpt-5.2")
agent = create_agent(
    model=supervisor_model,
    tools=[call_weather_agent],
    name="supervisor", # 为监督智能体设置名称
)
```

接下来，我们向流式传输循环添加逻辑以报告哪个智能体正在发出令牌：

```
def _render_message_chunk(token: AIMessageChunk) -> None:
    """渲染消息块。"""
    if token.text:
        print(token.text, end="|")
    if token.tool_call_chunks:
        print(token.tool_call_chunks)

def _render_completed_message(message: AnyMessage) -> None:
    """渲染已完成的消息。"""
    if isinstance(message, AIMessage) and message.tool_calls:
        print(f"工具调用: {message.tool_calls}")
    if isinstance(message, ToolMessage):
        print(f"工具响应: {message.content_blocks}")

input_message = {"role": "user", "content": "What is the weather in Boston?"}
current_agent = None

for _, stream_mode, data in agent.stream(
    {"messages": [input_message]},
    stream_mode=["messages", "updates"],
```

```

subgraphs=True, # 启用子图流式传输
):
    if stream_mode == "messages":
        token, metadata = data
        if agent_name := metadata.get("lc_agent_name"): # 获取智能体名称
            if agent_name != current_agent: # 如果智能体切换了
                print(f"🤖 {agent_name}: ") # 打印智能体名称
                current_agent = agent_name # 更新当前智能体
            if isinstance(token, AIMessage):
                _render_message_chunk(token)
    if stream_mode == "updates":
        for source, update in data.items():
            if source in ("model", "tools"):
                _render_completed_message(update["messages"][-1])

```

输出:

```

🤖 supervisor:
[{'name': 'call_weather_agent', 'args': '', 'id': 'call_asorzUf0mB6sb7Mi
[{'name': None, 'args': '{}', 'id': None, 'index': 0, 'type': 'tool_call
[{'name': None, 'args': 'query', 'id': None, 'index': 0, 'type': 'tool_c
[{'name': None, 'args': '": "', 'id': None, 'index': 0, 'type': 'tool_cal
[{'name': None, 'args': 'Boston', 'id': None, 'index': 0, 'type': 'tool_
[{'name': None, 'args': ' weather', 'id': None, 'index': 0, 'type': 'too
[{'name': None, 'args': ' right', 'id': None, 'index': 0, 'type': 'tool_
[{'name': None, 'args': ' now', 'id': None, 'index': 0, 'type': 'tool_ca
[{'name': None, 'args': ' and', 'id': None, 'index': 0, 'type': 'tool_ca
[{'name': None, 'args': ' today's', 'id': None, 'index': 0, 'type': 'too
[{'name': None, 'args': ' forecast', 'id': None, 'index': 0, 'type': 'to
[{'name': None, 'args': '{}', 'id': None, 'index': 0, 'type': 'tool_call
Tool calls: [{'name': 'call_weather_agent', 'args': {'query': 'Boston we
🤖 weather_agent:
[{'name': 'get_weather', 'args': '', 'id': 'call_LZ89lT8fW6w8vqck5pZeaDI
[{'name': None, 'args': '{}', 'id': None, 'index': 0, 'type': 'tool_call
[{'name': None, 'args': 'city', 'id': None, 'index': 0, 'type': 'tool_ca
[{'name': None, 'args': '": "', 'id': None, 'index': 0, 'type': 'tool_cal

```

```
[{'name': None, 'args': 'Boston', 'id': None, 'index': 0, 'type': 'tool_
[{'name': None, 'args': '{}', 'id': None, 'index': 0, 'type': 'tool_call
Tool calls: [{'name': 'get_weather', 'args': {'city': 'Boston'}, 'id': '
Tool response: [{'type': 'text', 'text': "It's always sunny in Boston!"}]
Boston| weather| right| now|:| **|Sunny|**.

|Today|'s| forecast| for| Boston|:| **|Sunny| all| day|**.|Tool respons
🤖 supervisor:
Boston| weather| right| now|:| **|Sunny|**.

|Today|'s| forecast| for| Boston|:| **|Sunny| all| day|**.|
```

禁用流式传输

在某些应用程序中，您可能需要禁用给定模型的单个令牌流式传输。这在以下情况下很有用：

- * 使用多智能体系统来控制哪些智能体流式传输其输出
- * 混合支持流式传输和不支持流式传输的模型
- * 部署到 LangSmith 并希望防止某些模型输出被流式传输到客户端

在初始化模型时设置 `streaming=False`。

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-4o",
    streaming=False # 禁用流式传输
)
```

当部署到 LangSmith 时，在您不想流式传输到客户端的任何模型上设置 `streaming=False`。这在部署之前在您的图代码中配置。

并非所有聊天模型集成都支持 `streaming` 参数。如果您的模型不支持它，请改用 `disable_streaming=True`。此参数通过基类在所有聊天模型上可用。

有关更多详细信息，请参阅 LangGraph 流式传输指南。

相关

- [前端流式传输](#) — 使用 `useStream` 构建 React UI 以实现实时智能体交互
 - [使用聊天模型流式传输](#) — 直接从聊天模型流式传输令牌，无需使用智能体或图
 - [使用人在回路流式传输](#) — 在处理人工审查中断时流式传输智能体进度
 - [LangGraph 流式传输](#) — 高级流式传输选项，包括 `values`、`debug` 模式和子图流式传输
-

本文档由 LangChain 官方文档翻译而来

\newpage

11. 前端流式处理 (Streaming Frontend)

原文链接: <https://docs.langchain.com/oss/python/langchain/streaming/frontend>

使用 LangChain 智能体、LangGraph 图和自定义 API 的实时流式能力，构建生成式 UI。

`useStream` React Hook 为 LangGraph 的流式能力提供了无缝集成。它帮你处理流式传输、状态管理和分支逻辑中的所有复杂细节，让你专注于构建优秀的生成式 UI 体验。

核心特性： – **消息流式传输**：处理一串消息分片以组装成完整消息 – **自动状态管理**：自动管理消息、中断 (interrupt)、加载状态和错误 – **对话分支**：从聊天历史中的任意点创建分支对话路径 – **与 UI 解耦的设计**：你可以自带组件和样式

安装 (Installation)

安装 LangGraph SDK 以便在 React 应用中使用 `useStream` Hook。

(参考官方文档中的安装步骤，这里略。)

基本用法 (Basic usage)

`useStream` Hook 可以连接到任意 LangGraph 图： – 可以是你自己本地或自建服务暴露的端点 – 也可以是通过 LangSmith 部署的托管智能体

```
import { useStream } from "@langchain/langgraph-sdk/react";

function Chat() {
  const stream = useStream({
    assistantId: "agent",
    // 本地开发
    apiUrl: "http://localhost:2024",
    // 生产部署 (LangSmith 托管)
    // apiUrl: "https://your-deployment.us.langgraph.app"
  });

  const handleSubmit = (message: string) => {
    stream.submit({
      messages: [
        { content: message, type: "human" }
      ],
    });
  };

  return (
    <div>
      {stream.messages.map((message, idx) => (
        <div key={message.id ?? idx}>
          {message.type}: {message.content}
        </div>
      ))}

      {stream.isLoading && <div>Loading...</div>}
      {stream.error && <div>Error: {stream.error.message}</div>}
    </div>
  );
}
```

```
);  
}
```

了解如何将智能体部署到 LangSmith，以获得可用于生产的托管能力，包括内置可观测性、认证和扩缩容。

useStream 参数

- **assistantId** (string, 必填)
 - 要连接的智能体 ID。
 - 使用 LangSmith 部署时，必须与部署面板中显示的 agent ID 一致。
 - 对于自定义 API 或本地开发，可以是你的服务用来标识智能体的任意字符串。
- **apiUrl** (string)
 - LangGraph 服务器的 URL。
 - 默认值为 `http://localhost:2024`（本地开发）。
- **apiKey** (string)
 - 用于认证的 API Key。
 - 连接到 LangSmith 上已部署的智能体时必需。
- **threadId** (string)
 - 连接到已有的线程，而不是创建新线程。
 - 适合用于恢复历史对话。
- **onThreadId** ((id: string) => void)
 - 当创建新线程时触发的回调。
 - 用于持久化 threadId，以便后续继续使用。
- **reconnectOnMount** (boolean | () => Storage)
 - 在组件挂载时自动恢复进行中的 run。
 - 设为 `true` 时使用 `sessionStorage`，也可以传入自定义的 Storage 函数。

- **onCreated** ((run: Run) => void)
 - 当新 run 被创建时调用。
 - 可用于持久化 run 元数据，便于之后恢复。
- **onError** ((error: Error) => void)
 - 流式过程中发生错误时调用。
- **onFinish** ((state: StateType, run?: Run) => void)
 - 流式完成并获得最终状态时调用。
- **onCustomEvent** ((data: unknown, context: { mutate }) => void)
 - 处理来自智能体通过 `writer` 发出的自定义事件。
 - 详见“自定义流式事件”小节。
- **onUpdateEvent** ((data: unknown, context: { mutate }) => void)
 - 处理每个图步骤后的状态更新事件。
- **onMetadataEvent** ((metadata: { run_id, thread_id }) => void)
 - 处理附带 run 和 thread 信息的元数据事件。
- **messagesKey** (string, 默认 `"messages"`)
 - 图状态中存放消息数组的键名。
- **throttle** (boolean, 默认 `true`)
 - 是否对状态更新进行节流，以提升渲染性能。
 - 如需“立即更新”效果，可以关闭节流。
- **initialValues** (StateType | null)
 - 在首次流式加载前显示的初始状态值。
 - 适合用于立即展示已缓存的线程数据。

useStream 返回值

- **messages** (Message[])
 - 当前线程中的全部消息，包括用户消息和 AI 消息。
- **values** (StateType)
 - 当前图状态的值。
 - 类型会从智能体或图的类型参数中推断。
- **isLoading** (boolean)
 - 是否正在进行流式请求。
 - 可用于展示加载指示器。
- **error** (Error | null)
 - 流式过程中发生的错误；无错误时为 `null`。
- **interrupt** (Interrupt | undefined)
 - 当前需要用户输入的中断（如人在回路审批请求）。
- **toolCalls** (ToolCallWithResult[])
 - 所有消息中的工具调用，包括其结果和状态（`pending`、`completed` 或 `error`）。
- **submit(input, options?) => Promise**
 - 向智能体提交新的输入。
 - 当从 `interrupt` 中继续时，可将 `input` 设为 `null` 并传入相应命令。
 - `options` 可包含：
 - `checkpoint`：用于对话分支
 - `optimisticValues`：用于乐观更新
 - `threadId`：用于乐观创建线程
- **stop() => void**
 - 立即停止当前流式传输。

- `joinStream(runId: string) => void`
 - 通过 run ID 恢复已有的流。
 - 通常与 `onCreated` 联合使用，实现手动流恢复。
 - `setBranch(branch: string) => void`
 - 切换到会话历史中的其他分支。
 - `getToolCalls(message) => ToolCall[]`
 - 获取某个 AI 消息对应的所有工具调用。
 - `getMessagesMetadata(message) => MessageMetadata`
 - 获取某条消息的元数据，比如：
 - `langgraph_node`：标识消息来源节点
 - `firstSeenState`：用于分支管理
 - `experimental_branchTree` (`BranchTree`)
 - 线程的树形表示，用于非消息型图中的高级分支控制。
-

线程管理 (Thread management)

通过内置线程管理功能追踪会话。你可以访问当前线程 ID，并在新线程创建时收到通知：

```
import { useState } from "react";
import { useStream } from "@langchain/langgraph-sdk/react";

function Chat() {
  const [threadId, setThreadId] = useState<string | null>(null);

  const stream = useStream({
    apiUrl: "http://localhost:2024",
    assistantId: "agent",
    threadId: threadId,
```

```
    onThreadId: setThreadId,  
  });  
  
  // 当创建新线程时, threadId 会被更新  
  // 可以将它存入 URL 参数或 localStorage 以便持久化  
}
```

推荐持久化 `threadId` , 以便用户在刷新页面后继续先前对话。

刷新后恢复 (Resume after page refresh)

通过设置 `reconnectOnMount: true` , `useStream` Hook 可以在组件挂载时自动恢复进行中的 run。这在页面刷新后继续流式处理时非常有用, 确保在刷新期间生成的消息和事件不会丢失。

```
const stream = useStream({  
  apiUrl: "http://localhost:2024",  
  assistantId: "agent",  
  reconnectOnMount: true,  
});
```

默认情况下, 创建的 run ID 存储在 `window.sessionStorage` 中。你可以通过传入自定义存储函数来替换:

```
const stream = useStream({  
  apiUrl: "http://localhost:2024",  
  assistantId: "agent",  
  reconnectOnMount: () => window.localStorage,  
});
```

如果你希望手动控制恢复过程, 可以使用 run 回调持久化元数据, 并通过 `joinStream` 恢复:

```
import { useStream } from "@langchain/langgraph-sdk/react";
```

```
function Chat() {
  const stream = useStream({
    apiUrl: "http://localhost:2024",
    assistantId: "agent",
    onCreate: (run) => {
      // 持久化 run.id 以便之后恢复
      window.localStorage.setItem("lastRunId", run.id);
    },
  });

  const resume = () => {
    const runId = window.localStorage.getItem("lastRunId");
    if (runId) {
      stream.joinStream(runId);
    }
  };

  // ... UI 中提供一个按钮调用 resume()
}
```

乐观更新 (Optimistic updates)

`useStream` 支持乐观更新，让 UI 可以在服务器响应之前先行显示预期结果，从而提升交互流畅性。

乐观创建线程 (Optimistic thread creation)

你可以在服务器真正创建线程之前，先在 UI 中展示一个“临时”线程：

```
stream.submit(
  {
    messages: [
      { type: "human", content: "Hello" },
    ],
  },
);
```

```
{
  threadId: "temp-thread-id", // 乐观线程 ID
}
);
```

当服务器返回真实 `threadId` 时，可以通过 `onThreadId` 进行替换和持久化。

缓存线程展示 (Cached thread display)

你可以通过 `initialValues` 将已缓存的线程状态传递给 `useStream`，在首次加载时立即展示旧消息，同时后台启动新的流式请求：

```
const cachedState = loadCachedStateSomehow();

const stream = useStream({
  apiUrl: "http://localhost:2024",
  assistantId: "agent",
  initialValues: cachedState, // 先展示缓存，再等待流式更新
});
```

分支 (Branching)

通过 `setBranch` 和 `experimental_branchTree`，你可以在前端实现类似“对话时间线/树形分支”的体验，从历史中的任意节点开始一个新的对话路径。

(这一部分多与具体 UI 实现相关，参考官方示例进行自定义即可。)

类型安全流式处理 (Type-safe streaming)

你可以结合 TypeScript 的类型系统，为 `useStream` 提供你的状态类型，从而获得端到端的类型安全体验。

搭配 `createAgent` 使用 (With `createAgent`)

如果后端是用 `create_agent` 构建的智能体，并且其状态 `schema` 有确定结构，可以在前端定义对应的 TypeScript 类型，并在 `useStream` 的泛型参数中使用。

搭配 `StateGraph` / 注解类型 (With `StateGraph` / `Annotation types`)

对于 `LangGraph` 的自定义状态，你可以： – 在 Python 端通过 `TypedDict` / `dataclass` / `Annotation` 定义状态 – 在前端定义对应的 TypeScript 类型 – 通过 `useStream<StateType>()` 获得类型安全的 `values` / `messages`

高级类型配置 (Advanced type configuration)

在更复杂的场景（多工具、多节点、多种消息类型）下，可以为消息、工具调用结果、分支树等定义更精细的 TypeScript 类型，以构建强类型的前端交互。

渲染工具调用 (Rendering tool calls)

`useStream` 暴露的 `toolCalls` 和 `getToolCalls` 方法可以帮助你在 UI 中渲染工具调用及其状态（进行中 / 完成 / 错误），例如：

- 在消息列表下方展示“工具调用卡片”
- 为进行中的工具显示 spinner，为完成的工具显示结果摘要

（可参考官方示例中 `tool-calling-agent` 的前端实现。）

自定义流式事件 (Custom streaming events)

你可以在后端通过工具或节点中的 `writer` 发送自定义事件，在前端通过 `onCustomEvent` 进行处理。

后端 `agent.py` 示例：

```

import asyncio
import time
from langchain import create_agent, tool
from langchain.types import ToolRuntime

@tool
async def analyze_data(data_source: str, *, config: ToolRuntime) -> str:
    """带进度更新地分析数据。"""
    steps = ["Connecting...", "Fetching...", "Processing...", "Done!"]

    for i, step in enumerate(steps):
        # 在执行过程中发出进度事件
        if config.writer:
            config.writer({
                "type": "progress",
                "id": f"analysis-{int(time.time()) * 1000}",
                "message": step,
                "progress": ((i + 1) / len(steps)) * 100,
            })
        await asyncio.sleep(0.5)

    return '{"result": "Analysis complete"}'

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[analyze_data],
)

```

前端可以在 `Chat.tsx` 里通过 `onCustomEvent` 解析这些事件，绘制进度条、状态徽章或文件操作卡片等。

事件处理（Event handling）

`useStream` 提供了多种回调选项，用于处理不同类型的流式事件。你无需显式配置 stream modes，只需为需要的事件类型传入对应回调：

可用回调（Available callbacks）

回调名	描述	对应模式
onUpdateEvent	每个图步骤后的状态更新事件	updates
onCustomEvent	来自图的自定义事件（通过 writer 发送）	custom
onMetadataEvent	包含 run 与 thread 元数据的事件	metadata
onError	流式过程中发生错误时调用	—
onFinish	流式完成时调用，携带最终状态	—

多智能体流式处理（Multi-agent streaming）

在多智能体系统或包含多个节点的图中，可以使用消息元数据来区分每条消息的来源节点。这在多个 LLM 并行运行、且你希望以不同样式展示它们的输出时特别有用。

后端可以通过给不同节点 / 智能体命名，并在前端读取消息元数据中的 `langgraph_node` 或 `lc_agent_name` 来区分来源。前端再根据来源为不同智能体使用不同的颜色、头像或布局。

官方文档中的 parallel-research 示例展示了： – 三个并行研究智能体（分析型 / 创意型 / 实用型） – 各自输出以不同风格渲染

人在回路（Human-in-the-loop）

配合 `human_in_the_loop_middleware`，你可以在： – 发送邮件 – 删除文件 – 调用潜在危险工具 等操作前插入人工审批。前端通过 `interrupt` 信息渲染一个审批 UI（例如“同意 / 拒绝 / 编辑”按钮），并通过 `submit` 将决策传回图中。

完整示例可参考官方 human-in-the-loop 示例，其中包括：

- 审批卡片 – 决策按钮 (approve / reject / edit)
- 审批结果回写到对话流中

推理模型 (Reasoning models)

扩展推理 / 思维链的支持目前仍在实验阶段。不同提供商 (OpenAI vs Anthropic) 的“推理 token”流式接口略有不同，未来可能会进一步抽象统一。

对于带扩展推理能力的模型（如 OpenAI 的 o1 系列或 Anthropic 的扩展 thinking 模型），推理过程通常嵌入在消息内容中，你可以在前端将其单独提取并以特殊样式展示，例如：

- 折叠的“思考过程”面板
- 与最终回答分区展示

自定义状态类型 (Custom state types)

对于自定义的 LangGraph 应用，你可以将工具调用类型等信息嵌入到状态的 `messages` 或其他字段中，然后在前端定义对应的 TypeScript 类型，通过 `useStream` 的泛型获得类型安全的访问方式。

自定义传输层 (Custom transport)

对于自定义 API 端点或非标准部署，可以通过 `transport` 选项结合 `FetchStreamTransport` 连接任意支持流式的 API。

相关 (Related)

- [Streaming overview](#) — 服务器端流式处理（智能体端）的详细说明
 - [useStream API Reference](#) — `useStream` 的完整 API 文档
 - [Agent Chat UI](#) — LangGraph 智能体的预构建聊天界面
 - [Human-in-the-loop](#) — 配置人工审查中断的指南
 - [Multi-agent systems](#) — 构建多 LLM 智能体系统的指南
-

12. 结构化输出 (Structured output)

原文链接: <https://docs.langchain.com/oss/python/langchain/structured-output>

结构化输出允许智能体以特定的、可预测的格式返回数据。相比解析自然语言响应，你可以直接得到 JSON 对象、Pydantic 模型或 dataclass 这样的结构化数据，供应用程序直接使用。

LangChain 的 `create_agent` 会自动处理结构化输出： – 用户指定期望的结构化输出 schema； – 当模型生成结构化数据时，LangChain 会捕获并校验； – 校验通过后会放在智能体状态中的 `"structured_response"` 键下返回。

```
def create_agent(
    ...
    response_format: Union[
        ToolStrategy[StructuredResponseT],
        ProviderStrategy[StructuredResponseT],
        type[StructuredResponseT],
        None,
    ]
```

响应格式 (Response format)

使用 `response_format` 来控制智能体如何返回结构化数据：

- `ToolStrategy[StructuredResponseT]`：使用工具调用来实现结构化输出
- `ProviderStrategy[StructuredResponseT]`：使用模型提供商的原生结构化输出能力
- `type[StructuredResponseT]`：仅传入 schema 类型，由 LangChain 根据模型能力自动选择最优策略
- `None`：不显式请求结构化输出

当直接提供 schema 类型时，LangChain 会自动选择：

- 如果所选模型 / 提供商支持原生结构化输出（如 OpenAI、Anthropic (Claude)、xAI (Grok)），则使用 `ProviderStrategy`；
- 否则对其他模型使用 `ToolStrategy`。

在 `langchain>=1.1` 时，是否支持原生结构化输出会从模型的 `profile` 数据中动态读取。如果没有 profile 数据，你可以自定义 profile 或手动指定：

```
custom_profile = {
    "structured_output": True,
    # ...
}
model = init_chat_model("...", profile=custom_profile)
```

如果同时指定了 tools，则模型必须支持“工具 + 结构化输出”同时使用。

最终的结构化结果会放在智能体最终状态中的 `structured_response` 键下。

Provider 策略 (Provider strategy)

某些模型提供商在其 API 中原生支持结构化输出（如 OpenAI、xAI (Grok)、Gemini、Anthropic (Claude)）。在这些场景中，优先使用 provider 原生结构化输出通常更可靠。

使用此策略时，可以配置 `ProviderStrategy`：

```
class ProviderStrategy(Generic[SchemaT]):
    schema: type[SchemaT]
    strict: bool | None = None
```

`strict` 参数需要 `langchain>=1.2`。

- **schema (必填)**
- 定义结构化输出格式的 schema。

- 支持：
 - **Pydantic 模型**： `BaseModel` 子类，支持字段校验。返回验证后的 Pydantic 实例。
 - **dataclass**：带类型注解的 Python dataclass。返回字典。
 - **TypedDict**：类型化字典类。返回字典。
 - **JSON Schema**：JSON schema 形式的字典。返回字典。
- **strict (可选)**
 - 是否启用严格 schema 约束。
 - 一些提供商（如 OpenAI、xAI）支持。
 - 默认为 `None`（不启用）。

当你将 schema 类型直接传给 `create_agent.response_format` 时，如果模型支持原生结构化输出，LangChain 会自动使用 `ProviderStrategy`：

Pydantic 模型示例：

```
from pydantic import BaseModel, Field
from langchain.agents import create_agent

class ContactInfo(BaseModel):
    """人的联系信息。"""
    name: str = Field(description="此人的姓名")
    email: str = Field(description="邮箱地址")
    phone: str = Field(description="电话号码")

agent = create_agent(
    model="gpt-5",
    response_format=ContactInfo,  # 自动选择 ProviderStrategy
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Extract contact info from:"
}
```

```
print(result["structured_response"])
# ContactInfo(name='John Doe', email='john@example.com', phone='(555) 123-4567')
```

当提供商原生支持结构化输出时，写 `response_format=ProductReview` 等价于 `response_format=ProviderStrategy(ProductReview)`。若模型不支持原生结构化输出，智能体则会回退到“工具调用策略（ToolStrategy）”。

工具调用策略（Tool calling strategy）

对于不支持原生结构化输出的模型，LangChain 会使用工具调用来实现结构化输出。这适用于所有支持工具调用的模型（大部分现代模型）。

配置 `ToolStrategy` 示例：

```
class ToolStrategy(Generic[SchemaT]):
    schema: type[SchemaT]
    tool_message_content: str | None
    handle_errors: Union[
        bool,
        str,
        type[Exception],
        tuple[type[Exception], ...],
        Callable[[Exception], str],
    ]
```

- **schema（必填）**
 - 定义结构化输出格式的 schema。
- **支持：**
 - **Pydantic 模型：** `BaseModel` 子类，返回验证后的 Pydantic 实例；
 - **dataclass：** 返回字典；
 - **TypedDict：** 返回字典；
 - **JSON Schema：** 返回字典；
 - **联合类型（Union）：** 多个 schema 备选，模型会根据上下文选择最合适的一个。

- `tool_message_content` (可选)
- 当生成结构化输出时，在对话历史中的工具消息内容。
- 如果不提供，默认会包含显示结构化响应数据的消息。
- `handle_errors` (可选)
- 用于处理结构化输出校验失败的错误策略，默认为 `True`。
- 取值含义：
 - `True`：捕获所有错误，使用默认错误模板进行重试提示；
 - `str`：捕获所有错误，使用此自定义消息作为错误提示；
 - `type[Exception]`：仅捕获这种异常类型，其余异常直接抛出；
 - `tuple[type[Exception], ...]`：仅捕获指定多种异常类型；
 - `Callable[[Exception], str]`：自定义错误处理函数，返回要发送给模型的错误消息；
 - `False`：不做重试，所有异常直接向外抛出。

Pydantic 模型示例：

```
from pydantic import BaseModel, Field
from typing import Literal
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

class ProductReview(BaseModel):
    """对产品评论的分析结果。"""
    rating: int | None = Field(description="产品评分", ge=1, le=5)
    sentiment: Literal["positive", "negative"] = Field(description="评论情感")
    key_points: list[str] = Field(description="评论的关键点。小写、1-3 个词。")

agent = create_agent(
    model="gpt-5",
    tools=tools,
    response_format=ToolStrategy(ProductReview),
)
```

```

result = agent.invoke({
    "messages": [{"role": "user", "content": "Analyze this review: 'Great product, fast shipping.'"}])

result["structured_response"]
# ProductReview(rating=5, sentiment='positive', key_points=['fast shipping'])

```

自定义工具消息内容 (Custom tool message content)

`tool_message_content` 参数允许你自定义在对话历史中显示的工具消息内容，当结构化输出生成时会使用该内容：

```

from pydantic import BaseModel, Field
from typing import Literal
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

class MeetingAction(BaseModel):
    """从会议记录中提取的行动项。"""
    task: str = Field(description="需要完成的具体任务")
    assignee: str = Field(description="负责该任务的人")
    priority: Literal["low", "medium", "high"] = Field(description="优先级")

agent = create_agent(
    model="gpt-5",
    tools=[],
    response_format=ToolStrategy(
        schema=MeetingAction,
        tool_message_content="Action item captured and added to meeting notes"
    ),
)

agent.invoke({

```

```
"messages": [{"role": "user", "content": "From our meeting: Sarah ne  
}]
```

生成的对话片段示例：

```
===== Human Message =====  
  
From our meeting: Sarah needs to update the project timeline as soon as p  
===== Ai Message =====  
Tool Calls:  
  MeetingAction (call_1)  
  Call ID: call_1  
  Args:  
    task: Update the project timeline  
    assignee: Sarah  
    priority: high  
===== Tool Message =====  
Name: MeetingAction  
  
Action item captured and added to meeting notes!
```

如果不设置 `tool_message_content`，最终的 ToolMessage 会是类似：

```
===== Tool Message =====  
Name: MeetingAction  
  
Returning structured response: {'task': 'update the project timeline', 'o
```

错误处理（Error handling）

使用工具调用生成结构化输出时，模型可能会出错。LangChain 提供了智能重试机制来自动处理这些错误。

多个结构化输出错误 (Multiple structured outputs error)

当模型错误地调用了多个结构化输出工具时，智能体会在 ToolMessage 中给出错误反馈，并提示模型重试：

```
from pydantic import BaseModel, Field
from typing import Union
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

class ContactInfo(BaseModel):
    name: str = Field(description="人的姓名")
    email: str = Field(description="邮箱地址")

class EventDetails(BaseModel):
    event_name: str = Field(description="事件名称")
    date: str = Field(description="事件日期")

agent = create_agent(
    model="gpt-5",
    tools=[],
    response_format=ToolStrategy(Union[ContactInfo, EventDetails]), # 默认为 ToolStrategy
)

agent.invoke({
    "messages": [{"role": "user", "content": "Extract info: John Doe (john@example.com) is organizing Tech Conference"}])
```

当模型第一次错误地返回两个结构化输出时，对话片段类似：

```
===== Human Message =====

Extract info: John Doe (john@example.com) is organizing Tech Conference
```


===== Ai Message =====

Tool Calls:

 ContactInfo (call_1)

 Call ID: call_1

 Args:

 name: John Doe

 email: john@example.com

 EventDetails (call_2)

 Call ID: call_2

 Args:

 event_name: Tech Conference

 date: March 15th

===== Tool Message =====

Name: ContactInfo

Error: Model incorrectly returned multiple structured responses (ContactInfo, EventDetails)
Please fix your mistakes.

===== Tool Message =====

Name: EventDetails

Error: Model incorrectly returned multiple structured responses (ContactInfo, EventDetails)
Please fix your mistakes.

===== Ai Message =====

Tool Calls:

 ContactInfo (call_3)

 Call ID: call_3

 Args:

 name: John Doe

 email: john@example.com

===== Tool Message =====

Name: ContactInfo

Returning structured response: {'name': 'John Doe', 'email': 'john@example.com'}

Schema 校验错误 (Schema validation error)

当结构化输出不符合预期 schema 时，智能体会给出详细的错误反馈并重试：

```
from pydantic import BaseModel, Field
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

class ProductRating(BaseModel):
    rating: int | None = Field(description="评分 1-5", ge=1, le=5)
    comment: str = Field(description="评论内容")

agent = create_agent(
    model="gpt-5",
    tools=[],
    response_format=ToolStrategy(ProductRating), # 默认 handle_errors=True
    system_prompt="You are a helpful assistant that parses product reviews"
)

agent.invoke({
    "messages": [{"role": "user", "content": "Parse this: Amazing product"}
})
```

首次错误返回后的对话片段：

```
===== Human Message =====

Parse this: Amazing product, 10/10!
===== Ai Message =====

Tool Calls:
  ProductRating (call_1)
Call ID: call_1
Args:
  rating: 10
```

```
comment: Amazing product

===== Tool Message =====
Name: ProductRating

Error: Failed to parse structured output for tool 'ProductRating': 1 val
Input should be less than or equal to 5 [type=less_than_equal, input_v
Please fix your mistakes.

===== Ai Message =====
Tool Calls:
  ProductRating (call_2)
Call ID: call_2
Args:
  rating: 5
  comment: Amazing product

===== Tool Message =====
Name: ProductRating

Returning structured response: {'rating': 5, 'comment': 'Amazing product
```

错误处理策略 (Error handling strategies)

你可以通过 `handle_errors` 参数自定义错误处理行为：

自定义错误消息 (Custom error message)

```
ToolStrategy(
    schema=ProductRating,
    handle_errors="Please provide a valid rating between 1-5 and include
)
```

如果 `handle_errors` 是字符串，智能体会**始终**用这段固定的 ToolMessage 引导模型重试：

```
===== Tool Message =====  
Name: ProductRating  
  
Please provide a valid rating between 1-5 and include a comment.
```

只处理特定异常 (Handle specific exceptions only)

```
ToolStrategy(  
    schema=ProductRating,  
    handle_errors=ValueError, # 仅在 ValueError 时重试，其他异常直接抛出  
)
```

如果 `handle_errors` 是某个异常类型，则只有在该异常被抛出时才会重试（并使用默认错误消息）；其他异常会被直接抛出。

处理多种异常类型 (Handle multiple exception types)

```
ToolStrategy(  
    schema=ProductRating,  
    handle_errors=(ValueError, TypeError), # 在 ValueError 或 TypeError 时  
)
```

如果 `handle_errors` 是异常类型元组，则仅在抛出的异常属于其中任一种时才会重试，其余异常直接抛出。

自定义错误处理函数 (Custom error handler function)

```
from typing import Union  
from langchain.agents.structured_output import (  
    StructuredOutputValidationError,  
    MultipleStructuredOutputsError,  
)
```

```

def custom_error_handler(error: Exception) -> str:
    """根据错误类型返回不同的提示内容。"""
    if isinstance(error, StructuredOutputValidationError):
        return "There was an issue with the format. Try again."
    elif isinstance(error, MultipleStructuredOutputsError):
        return "Multiple structured outputs were returned. Pick the most
    else:
        return f"Error: {str(error)}"

agent = create_agent(
    model="gpt-5",
    tools=[],
    response_format=ToolStrategy(
        schema=Union[ContactInfo, EventDetails],
        handle_errors=custom_error_handler,
    ),
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Extract info: John Doe (joh
})

for msg in result["messages"]:
    # 如果是 ToolMessage 对象
    if type(msg).__name__ == "ToolMessage":
        print(msg.content)
    # 如果是字典, 或你想要一个兜底处理
    elif isinstance(msg, dict) and msg.get("tool_call_id"):
        print(msg["content"])

```

对于 `StructuredOutputValidationError` :

```

===== Tool Message =====
Name: ToolStrategy

```

There was an issue with the format. Try again.

对于 `MultipleStructuredOutputsError` :

```
===== Tool Message =====  
Name: ToolStrategy  
  
Multiple structured outputs were returned. Pick the most relevant one.
```

其他错误:

```
===== Tool Message =====  
Name: ToolStrategy  
  
Error: <error message>
```

关闭错误处理 (No error handling)

```
response_format = ToolStrategy(  
    schema=ProductRating,  
    handle_errors=False, # 所有错误直接抛出  
)
```

此时，结构化输出相关的所有异常都会直接抛出给调用方，你可以在应用层自行捕获并处理。

本文档由 LangChain 官方文档翻译而来

\newpage

13. 中间件概述 (Middleware Overview)

原文链接: <https://docs.langchain.com/oss/python/langchain/middleware/overview>

在每一步精细控制和自定义智能体的执行过程。

中间件 (Middleware) 提供了一种方式, 让你可以更紧密地控制智能体内部发生的事情。它在以下场景中非常有用:

- **行为追踪**: 通过日志、埋点、分析与调试跟踪智能体行为。
- **请求 / 响应改写**: 在模型调用前后修改提示词 (prompt)、工具选择和输出格式。
- **可靠性增强**: 添加重试、回退 (fallback) 和提前终止逻辑。
- **策略与安全**: 实现速率限制、Guardrails、PII (敏感信息) 检测等。

你可以在调用 `create_agent` 时, 通过 `middleware` 参数添加一个或多个中间件:

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware, HumanInTheLoopMiddleware

agent = create_agent(
    model="gpt-4o",
    tools=[...],
    middleware=[
        SummarizationMiddleware(...),          # 自动对长对话做摘要
        HumanInTheLoopMiddleware(...),        # 在关键步骤引入人工审批
    ],
)
```

智能体循环 (The agent loop)

核心的智能体循环大致包含以下步骤:

1. 调用模型 (Model), 让其基于当前状态和消息进行推理;
2. 根据模型输出选择并调用工具 (Tools);
3. 将工具结果反馈给模型, 进入下一轮推理;

4. 当模型不再请求任何工具调用时，生成最终回答并结束循环。

中间件在这一循环中暴露了多个钩子（hook），你可以在每一步“之前 / 之后”插入自定义逻辑，例如：

- **before_model / after_model**：模型调用前后拦截与处理消息、状态；
- **wrap_model_call**：统一包装模型调用，加入重试、日志或模型路由等逻辑；
- **wrap_tool_call**：在工具调用前后做参数检查、错误处理或安全审计；
- **动态系统提示 / 动态上下文**：根据当前状态和上下文生成 system prompt。

这些钩子让你可以在不修改智能体核心逻辑的前提下，灵活扩展和控制执行流程。

更多资源（Additional resources）

- **Built-in middleware** — 了解 LangChain 为常见用例提供的内置中间件（如摘要、日志、人类在回路等）。
 - **Custom middleware** — 学习如何使用钩子和装饰器构建自己的中间件。
 - **Middleware API reference** — 查阅中间件相关的完整 API 参考文档。
 - **Testing agents** — 使用 LangSmith 测试和评估你的智能体行为与质量。
-

本文档由 LangChain 官方文档翻译而来

\newpage

14. 内置中间件（Built-in middleware）

原文链接: <https://docs.langchain.com/oss/python/langchain/middleware/built-in>

为常见智能体场景准备的预构建中间件。

LangChain 为常见用例提供了一组预构建中间件。这些中间件都可以直接用于生产环境，并且可以根据你的具体需求进行配置。

与提供商无关的中间件（Provider-agnostic middleware）

以下中间件可以与任意 LLM 提供商一起使用：

中间件	描述
Summarization（摘要）	在接近 token 上限时自动总结对话历史，保留最近消息并压缩较早上下文。
Human-in-the-loop（人在回路）	在工具调用执行前暂停，由人类进行审批。
Model call limit（模型调用上限）	限制模型调用次数，以防止成本过高。
Tool call limit（工具调用上限）	通过限制调用次数来控制工具执行。
Model fallback（模型回退）	主模型失败时自动回退到备用模型。
PII detection（PII 检测）	检测并处理个人可识别信息（PII）。
To-do list（待办列表）	让智能体具备任务规划和跟踪能力。
LLM tool selector（工具选择器）	使用 LLM 在调用主模型前先筛选出相关工具。
Tool retry（工具重试）	自动对失败的工具调用进行指数退避重试。
Model retry（模型重试）	自动对失败的模型调用进行指数退避重试。
LLM tool emulator（工具模拟器）	使用 LLM 模拟工具执行，便于测试。
Context editing（上下文编辑）	通过清理或裁剪工具调用结果来管理对话上下文。
Shell tool（Shell 工具）	为智能体暴露持久化 Shell 会话以执行命令。
File search（文件搜索）	提供基于 Glob 和 Grep 的文件系统搜索工具。

下面选取几个核心中间件进行简要说明和示例。

Summarization（摘要中间件）

用途：

- 对话较长，可能超过上下文窗口；
- 多轮对话，需要保留全局语境；
- 希望在不丢失关键信息的前提下压缩历史。

API 参考： `SummarizationMiddleware`

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware

agent = create_agent(
    model="gpt-4o",
    tools=[your_weather_tool, your_calculator_tool],
    middleware=[
        SummarizationMiddleware(
            model="gpt-4o-mini",          # 用于生成摘要的模型
            trigger=("tokens", 4000),     # 当 token 数 >= 4000 时触发摘要
            keep=("messages", 20),        # 摘要后保留最近 20 条消息
        ),
    ],
)
```

配置选项（关键字段）

下面的 `fraction` 条件依赖于模型的 profile 数据（`langchain>=1.1`）。如果没有 profile，可以手动指定。

```
from langchain.chat_models import init_chat_model

custom_profile = {
```

```

    "max_input_tokens": 100_000,
    # ...
}
model = init_chat_model("gpt-4o", profile=custom_profile)

```

- **model** (必填)
- 用于生成摘要的模型（字符串 ID 或 `BaseChatModel` 实例）。
- **trigger**
- 触发摘要的条件，可以是单个条件或条件列表（列表时为“任一满足即触发”）：
 - `("fraction", 0.8)`：上下文达到模型最大长度的 80%；
 - `("tokens", 4000)`：总 token 数达到 4000；
 - `("messages", 10)`：消息数达到 10。
- **keep** (默认 `("messages", 20)`)
- 摘要后要保留多少上下文，只能指定一种：
 - `("fraction", 0.3)`：保留上下文窗口的 30%；
 - `("tokens", 1000)`：保留 1000 个 token；
 - `("messages", 20)`：保留最近 20 条消息。
- **summary_prompt / summary_prefix / trim_tokens_to_summarize** 等
- 控制摘要提示词、摘要前缀和用于摘要的最大 token 数等，具体可见官方 API 文档。

完整示例（单条件 / 多条件 / fraction）：

```

from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware

# 单条件：tokens >= 4000 时触发
agent = create_agent(
    model="gpt-4o",
    tools=[your_weather_tool, your_calculator_tool],
    middleware=[

```

```

        SummarizationMiddleware(
            model="gpt-4o-mini",
            trigger=("tokens", 4000),
            keep=("messages", 20),
        ),
    ],
)

# 多条件: tokens >= 3000 或 messages >= 6
agent2 = create_agent(
    model="gpt-4o",
    tools=[your_weather_tool, your_calculator_tool],
    middleware=[
        SummarizationMiddleware(
            model="gpt-4o-mini",
            trigger=[
                ("tokens", 3000),
                ("messages", 6),
            ],
            keep=("messages", 20),
        ),
    ],
)

# 使用 fraction 作为阈值
agent3 = create_agent(
    model="gpt-4o",
    tools=[your_weather_tool, your_calculator_tool],
    middleware=[
        SummarizationMiddleware(
            model="gpt-4o-mini",
            trigger=("fraction", 0.8), # 上下文达到 80%
            keep=("fraction", 0.3),   # 保留 30%
        ),
    ],
)

```

Human-in-the-loop（人在回路）

用途：

- 高风险操作（数据库写入、资金操作等）需要人工审批；
- 合规要求必须有人审查；
- 长对话中希望由人类指导智能体策略。

API 参考： `HumanInTheLoopMiddleware`

该中间件会在某些工具调用前插入“中断（interrupt）”，前端可以通过 `interrupt` 信息展示审批界面（如 approve / reject / edit），并将用户决定再传回智能体继续执行。

（具体代码较长，可参考官方 human-in-the-loop 示例。）

Model / Tool 限制与重试

Model call limit / Tool call limit

- **Model call limit**：限制单次 run 或整个会话中的模型调用次数，避免意外循环或高额费用。
- **Tool call limit**：类似，但针对工具调用次数进行限制。

这类中间件通常通过计数器 + 阈值的方式实现，超出后会抛出错误或返回特定提示。

Model retry / Tool retry

- **Model retry**：为模型调用添加自动重试和指数退避逻辑，适合处理网络抖动、临时错误等；
- **Tool retry**：为工具调用添加同样的重试机制。

在创建智能体时加入这些中间件即可自动获得更健壮的调用链。

PII 检测（PII detection）与自定义 PII 类型

PII 检测中间件可以帮助你发现和处理个人可识别信息（PII），例如： – 邮箱、电话号码、地址； – 身份证号、信用卡号等。

通常可以配置： – 检测哪些类型的 PII； – 发现后是替换（mask）、删除，还是阻止继续输出； – 是否记录相关日志以满足审计需求。

你也可以自定义 PII 类型，通过正则或自定义逻辑扩展检测规则。

To-do list（待办列表中间件）

为智能体提供“任务规划 + 跟踪”能力，例如： – 将复杂指令拆分为多个子任务； – 记录每个任务的状态（待办 / 进行中 / 完成）； – 在对话中更新和展示任务进度。

适合构建“智能执行助手”、自动化流程执行等场景。

LLM tool selector（工具选择器）

在调用主模型前，可以先用一个轻量模型 / 单独中间件，基于用户请求选择： – 哪些工具可能相关； – 哪些工具应被禁用或忽略。

这对于： – 工具数量很多； – 某些工具代价较高； – 需要基于上下文动态裁剪工具集的场景非常有用，可以降低成本并提高调用精度。

LLM tool emulator（工具模拟器）

在测试阶段，你可能并不希望真正调用外部 API 或风险操作。

LLM tool emulator 中间件可以： – 不实际调用工具，而是用 LLM“假装”执行工具； – 根据工具描述和参数生成模拟结果； – 帮助你在无真实依赖的环境下调试智能体逻辑。

Context editing（上下文编辑）

Context editing 中间件可以帮助你达到 token 上限时“整理对话上下文”，最常见的是清理旧的工具结果。

典型策略：`ClearToolUsesEdit` —— 清除较早的工具输出，保留最近 N 个工具结果。

```
from langchain.agents import create_agent
from langchain.agents.middleware import ContextEditingMiddleware, ClearToolUsesEdit

agent = create_agent(
    model="gpt-4o",
    tools=[search_tool, your_calculator_tool, database_tool],
    middleware=[
        ContextEditingMiddleware(
            edits=[
                ClearToolUsesEdit(
                    trigger=2000,          # token 数达到 2000 时触发
                    keep=3,                # 保留最近 3 个工具结果
                    clear_tool_inputs=False,
                    exclude_tools=[],
                    placeholder="[cleared]", # 被清理内容的占位符
                ),
            ],
        ),
    ],
)
```

这样既能控制上下文长度，又能保留最近的关键工具调用结果。

Shell tool（Shell 工具中间件）

Shell 工具中间件为智能体提供一个持久化的 **Shell 会话**，适合：

- 执行系统命令；
- 部署 / 运维自动化；
- 运行脚本、文件操作和测试流程。

安全提示：应根据部署环境选择合适的执行策略（`HostExecutionPolicy`、`DockerExecutionPolicy` 或 `CodexSandboxExecutionPolicy`），以匹配你的安全要求。

限制：当前持久化 Shell 会话尚不支持与“人在回路中断”配合使用。

API 参考： `ShellToolMiddleware`

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
    ShellToolMiddleware,
    HostExecutionPolicy,
)

agent = create_agent(
    model="gpt-4o",
    tools=[search_tool],
    middleware=[
        ShellToolMiddleware(
            workspace_root="/workspace",      # Shell 会话的工作目录
            execution_policy=HostExecutionPolicy(), # 执行策略
        ),
    ],
)
```

重要配置项

- **workspace_root**: Shell 会话的根目录
 - **startup_commands / shutdown_commands**: 会话启动 / 关闭时自动执行的命令
 - **execution_policy**: 执行策略（宿主机 / Docker / Sandbox）
 - **redaction_rules**: 输出脱敏规则（如隐藏 API Key）
 - **shell_command**: 指定 Shell 可执行文件（默认 `/bin/bash`）
 - **env**: 为 Shell 会话注入的环境变量
-

File search（文件搜索中间件）

文件搜索中间件为智能体提供： – **Glob 搜索**：按模式快速匹配文件（如 `**/*.py`、`src/**/*.ts`）； – **Grep 搜索**：基于正则在文件内容中搜索；

API 参考： `FilesystemFileSearchMiddleware`

```
from langchain.agents import create_agent
from langchain.agents.middleware import FilesystemFileSearchMiddleware

agent = create_agent(
    model="gpt-4o",
    tools=[],
    middleware=[
        FilesystemFileSearchMiddleware(
            root_path="/workspace", # 搜索的根目录
            use_ripgrep=True,       # 优先使用 ripgrep
        ),
    ],
)
```

常见用法： – 代码库探索和分析； – 按文件名模式查找文件； – 在大规模代码库中通过正则查找特定片段。

该中间件会为智能体注入： – `glob_search`：匹配文件路径，并按修改时间排序； – `grep_search`：按内容搜索，并支持多种输出模式（仅文件列表 / 匹配内容 / 计数）。

针对特定提供商的中间件（Provider-specific middleware）

这些中间件针对特定 LLM 提供商进行了优化。详情和完整示例请参考各提供商文档：

- **Anthropic**：为 Claude 模型提供提示缓存、bash 工具、文本编辑器、记忆和文件搜索等中间件。

- **OpenAI**: 为 OpenAI 模型提供内容审核 (content moderation) 中间件。

本文档由 LangChain 官方文档翻译而来

\newpage

15. 自定义中间件 (Custom middleware)

原文链接: <https://docs.langchain.com/oss/python/langchain/middleware/custom>

通过在智能体执行流程的特定节点实现钩子 (hooks)，你可以构建自定义中间件，对智能体行为进行精细控制。

钩子 (Hooks)

中间件提供两类钩子来拦截智能体执行：

- **节点式钩子 (Node-style hooks)**：在特定执行点顺序运行，适合日志、校验和状态更新。
- **包装式钩子 (Wrap-style hooks)**：包裹模型或工具调用，适合重试、缓存、变换等场景。

节点式钩子 (Node-style hooks)

节点式钩子在特定执行点按顺序运行，常用于： – 记录日志 – 校验请求/响应 – 更新状态

可用的节点式钩子： – `before_agent`：智能体开始执行前触发（每次调用一次） – `before_model`：每次模型调用前触发 – `after_model`：每次模型响应后触发 – `after_agent`：智能体执行完成后触发（每次调用一次）

示例：使用装饰器

```
from langchain.agents.middleware import before_model, after_model, AgentMiddleware
from langchain.messages import AIMessage
```

```

from langgraph.runtime import Runtime
from typing import Any

@before_model(can_jump_to=["end"])
def check_message_limit(state: AgentState, runtime: Runtime) -> dict[str, Any]:
    """限制对话消息数量，超过阈值则结束对话。"""
    if len(state["messages"]) >= 50:
        return {
            "messages": [AIMessage("Conversation limit reached.")],
            "jump_to": "end",
        }
    return None

@after_model
def log_response(state: AgentState, runtime: Runtime) -> dict[str, Any]:
    """打印模型最新一条响应内容。"""
    print(f"Model returned: {state['messages'][-1].content}")
    return None

```

示例：使用类

```

from langchain.agents.middleware import AgentMiddleware, AgentState, hook
from langchain.messages import AIMessage
from langgraph.runtime import Runtime
from typing import Any

class MessageLimitMiddleware(AgentMiddleware):
    def __init__(self, max_messages: int = 50):
        super().__init__()
        self.max_messages = max_messages

    @hook_config(can_jump_to=["end"])
    def before_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any]:

```

```

        """当消息数量达到上限时结束对话。"""
        if len(state["messages"]) == self.max_messages:
            return {
                "messages": [AIMessage("Conversation limit reached.")],
                "jump_to": "end",
            }
        return None

    def after_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any]:
        """记录最新一条模型输出。"""
        print(f"Model returned: {state['messages'][-1].content}")
        return None

```

包装式钩子 (Wrap-style hooks)

包装式钩子用于拦截执行并控制 handler 何时被调用。适用场景： – 重试逻辑 – 缓存 – 请求或响应变换

你可以决定 handler： – 调用 0 次（短路、直接返回） – 调用 1 次（正常流程） – 调用多次（重试）

可用的包装式钩子： – `wrap_model_call`：包裹每次模型调用 – `wrap_tool_call`：包裹每次工具调用

示例：使用装饰器实现模型重试

```

from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@wrap_model_call
def retry_model(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """为模型调用添加最多 3 次重试。"""

```

```
for attempt in range(3):
    try:
        return handler(request)
    except Exception as e:
        if attempt == 2:
            raise
        print(f"Retry {attempt + 1}/3 after error: {e}")
```

示例：使用类实现模型重试

```
from langchain.agents.middleware import AgentMiddleware, ModelRequest, ModelResponse
from typing import Callable

class RetryMiddleware(AgentMiddleware):
    def __init__(self, max_retries: int = 3):
        super().__init__()
        self.max_retries = max_retries

    def wrap_model_call(
        self,
        request: ModelRequest,
        handler: Callable[[ModelRequest], ModelResponse],
    ) -> ModelResponse:
        """为模型调用添加可配置的重试次数。"""
        for attempt in range(self.max_retries):
            try:
                return handler(request)
            except Exception as e:
                if attempt == self.max_retries - 1:
                    raise
                print(f"Retry {attempt + 1}/{self.max_retries} after error: {e}")
```

创建中间件 (Create middleware)

你可以通过两种方式创建中间件：

- **基于装饰器 (Decorator-based middleware)**：适合单个钩子、轻量逻辑和快速原型。
 - **基于类 (Class-based middleware)**：适合多个钩子、复杂配置及可复用组件。
-

基于装饰器的中间件 (Decorator-based middleware)

适合只需要单个钩子的场景，使用装饰器包装独立函数。

可用装饰器：

- 节点式：
 - `@before_agent`
 - `@before_model`
 - `@after_model`
 - `@after_agent`
- 包装式：
 - `@wrap_model_call`
 - `@wrap_tool_call`
- 便捷装饰器：
 - `@dynamic_prompt`：用于动态生成系统提示 (system prompt)

示例

```
from langchain.agents.middleware import (
    before_model,
    wrap_model_call,
    AgentState,
    ModelRequest,
    ModelResponse,
)
from langchain.agents import create_agent
from langgraph.runtime import Runtime
from typing import Any, Callable
```

```

@before_model
def log_before_model(state: AgentState, runtime: Runtime) -> dict[str, Any]:
    """在调用模型前记录消息数量。"""
    print(f>About to call model with {len(state['messages'])} messages")
    return None

@wrap_model_call
def retry_model(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """为模型调用添加简单的 3 次重试逻辑。"""
    for attempt in range(3):
        try:
            return handler(request)
        except Exception as e:
            if attempt == 2:
                raise
            print(f"Retry {attempt + 1}/3 after error: {e}")

agent = create_agent(
    model="gpt-4o",
    middleware=[log_before_model, retry_model],
    tools=[...],
)

```

适用场景： – 只需一个钩子 – 不需要复杂配置 – 快速试验 / 原型开发

基于类的中间件 (Class-based middleware)

对于需要多个钩子或复杂配置的场景，使用继承自 `AgentMiddleware` 的类更合适。你可以在同一个类中实现多个钩子，甚至为同一个钩子提供同步和异步版本。

示例

```
from langchain.agents.middleware import (
    AgentMiddleware,
    AgentState,
    ModelRequest,
    ModelResponse,
)
from langgraph.runtime import Runtime
from typing import Any, Callable

class LoggingMiddleware(AgentMiddleware):
    def before_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any]:
        """模型调用前记录消息数量。"""
        print(f>About to call model with {len(state['messages'])} messages")
        return None

    def after_model(self, state: AgentState, runtime: Runtime) -> dict[str, Any]:
        """模型调用后记录最新输出。"""
        print(f"Model returned: {state['messages'][-1].content}")
        return None

agent = create_agent(
    model="gpt-4o",
    middleware=[LoggingMiddleware()],
    tools=[...],
)
```

适用场景： – 需要为同一中间件定义多个钩子 – 需要同时支持 sync / async 版本 – 需要复杂配置（阈值、自定义模型等） – 希望在多个项目间复用中间件

自定义状态模式（Custom state schema）

中间件可以扩展智能体的状态，添加自定义字段，这允许你：

- **跨执行跟踪状态**：维护计数器、标志或其他在整个执行期间持久存在的值；
- **在钩子之间共享数据**：例如从 `before_model` 向 `after_model` 传递信息；
- **实现横切关注点**：如限流、使用量统计、用户上下文或审计日志；
- **基于累积状态做决策**：决定是否继续执行、跳转到其他节点或修改请求。

在 Python 端通常通过扩展 `AgentState`（`TypedDict`）来定义自定义状态结构，并在 `create_agent` 中通过 `state_schema` 传入。然后在中间件钩子中使用 `state["your_field"]` 即可读写这些字段。

执行顺序（Execution order）

当你注册多个中间件时： – 节点式钩子会按中间件在列表中的顺序依次执行； – 包装式钩子则类似“洋葱模型”，后注册的在内层，先注册的在外层包裹。

设计中间件顺序时，可以考虑： – 日志 / 追踪类中间件放在外层； – 重试 / 回退策略中间件包裹在更接近模型 / 工具的一层； – 安全 / Guardrails 中间件尽量靠近实际执行点。

Agent 跳转（Agent jumps）

节点式钩子可以通过返回包含 `"jump_to"` 字段的更新，让智能体跳转到特定节点（例如 `"end"`）。这常用于： – 在满足某些条件时提前结束对话； – 根据状态在不同子图 / 节点之间路由； – 实现自定义的控制流。

示例见前面的 `check_message_limit` / `MessageLimitMiddleware`。

最佳实践（Best practices）

- 将业务无关的横切逻辑（日志、限流、监控）放到中间件中，不要写死在业务代码里。

- 对常用模式（例如重试、动态模型选择、工具监控）优先考虑复用官方或自封装中间件。
 - 使用类中间件来封装可复用的、可配置的逻辑，并在多个项目中共享。
 - 在修改系统消息、状态或工具列表时，优先使用 `ModelRequest.override(...)`、`state` 和 `runtime` 提供的标准接口。
-

示例 (Examples)

动态模型选择 (Dynamic model selection)

根据请求的复杂度或对话长度动态选择模型：

装饰器写法

```
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from langchain.chat_models import init_chat_model
from typing import Callable

complex_model = init_chat_model("gpt-4o")
simple_model = init_chat_model("gpt-4o-mini")

@wrap_model_call
def dynamic_model(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """根据对话长度选择不同模型。"""
    if len(request.messages) > 10:
        model = complex_model
    else:
        model = simple_model
    return handler(request.override(model=model))
```

类写法

```
from langchain.agents.middleware import AgentMiddleware, ModelRequest, ModelResponse
from langchain.chat_models import init_chat_model
from typing import Callable

complex_model = init_chat_model("gpt-4o")
simple_model = init_chat_model("gpt-4o-mini")

class DynamicModelMiddleware(AgentMiddleware):
    def wrap_model_call(
        self,
        request: ModelRequest,
        handler: Callable[[ModelRequest], ModelResponse],
    ) -> ModelResponse:
        """根据对话长度选择不同模型。"""
        if len(request.messages) > 10:
            model = complex_model
        else:
            model = simple_model
        return handler(request.override(model=model))
```

工具调用监控 (Tool call monitoring)

装饰器写法

```
from langchain.agents.middleware import wrap_tool_call
from langchain.tools.tool_node import ToolCallRequest
from langchain.messages import ToolMessage
from langgraph.types import Command
from typing import Callable
```

```

@wrap_tool_call
def monitor_tool(
    request: ToolCallRequest,
    handler: Callable[[ToolCallRequest], ToolMessage | Command],
) -> ToolMessage | Command:
    """在工具执行前后打印调用信息。"""
    print(f"Executing tool: {request.tool_call['name']}")
    print(f"Arguments: {request.tool_call['args']}")
    try:
        result = handler(request)
        print("Tool completed successfully")
        return result
    except Exception as e:
        print(f"Tool failed: {e}")
        raise

```

类写法

```

from langchain.tools.tool_node import ToolCallRequest
from langchain.agents.middleware import AgentMiddleware
from langchain.messages import ToolMessage
from langgraph.types import Command
from typing import Callable

class ToolMonitoringMiddleware(AgentMiddleware):
    def wrap_tool_call(
        self,
        request: ToolCallRequest,
        handler: Callable[[ToolCallRequest], ToolMessage | Command],
    ) -> ToolMessage | Command:
        """监控工具调用输入与结果。"""
        print(f"Executing tool: {request.tool_call['name']}")
        print(f"Arguments: {request.tool_call['args']}")
        try:
            result = handler(request)

```

```
        print("Tool completed successfully")
        return result
    except Exception as e:
        print(f"Tool failed: {e}")
        raise
```

动态选择工具 (Dynamically selecting tools)

在运行时选择与当前上下文最相关的一小部分工具，以提升性能和准确性：

- **更短的提示词**：减少暴露给模型的工具数量；
- **更高的准确率**：从更小的候选集中选择；
- **权限控制**：根据用户权限动态过滤工具。

装饰器写法

```
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@wrap_model_call
def select_tools(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """根据状态/上下文动态筛选工具。"""
    relevant_tools = select_relevant_tools(request.state, request.runtime_tools)
    return handler(request.override(tools=relevant_tools))

agent = create_agent(
    model="gpt-4o",
    tools=all_tools,  # 所有工具预注册
```

```
        middleware=[select_tools], # 由中间件动态裁剪工具集
    )
```

类写法

```
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware, ModelRequest, ModelResponse
from typing import Callable

class ToolSelectorMiddleware(AgentMiddleware):
    def wrap_model_call(
        self,
        request: ModelRequest,
        handler: Callable[[ModelRequest], ModelResponse],
    ) -> ModelResponse:
        """根据状态/上下文动态筛选工具。"""
        relevant_tools = select_relevant_tools(request.state, request.run_history)
        return handler(request.override(tools=relevant_tools))

agent = create_agent(
    model="gpt-4o",
    tools=all_tools,
    middleware=[ToolSelectorMiddleware()],
)
```

处理系统消息 (Working with system messages)

可以在中间件中通过 `ModelRequest.system_message` 字段修改系统消息。该字段始终是一个 `SystemMessage` 对象（即使创建智能体时使用的是字符串形式的 `system_prompt`）。

示例：在 system message 中追加上下文

```
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from langchain.messages import SystemMessage
from typing import Callable

@wrap_model_call
def add_context(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """在系统消息中追加额外文本上下文。"""
    # 始终使用 content_blocks 进行操作
    new_content = list(request.system_message.content_blocks) + [
        {"type": "text", "text": "Additional context."}
    ]
    new_system_message = SystemMessage(content=new_content)
    return handler(request.override(system_message=new_system_message))
```

示例：配合 Anthropic 使用缓存控制 (cache control)

对于 Anthropic 模型，可以通过结构化内容块与 `cache_control` 指令对大型系统提示进行缓存：

```
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from langchain.messages import SystemMessage
from typing import Callable

@wrap_model_call
def add_cached_context(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """将可缓存的大文档添加到系统消息中。"""
```

```
new_content = list(request.system_message.content_blocks) + [
    {
        "type": "text",
        "text": "Here is a large document to analyze:\n\n<document>.",
        "cache_control": {"type": "ephemeral"}, # 此内容将被缓存
    }
]

new_system_message = SystemMessage(content=new_content)
return handler(request.override(system_message=new_system_message))
```

注意： – `ModelRequest.system_message` 总是 `SystemMessage` 对象； – 使用 `SystemMessage.content_blocks` 以块列表形式操作内容； – 修改系统消息时应基于现有 `content_blocks` 追加，而非完全覆盖，除非你确实想替换全部内容； – 对于复杂缓存控制场景，也可以直接将 `SystemMessage` 实例传给 `create_agent` 的 `system_prompt` 参数。

更多资源 (Additional resources)

- **Middleware API reference** — 中间件完整 API 参考
- **Built-in middleware** — LangChain 提供的内置中间件列表与示例
- **Testing agents** — 使用 LangSmith 对智能体进行测试与评估

本文档由 LangChain 官方文档翻译而来

\newpage

16. 防护栏 (Guardrails)

原文链接: <https://docs.langchain.com/oss/python/langchain/guardrails>

为智能体实现安全检查与内容过滤

防护栏（Guardrails）通过在智能体执行的关键节点验证和过滤内容，帮助你构建安全、合规的 AI 应用。它们可以： – 检测敏感信息 – 执行内容策略 – 验证输出 – 在问题发生前阻止不安全行为

常见用例包括： * 防止 PII（个人可识别信息）泄露 * 检测并阻止提示注入攻击 * 阻止不当或有害内容 * 执行业务规则和合规要求 * 验证输出质量和准确性

你可以使用中间件在策略点拦截执行（智能体开始前、完成后，或围绕模型和工具调用）来实现防护栏。

防护栏可以通过两种互补方式实现：

确定性防护栏（Deterministic guardrails）

使用基于规则的逻辑，如正则表达式模式、关键词匹配或显式检查。速度快、可预测且成本低，但可能遗漏细微的违规行为。

基于模型的防护栏（Model-based guardrails）

使用 LLM 或分类器通过语义理解评估内容。能捕获规则遗漏的细微问题，但速度较慢且成本更高。

LangChain 既提供内置防护栏（如 PII 检测、人在回路），也提供灵活的中间件系统，让你可以使用任一方法构建自定义防护栏。

内置防护栏（Built-in guardrails）

PII 检测（PII detection）

LangChain 提供内置中间件来检测和处理对话中的个人可识别信息（PII）。该中间件可以检测常见的 PII 类型，如邮箱、信用卡号、IP 地址等。

PII 检测中间件适用于： – 医疗和金融等有合规要求的应用 – 需要清理日志的客服智能体 – 任何处理敏感用户数据的应用

PII 中间件支持多种处理策略：

策略	描述	示例
redact	替换为 [REDACTED_{PII_TYPE}]	[REDACTED_EMAIL]
mask	部分遮蔽（如后 4 位）	****_****_****-1234
hash	替换为确定性哈希	a8f5f167...
block	检测到时抛出异常	抛出错误

```
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
    model="gpt-4o",
    tools=[customer_service_tool, email_tool],
    middleware=[
        # 在发送给模型前，对用户输入中的邮箱进行脱敏
        PIIMiddleware(
            "email",
            strategy="redact",
            apply_to_input=True,
        ),
        # 对用户输入中的信用卡号进行遮蔽
        PIIMiddleware(
            "credit_card",
            strategy="mask",
            apply_to_input=True,
        ),
        # 阻止 API 密钥 - 检测到时抛出错误
        PIIMiddleware(
            "api_key",
            detector=r"sk-[a-zA-Z0-9]{32}",
            strategy="block",
            apply_to_input=True,
        ),
    ],
)
```

```
)

# 当用户提供 PII 时，将根据策略进行处理
result = agent.invoke({
    "messages": [{"role": "user", "content": "My email is [email protect
})
```

内置 PII 类型和配置：

内置 PII 类型： * email – 邮箱地址 * credit_card – 信用卡号（Luhn 校验） * ip – IP 地址 * mac_address – MAC 地址 * url – URL

配置选项：

参数	描述	默认值
pii_type	要检测的 PII 类型（内置或自定义）	必填
strategy	如何处理检测到的 PII ("block", "redact", "mask", "hash")	"redact"
detector	自定义检测函数或正则表达式模式	None（使用内置）
apply_to_input	在模型调用前检查用户消息	True
apply_to_output	在模型调用后检查 AI 消息	False
apply_to_tool_results	在执行后检查工具结果消息	False

有关 PII 检测功能的完整详细信息，请参阅中间件文档。

人在回路（Human-in-the-loop）

LangChain 提供内置中间件，要求在执行敏感操作前获得人工审批。这是对高风险决策最有效的防护栏之一。

人在回路中间件适用于： – 金融交易和转账 – 删除或修改生产数据 – 向外部发送通信 – 任何具有重大业务影响的操作

```

from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

agent = create_agent(
    model="gpt-4o",
    tools=[search_tool, send_email_tool, delete_database_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                # 敏感操作需要审批
                "send_email": True,
                "delete_database": True,
                # 安全操作自动批准
                "search": False,
            }
        ),
    ],
    # 在中断之间持久化状态
    checkpointers=[InMemorySaver()],
)

# 人在回路需要 thread ID 以持久化
config = {"configurable": {"thread_id": "some_id"}}

# 智能体将在执行敏感工具前暂停并等待审批
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Send an email to the team"}]},
    config=config
)

result = agent.invoke(
    Command(resume={"decisions": [{"type": "approve"}]}),
    config=config # 使用相同的 thread ID 以恢复暂停的对话
)

```

有关实现审批工作流的完整详细信息，请参阅人在回路文档。

自定义防护栏 (Custom guardrails)

对于更复杂的防护栏，你可以创建在智能体执行前后运行的自定义中间件。这让你可以完全控制验证逻辑、内容过滤和安全检查。

智能体执行前的防护栏 (Before agent guardrails)

使用 "before agent" 钩子在每次调用的开始验证请求一次。这对于会话级检查很有用，如身份验证、速率限制或在任何处理开始前阻止不当请求。

类语法示例

```
from typing import Any
from langchain.agents.middleware import AgentMiddleware, AgentState, hook
from langgraph.runtime import Runtime

class ContentFilterMiddleware(AgentMiddleware):
    """确定性防护栏：阻止包含禁用关键词的请求。"""

    def __init__(self, banned_keywords: list[str]):
        super().__init__()
        self.banned_keywords = [kw.lower() for kw in banned_keywords]

    @hook_config(can_jump_to=["end"])
    def before_agent(self, state: AgentState, runtime: Runtime) -> dict[str, Any]:
        """在智能体开始前检查用户消息是否包含禁用关键词。"""
        # 获取第一条用户消息
        if not state["messages"]:
            return None

        first_message = state["messages"][0]
        if first_message.type != "human":
            return None
```

```

        content = first_message.content.lower()

        # 检查禁用关键词
        for keyword in self.banned_keywords:
            if keyword in content:
                # 在任何处理前阻止执行
                return {
                    "messages": [{
                        "role": "assistant",
                        "content": "I cannot process requests containing
                    }],
                    "jump_to": "end"
                }

        return None

# 使用自定义防护栏
from langchain.agents import create_agent

agent = create_agent(
    model="gpt-4o",
    tools=[search_tool, calculator_tool],
    middleware=[
        ContentFilterMiddleware(
            banned_keywords=["hack", "exploit", "malware"]
        ),
    ],
)

# 此请求将在任何处理前被阻止
result = agent.invoke({
    "messages": [{"role": "user", "content": "How do I hack into a databa
})

```

装饰器语法示例

```
from typing import Any
from langchain.agents.middleware import before_agent, AgentState
from langgraph.runtime import Runtime

@before_agent(can_jump_to=["end"])
def content_filter(state: AgentState, runtime: Runtime) -> dict[str, Any]:
    """在智能体开始前检查内容。"""
    banned_keywords = ["hack", "exploit", "malware"]

    if not state["messages"]:
        return None

    first_message = state["messages"][0]
    if first_message.type != "human":
        return None

    content = first_message.content.lower()

    for keyword in banned_keywords:
        if keyword in content:
            return {
                "messages": [{
                    "role": "assistant",
                    "content": "I cannot process requests containing inappropriate keywords."
                }],
                "jump_to": "end"
            }

    return None
```

智能体执行后的防护栏（After agent guardrails）

使用 "after agent" 钩子在返回给用户前验证最终输出一次。这对于基于模型的安全检查、质量验证或对完整智能体响应的最终合规扫描很有用。

类语法示例

```
from langchain.agents.middleware import AgentMiddleware, AgentState, hook
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
from typing import Any

class SafetyGuardrailMiddleware(AgentMiddleware):
    """基于模型的防护栏：使用 LLM 评估响应安全性。"""

    def __init__(self):
        super().__init__()
        self.safety_model = init_chat_model("gpt-4o-mini")

    @hook_config(can_jump_to=["end"])
    def after_agent(self, state: AgentState, runtime: Runtime) -> dict[str, Any]:
        """在智能体完成后检查最终响应的安全性。"""
        # 获取最终的 AI 响应
        if not state["messages"]:
            return None

        last_message = state["messages"][-1]
        if not isinstance(last_message, AIMessage):
            return None

        # 使用模型评估安全性
        safety_prompt = f"""Evaluate if this response is safe and appropriate for use in a chatbot.
Respond with only 'SAFE' or 'UNSAFE'."""

        Response: {last_message.content}"""
```



```

        result = self.safety_model.invoke([{"role": "user", "content": s

    if "UNSAFE" in result.content:
        last_message.content = "I cannot provide that response. Plea

    return None

# 使用安全防护栏
from langchain.agents import create_agent

agent = create_agent(
    model="gpt-4o",
    tools=[search_tool, calculator_tool],
    middleware=[SafetyGuardrailMiddleware()],
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "How do I make explosives?"}
})

```

装饰器语法示例

```

from langchain.agents.middleware import after_agent, AgentState
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
from typing import Any

safety_model = init_chat_model("gpt-4o-mini")

@after_agent(can_jump_to=["end"])
def safety_check(state: AgentState, runtime: Runtime) -> dict[str, Any]:
    """在智能体完成后进行安全性检查。"""
    if not state["messages"]:
        return None

```

```

last_message = state["messages"][-1]
if not isinstance(last_message, AIMessage):
    return None

safety_prompt = f"""Evaluate if this response is safe and appropriate.
Respond with only 'SAFE' or 'UNSAFE'."""

Response: {last_message.content}"""

result = safety_model.invoke([{"role": "user", "content": safety_prompt}])

if "UNSAFE" in result.content:
    last_message.content = "I cannot provide that response. Please rephrase."

return None

```

组合多个防护栏 (Combine multiple guardrails)

你可以通过将多个防护栏添加到中间件数组中来堆叠它们。它们按顺序执行，允许你构建分层保护：

```

from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware, HumanInTheLoopMiddleware

agent = create_agent(
    model="gpt-4o",
    tools=[search_tool, send_email_tool],
    middleware=[
        # 第 1 层：确定性输入过滤器（智能体执行前）
        ContentFilterMiddleware(banned_keywords=["hack", "exploit"]),

        # 第 2 层：PII 保护（模型调用前后）
        PIIMiddleware("email", strategy="redact", apply_to_input=True),
        PIIMiddleware("email", strategy="redact", apply_to_output=True),
    ]
)

```

```
# 第 3 层：敏感工具的人工审批
HumanInTheLoopMiddleware(interrupt_on={"send_email": True}),

# 第 4 层：基于模型的安全检查（智能体执行后）
SafetyGuardrailMiddleware(),

],
)
```

这种分层方法提供了多层保护： 1. **输入过滤**：在请求进入系统前阻止明显不当的内容 2. **PII 保护**：在输入和输出中检测并处理敏感信息 3. **人工审批**：对高风险操作进行人工审查 4. **输出验证**：在返回给用户前进行最终安全检查

更多资源 (Additional resources)

- **Middleware documentation** – 自定义中间件的完整指南
 - **Middleware API reference** – 中间件 API 的完整参考
 - **Human-in-the-loop** – 为敏感操作添加人工审查
 - **Testing agents** – 测试安全机制的策略
-

本文档由 LangChain 官方文档翻译而来

\newpage

17. 运行时 (Runtime)

原文链接: <https://docs.langchain.com/oss/python/langchain/runtime>

概述 (Overview)

`create_agent` 在底层运行在 LangGraph 的 runtime 之上。LangGraph 暴露了一个 `Runtime` 对象，包含以下信息：

1. **Context (上下文)**：一次智能体调用的静态信息，例如用户 ID、数据库连接或其他依赖

2. **Store（存储）**：用于长期记忆的 `BaseStore` 实例
3. **Stream writer（流写入器）**：用于通过 `"custom"` 流模式发送自定义流式信息的对象

运行时上下文为你的工具和中间件提供了一种 **依赖注入（dependency injection）** 的机制。

相比在代码中写死配置或使用全局变量，你可以在调用智能体时注入依赖（例如数据库连接、用户 ID 或配置），这让工具更易于测试、复用和扩展。

你可以在 **工具** 和 **中间件** 中访问运行时信息。

访问方式（Access）

使用 `create_agent` 创建智能体时，可以通过 `context_schema` 定义运行时 `context` 的结构。当调用智能体时，通过 `context` 参数传入本次运行的配置：

```
from dataclasses import dataclass

from langchain.agents import create_agent

@dataclass
class Context:
    user_name: str

agent = create_agent(
    model="gpt-5-nano",
    tools=[...],
    context_schema=Context, # 定义运行时上下文结构
)

agent.invoke(
    {"messages": [{"role": "user", "content": "What's my name?"}]},
```

```
context=Context(user_name="John Smith"), # 为本次调用注入上下文
)
```

`Context` 的结构是你自定义的，可以根据业务需要添加更多字段（如 `user_id`、权限信息、区域设置等）。

在工具中访问运行时（Inside tools）

你可以在工具内部访问运行时信息，以便：

- 读取上下文（如 `user_id`、`user_name`）
- 读写长期记忆（store）
- 通过自定义流模式写入进度或状态更新

在工具签名中使用 `ToolRuntime` 参数即可访问 `Runtime` 对象：

```
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime

@dataclass
class Context:
    user_id: str

@tool
def fetch_user_email_preferences(runtime: ToolRuntime[Context]) -> str:
    """从存储中获取用户的邮件偏好。"""
    # 从上下文中读取用户 ID
    user_id = runtime.context.user_id

    # 默认偏好
    preferences: str = "The user prefers you to write a brief and polite

    # 如果配置了 store，则尝试从长期记忆中读取
    if runtime.store:
        if memory := runtime.store.get(("users",), user_id):
            preferences = memory.value["preferences"]
```

```
return preferences
```

这里：

- `runtime.context` 提供运行时上下文（由 `context_schema` 定义）；
- `runtime.store` 提供长期存储接口，可以按 key 读写数据；
- 若你在工具中需要流式输出（如进度条信息），可以通过 `runtime.stream_writer` 写入。

在中间件中访问运行时（Inside middleware）

你也可以在中间件中访问运行时信息，以便：

- 创建动态系统提示（dynamic prompt）
- 根据用户上下文修改消息
- 按用户、环境或业务逻辑控制智能体行为

在中间件装饰器中，通过 `request.runtime` 访问 `Runtime` 对象。`Runtime` 内嵌在传入中间件函数的 `ModelRequest` 参数中。

```
from dataclasses import dataclass

from langchain.messages import AnyMessage
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import dynamic_prompt, ModelRequest, be
from langgraph.runtime import Runtime


@dataclass
class Context:
    user_name: str


# 动态系统提示
@dynamic_prompt
def dynamic_system_prompt(request: ModelRequest) -> str:
    """根据运行时上下文动态生成系统提示。"""
    user_name = request.runtime.context.user_name # 从运行时上下文读取用户名
    system_prompt = f"You are a helpful assistant. Address the user as {"
    return system_prompt
```

```

# 模型调用前的钩子
@before_model
def log_before_model(state: AgentState, runtime: Runtime[Context]) -> dict:
    """在模型调用前打印当前用户信息。"""
    print(f"Processing request for user: {runtime.context.user_name}")
    return None

# 模型调用后的钩子
@after_model
def log_after_model(state: AgentState, runtime: Runtime[Context]) -> dict:
    """在模型调用后打印当前用户信息。"""
    print(f"Completed request for user: {runtime.context.user_name}")
    return None

agent = create_agent(
    model="gpt-5-nano",
    tools=[...],
    middleware=[dynamic_system_prompt, log_before_model, log_after_model],
    context_schema=Context, # 声明上下文结构
)

agent.invoke(
    {"messages": [{"role": "user", "content": "What's my name?"}]},
    context=Context(user_name="John Smith"), # 为本次请求注入上下文
)

```

在这个示例中： – `dynamic_system_prompt` 使用 `request.runtime.context` 构造个性化的 system prompt； – `log_before_model` / `log_after_model` 通过 `runtime.context` 访问用户信息，用于日志或审计； – 所有这些逻辑都不依赖全局变量，而是依赖于运行时注入的上下文和 store，使得中间件在测试和复用更加方便。

18. 智能体中的上下文工程 (Context engineering in agents)

原文链接: <https://docs.langchain.com/oss/python/langchain/context-engineering>

概述 (Overview)

构建智能体 (或任何 LLM 应用) 最难的部分, 是让它“足够可靠”。很多原型在 demo 时看起来可用, 但在真实场景中经常失败。

为什么智能体会失败? (Why do agents fail?)

当智能体失败时, 通常是因为智能体内部的 LLM 调用了“错误的动作”, 或者没有按预期做事。LLM 失败大致有两种原因:

1. 底层 LLM 本身能力不够
2. 没有向 LLM 传递“正确的上下文”

更多时候, 真正的原因其实是 **第二种**: 上下文不对。

上下文工程 (Context engineering) 的核心, 是以正确的格式把“正确的信息与工具”交给 LLM, 让它能够完成任务。这是 AI 工程师的首要工作。缺少“正确上下文”是阻碍智能体可靠性的首要因素, 而 LangChain 的智能体抽象正是为上下文工程而设计的。

如果你刚接触上下文工程, 推荐先阅读“上下文类型的概念性概览”, 理解各种上下文类型以及何时使用。

智能体循环（The agent loop）

一个典型的智能体循环包含两个主要步骤：

- 1. 模型调用（Model call）：
- 2. 使用 prompt 和可用工具调用 LLM；
- 3. 返回“直接响应”或“工具调用请求”。
- 4. 工具执行（Tool execution）：
- 5. 执行 LLM 请求的工具；
- 6. 返回工具结果。

这个循环会持续，直到 LLM 决定“结束”。

你能控制什么（What you can control）

要构建可靠的智能体，你需要控制： – 每一步模型调用时发生什么； – 在这些步骤之间发生什么。

可以从三种“上下文类型”来理解：

上下文类型	你能控制的内容	短暂 / 持久
模型上下文（Model Context）	每次模型调用时传入的内容（指令、消息历史、工具、响应格式）	短暂（per-call）
工具上下文（Tool Context）	工具可以访问 / 读写的内容（state、store、runtime context）	持久
生命周期上下文（Life-cycle Context）	模型与工具调用之间发生的事（摘要、护栏、日志等）	持久

- 短暂上下文（Transient context）：
- 单次 LLM 调用能看到的所有内容；
- 可以修改消息、工具或 prompt，而不改变持久 state。
- 持久上下文（Persistent context）：
- 在多轮对话中保存在 state / store 里的内容；

- 生命周期钩子和工具写入会永久修改它。

数据源（Data sources）

在整个过程中，智能体会读写不同的数据源：

数据源	别名	作用域	示例
Runtime Context	静态配置	会话级（per thread）	用户 ID、API Key、数据库连接、权限、环境配置等
State	短期记忆	会话级	当前消息、上传文件、认证状态、工具结果等
Store	长期记忆	跨会话	用户偏好、提炼出的洞见、记忆、历史数据等

LangChain 的 **中间件（Middleware）** 是上下文工程的底层机制。通过中间件，你可以在智能体生命周期的任意步骤： – 更新上下文（update context）； – 跳转到其他步骤（jump in lifecycle）。

下面分模块介绍： – 模型上下文（Model context） – 工具上下文（Tool context） – 生命周期上下文（Life-cycle context）

模型上下文（Model context）

模型上下文决定了每次模型调用时传入的内容： – System Prompt（系统提示） – Messages（消息列表 / 对话历史） – Tools（可用工具） – Model（使用哪个模型及其配置） – Response Format（期望的输出格式）

这些内容都可以来自： – **State（短期记忆）** – **Store（长期记忆）** – **Runtime Context（运行时上下文）**

System Prompt（系统提示）

系统提示用于设定 LLM 的行为与能力。不同用户、上下文或对话阶段，往往需要不同的指令。成功的智能体会基于记忆、偏好和配置，为当前会话状态提供“正确”的系统提示。

从 State 读取（基于消息数量调整）

```
from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest

@dynamic_prompt
def state_aware_prompt(request: ModelRequest) -> str:
    # request.messages 是 request.state["messages"] 的便捷访问方式
    message_count = len(request.messages)

    base = "You are a helpful assistant."

    if message_count > 10:
        base += "\nThis is a long conversation - be extra concise."

    return base

agent = create_agent(
    model="gpt-4o",
    tools=[...],
    middleware=[state_aware_prompt],
)
```

从 Store 读取（基于长期偏好调整）

```
from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest
from langgraph.store.memory import InMemoryStore
```

```
@dataclass
class Context:
    user_id: str

@dynamic_prompt
def store_aware_prompt(request: ModelRequest) -> str:
    """根据长期记忆中的用户偏好调整系统提示。"""
    user_id = request.runtime.context.user_id

    # 从 Store 中读取用户偏好
    store = request.runtime.store
    user_prefs = store.get(("preferences",), user_id)

    base = "You are a helpful assistant."

    if user_prefs:
        style = user_prefs.value.get("communication_style", "balanced")
        base += f"\nUser prefers {style} responses."

    return base

agent = create_agent(
    model="gpt-4o",
    tools=[...],
    middleware=[store_aware_prompt],
    context_schema=Context,
    store=InMemoryStore(),
)
```

从 Runtime Context 读取（基于角色和环境调整）

```
from dataclasses import dataclass
from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest

@dataclass
class Context:
    user_role: str
    deployment_env: str

@dynamic_prompt
def context_aware_prompt(request: ModelRequest) -> str:
    """根据运行时上下文（角色、环境）调整系统提示。"""
    user_role = request.runtime.context.user_role
    env = request.runtime.context.deployment_env

    base = "You are a helpful assistant."

    if user_role == "admin":
        base += "\nYou have admin access. You can perform all operations"
    elif user_role == "viewer":
        base += "\nYou have read-only access. Guide users to read operat

    if env == "production":
        base += "\nBe extra careful with any data modifications."

    return base

agent = create_agent(
    model="gpt-4o",
    tools=[...],
    middleware=[context_aware_prompt],
```

```
context_schema=Context,  
)
```

Messages（消息）

消息构成了传给 LLM 的完整 prompt。管理好消息内容对 LLM 输出质量至关重要： – 需要包含足够的上下文以生成高质量回答； – 又不能过长导致成本与延迟过高或上下文上限问题。

常用手段包括： – 基于 **State** 做消息裁剪 / 过滤； – 从 **Store** 中注入额外信息（如历史关键摘要）； – 基于 **Runtime Context** 动态控制哪些消息应被包含。

（具体实现可参考“短期记忆（Short-term memory）”和“SummarizationMiddleware”文档。）

Tools（工具）

工具是智能体可以用来“采取行动”的接口。上下文工程中的关键问题包括： – **定义工具（Defining tools）**：提供合适的 docstring、参数名与类型； – **选择工具（Selecting tools）**：在不同上下文下动态裁剪可见工具集合。

你可以使用： – 静态工具列表（简单场景）； – LLM 辅助的工具选择中间件（如 `LLMToolSelectorMiddleware`）； – 自定义中间件（如前文的 `select_tools` 示例）。

Model（模型）

模型本身也是上下文工程的一部分： – 选择哪个 provider / 模型系列； – 使用何种温度、max_tokens 等参数； – 甚至在运行时基于上下文动态切换模型（如 `DynamicModelMiddleware`）。

Response Format（响应格式）

响应格式决定了模型最终输出的 schema： – 自然语言； – 结构化 JSON； – Pydantic / dataclass 对象等。

你可以在 `create_agent` 时通过 `response_format` 配置： – `ProviderStrategy` / `ToolStrategy`； – 或直接传入 schema 类型，由 LangChain 自动选择策略。

工具上下文 (Tool context)

工具既会 **读取** 上下文，也会 **写入** 上下文。

- 在最简单的情况下：
- 工具接收 LLM 的参数；
- 执行操作并返回 ToolMessage；
- 在更复杂的情况下：
- 工具会从 state / store / runtime context 读取额外信息；
- 并将重要信息写回 state / store，以供后续步骤使用。

Reads (读取)

多数真实环境中的工具需要： – 用户 ID（查询数据库）； – API Key（访问外部服务）； – 当前会话状态（决定是否继续执行）。

工具可以从： – **State** 读取当前会话信息； – **Store** 读取长期偏好 / 历史数据； – **Runtime Context** 读取配置（如 API key、DB 连接串等）。

从 State 读取会话信息

```
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent

@tool
def check_authentication(
    runtime: ToolRuntime,
) -> str:
    """检查用户是否已通过认证。"""
    # 从 State 读取当前认证状态
    current_state = runtime.state
    is_authenticated = current_state.get("authenticated", False)

    if is_authenticated:
        return "User is authenticated"
```

```

    else:
        return "User is not authenticated"

agent = create_agent(
    model="gpt-4o",
    tools=[check_authentication],
)

```

从 Store 读取用户偏好

```

from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:
    user_id: str

@tool
def get_preference(
    preference_key: str,
    runtime: ToolRuntime[Context],
) -> str:
    """从 Store 中获取用户偏好。"""
    user_id = runtime.context.user_id

    store = runtime.store
    existing_prefs = store.get(("preferences",), user_id)

    if existing_prefs:
        value = existing_prefs.value.get(preference_key)
        return f"{preference_key}: {value}" if value else f"No preference

```



```

    else:
        return "No preferences found"

agent = create_agent(
    model="gpt-4o",
    tools=[get_preference],
    context_schema=Context,
    store=InMemoryStore(),
)

```

从 Runtime Context 读取配置

```

from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent

@dataclass
class Context:
    user_id: str
    api_key: str
    db_connection: str

@tool
def fetch_user_data(
    query: str,
    runtime: ToolRuntime[Context],
) -> str:
    """使用运行时配置获取数据。"""
    user_id = runtime.context.user_id
    api_key = runtime.context.api_key
    db_connection = runtime.context.db_connection

    # 使用配置执行查询（伪代码）

```

```

        results = perform_database_query(db_connection, query, api_key)

        return f"Found {len(results)} results for user {user_id}"

agent = create_agent(
    model="gpt-4o",
    tools=[fetch_user_data],
    context_schema=Context,
)

# 调用时注入运行时上下文
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Get my data"}]},
    context=Context(
        user_id="user_123",
        api_key="sk-...",
        db_connection="postgresql://...",
    ),
)

```

Writes (写入)

工具结果不仅可以直接返回给模型，还可以用于： – 更新 state（会话内信息）； – 更新 store（跨会话长期信息）。

写入 State：使用 Command 更新会话状态

```

from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent
from langgraph.types import Command

@tool
def authenticate_user(
    password: str,

```

```

        runtime: ToolRuntime,
    ) -> Command:
        """验证用户并更新 State。"""
        # 这里用简单逻辑演示
        if password == "correct":
            # 写入 State：标记为已认证
            return Command(update={"authenticated": True})
        else:
            return Command(update={"authenticated": False})

agent = create_agent(
    model="gpt-4o",
    tools=[authenticate_user],
)

```

写入 Store：跨会话持久化数据

```

from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:
    user_id: str

@tool
def save_preference(
    preference_key: str,
    preference_value: str,
    runtime: ToolRuntime[Context],
) -> str:
    """将用户偏好存入 Store。"""

```

```

user_id = runtime.context.user_id

store = runtime.store
existing_prefs = store.get(("preferences",), user_id)

# 合并已有偏好
prefs = existing_prefs.value if existing_prefs else {}
prefs[preference_key] = preference_value

# 写入 Store：保存更新后的偏好
store.put(("preferences",), user_id, prefs)

return f"Saved preference: {preference_key} = {preference_value}"

agent = create_agent(
    model="gpt-4o",
    tools=[save_preference],
    context_schema=Context,
    store=InMemoryStore(),
)

```

更多关于在工具中访问 state、store 和 runtime context 的示例，可参考 **Tools** 文档。

生命周期上下文（Life-cycle context）

生命周期上下文控制在“核心智能体步骤之间”发生的事情，例如：

- 对话摘要 – 防护栏
- 日志 / 监控

如前文所述，中间件是实现生命周期上下文的关键机制。通过中间件，你可以：

1. **更新上下文（Update context）**：
2. 修改 state / store；
3. 更新消息历史；
4. 保存洞见或统计信息。
5. **跳转生命周期（Jump in the lifecycle）**：

6. 根据上下文跳转到不同步骤（例如：
 - 某条件满足时跳过工具执行；
 - 使用修改后的上下文重新调用模型）。

示例：摘要（Summarization）

最常见的生命周期模式之一，是在对话过长时自动压缩历史。与“仅对本次调用的消息做裁剪”的短暂处理不同，**摘要会持久更新 state**： – 用摘要永久替换旧消息； – 让后续轮次只看到压缩后的历史。

LangChain 提供了内置的 `SummarizationMiddleware`：

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware

agent = create_agent(
    model="gpt-4o",
    tools=[...],
    middleware=[
        SummarizationMiddleware(
            model="gpt-4o-mini",
            trigger={"tokens": 4000},      # tokens 超过 4000 时触发
            keep={"messages": 20},        # 保留最近 20 条消息
        ),
    ],
)
```

当对话超过 token 限制时，`SummarizationMiddleware` 会自动进行：1. 使用一个单独的 LLM 调用对较早的消息做摘要；2. 在 `State` 中用“摘要消息”替换原有旧消息（永久）；3. 最近的消息保持原样以维持上下文。

最佳实践（Best practices）

1. 从简单开始：
2. 先用静态 prompt 和固定工具集；

3. 确认原型可靠后再逐步增加动态上下文工程逻辑。
 4. **增量式测试**：
 5. 每次只引入一个新的上下文工程特性；
 6. 对比前后效果并记录。
 7. **监控性能**：
 8. 监控模型调用次数、token 用量和延迟；
 9. 使用 LangSmith 等工具做可观测性分析。
 10. **优先复用内置中间件**：
 11. 如 `SummarizationMiddleware`、工具选择中间件等；
 12. 在此基础上再扩展自定义逻辑。
 13. **清晰记录上下文策略**：
 14. 文档化“当前传给模型的上下文有哪些、为什么这样设计”；
 15. 方便团队协作与后续维护。
 16. **区分短暂与持久上下文**：
 17. 模型上下文的改变是“短暂”的（只影响当前调用）；
 18. 生命周期上下文的改变会持久写入 state / store。
-

相关资源 (Related resources)

- [Context conceptual overview](#) — 上下文类型与使用场景的概念性概览
 - [Middleware](#) — 中间件完整指南
 - [Tools](#) — 工具创建与上下文访问
 - [Memory](#) — 短期与长期记忆模式
 - [Agents](#) — 智能体核心概念
-

19. 模型上下文协议 (Model Context Protocol, MCP)

原文链接: <https://docs.langchain.com/oss/python/langchain/mcp>

Model Context Protocol (MCP) 是一个开放协议，用于标准化应用向 LLM 提供工具和上下文的方式。借助 `langchain-mcp-adapters` 库，LangChain 智能体可以使用定义在 MCP 服务器上的工具。

快速开始 (Quickstart)

安装 `langchain-mcp-adapters` :

```
pip install langchain-mcp-adapters
```

`langchain-mcp-adapters` 让智能体可以使用一个或多个 MCP 服务器上定义的工具。

`MultiServerMCPClient` 默认是无状态的：每次工具调用都会创建新的 MCP `ClientSession`，执行工具后清理会话。关于有状态会话的更多内容见后文“Stateful sessions”。

访问多个 MCP 服务器

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

client = MultiServerMCPClient(
    {
        "math": {
            "transport": "stdio", # 本地子进程通信
            "command": "python",
            # math_server.py 的绝对路径
        }
    }
)
```

```

        "args": ["/path/to/math_server.py"],
    },
    "weather": {
        "transport": "http", # 基于 HTTP 的远程服务器
        # 确保你的天气服务监听在 8000 端口
        "url": "http://localhost:8000/mcp",
    },
}

tools = await client.get_tools() # 从所有 MCP 服务器加载工具
agent = create_agent(
    "claude-sonnet-4-5-20250929",
    tools,
)

math_response = await agent.ainvoke(
    {"messages": [{"role": "user", "content": "what's (3 + 5) x 12?"}]}
)

weather_response = await agent.ainvoke(
    {"messages": [{"role": "user", "content": "what is the weather in ny"}]}
)

```

自定义服务器 (Custom servers)

要创建自定义 MCP 服务器，可以使用 FastMCP 库：

```
pip install fastmcp
```

可以使用如下示例测试你的智能体与 MCP 工具服务器的集成。

数学服务器 (stdio 传输)

```
from fastmcp import FastMCP

mcp = FastMCP("Math")

@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@mcp.tool()
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

传输 (Transports)

MCP 支持多种客户端-服务器通信传输方式。

HTTP

`http` 传输（也称为 `streamable-http`）使用 HTTP 请求进行通信。具体协议细节可见 MCP HTTP 传输规范。

```
client = MultiServerMCPClient(
    {
        "weather": {
```

```

        "transport": "http",
        "url": "http://localhost:8000/mcp",
    }
}
)

```

传递 headers (Passing headers)

连接到 HTTP MCP 服务器时，可以通过配置中的 `headers` 字段添加自定义请求头（例如认证或链路追踪）。这对 `sse`（已在 MCP 规范中弃用）和 `streamable_http` 传输都适用。

```

from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

client = MultiServerMCPClient(
    {
        "weather": {
            "transport": "http",
            "url": "http://localhost:8000/mcp",
            "headers": {
                "Authorization": "Bearer YOUR_TOKEN",
                "X-Custom-Header": "custom-value",
            },
        }
    }
)

tools = await client.get_tools()
agent = create_agent("openai:gpt-4.1", tools)
response = await agent.ainvoke({"messages": "what is the weather in nyc?"})

```

认证 (Authentication)

`langchain-mcp-adapters` 库在底层使用官方 MCP SDK，你可以通过实现 `httpx.Auth` 接口来提供自定义认证机制：

```
from langchain_mcp_adapters.client import MultiServerMCPClient

client = MultiServerMCPClient(
    {
        "weather": {
            "transport": "http",
            "url": "http://localhost:8000/mcp",
            "auth": auth, # 自定义 httpx.Auth 实例
        }
    }
)
```

你可以实现自定义认证类，也可以使用内置 OAuth 流程。

stdio

在 `stdio` 模式下，客户端以子进程方式启动服务器，并通过标准输入/输出通信。适合本地工具和简单部署。

与 HTTP 传输不同，`stdio` 连接在进程层面是 **有状态的**——子进程会在客户端连接生命周期内持续存在。但如果只使用默认的 `MultiServerMCPClient` 而不做显式会话管理，每次工具调用仍然会创建新会话。关于持久会话管理见下节。

```
client = MultiServerMCPClient(
    {
        "math": {
            "transport": "stdio",
            "command": "python",
            "args": ["/path/to/math_server.py"],
        }
    }
)
```

```
}  
)
```

有状态会话 (Stateful sessions)

默认情况下, `MultiServerMCPClient` 是 **无状态** 的: – 每次工具调用都会创建新的 MCP 会话; – 执行工具后立即清理。

如果你需要控制 MCP 会话的生命周期 (例如服务器在多次调用间保持上下文), 可以通过 `client.session()` 创建持久化的 `ClientSession`:

```
from langchain_mcp_adapters.client import MultiServerMCPClient  
from langchain_mcp_adapters.tools import load_mcp_tools  
from langchain.agents import create_agent  
  
client = MultiServerMCPClient({...})  
  
# 显式创建会话  
async with client.session("server_name") as session:  
    # 使用该会话加载工具、资源或提示  
    tools = await load_mcp_tools(session)  
    agent = create_agent(  
        "anthropic:claude-3-7-sonnet-latest",  
        tools,  
    )
```

核心特性 (Core features)

工具 (Tools)

MCP 服务器通过工具向 LLM 暴露可执行函数, 例如: – 查询数据库 – 调用外部 API – 与外部系统交互

LangChain 会将 MCP 工具转换为 LangChain 工具，从而可以直接在任意 LangChain 智能体或工作流中使用。

加载工具 (Loading tools)

使用 `client.get_tools()` 从 MCP 服务器加载工具并传给智能体：

```
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

client = MultiServerMCPClient({...})
tools = await client.get_tools()
agent = create_agent("claude-sonnet-4-5-20250929", tools)
```

结构化内容 (Structured content)

MCP 工具可以在返回给模型的人类可读文本之外，同时返回结构化内容（如 JSON）。

- 当 MCP 工具返回 `structuredContent` 时，适配器会将其包装为 `MCPToolArtifact`，并作为工具的 `artifact` 暴露；
- 你可以在 `ToolMessage` 上通过 `artifact` 字段访问它；
- 也可以通过拦截器自动处理或转换结构化内容。

（具体示例略，可在需要时参考原文档。）

资源 (Resources) 与提示 (Prompts)

MCP 还支持： – **Resources**：如文档、文件等可供模型访问的资源； – **Prompts**：由 MCP 服务器提供的可重用提示模板。

通过 `load_mcp_tools` 及相关 API，可以一并加载工具、资源与提示，并在 LangChain 智能体中使用。

高级特性 (Advanced features)

工具拦截器 (Tool interceptors)

工具拦截器允许你： – 在 MCP 工具调用前后插入自定义逻辑； – 记录日志、修改请求 / 响应、重试失败调用等。

访问运行时上下文 (Accessing runtime context)

在拦截器中可以访问 `MCPToolCallRequest`，并查看： – 工具名 – 参数 – 服务器信息等

状态更新与命令 (State updates and commands)

你也可以结合 LangGraph 的 `Command`、状态更新等机制，在 MCP 工具调用完成后更新智能体的 state / store。

自定义拦截器 (Custom interceptors)

拦截器函数通常签名类似：

```
async def interceptor(request: MCPToolCallRequest, handler):  
    # 调用前逻辑  
    result = await handler(request)  
    # 调用后逻辑  
    return result
```

将拦截器注册到 `MultiServerMCPClient`：

```
client = MultiServerMCPClient(  
    {...},  
    tool_interceptors=[logging_interceptor], # 自定义拦截器列表  
)
```

修改请求 (Modifying requests)

使用 `request.override()` 创建修改后的请求，遵循不可变模式，原始请求不会被改变。

示例：修改工具参数

```
async def double_args_interceptor(
    request: MCPToolCallRequest,
    handler,
):
    """在执行前将所有数值参数翻倍。"""
    modified_args = {k: v * 2 for k, v in request.args.items()}
    modified_request = request.override(args=modified_args)
    return await handler(modified_request)

# 原始调用：add(a=2, b=3) 会变为 add(a=4, b=6)
```

示例：在运行时修改 HTTP headers

```
async def auth_header_interceptor(
    request: MCPToolCallRequest,
    handler,
):
    """根据工具名称动态添加认证头。"""
    token = get_token_for_tool(request.name)
    modified_request = request.override(
        headers={"Authorization": f"Bearer {token}"}
    )
    return await handler(modified_request)
```

组合拦截器 (Composing interceptors)

多个拦截器按“洋葱模型”顺序组合：拦截器列表中第一个是最外层。

```
async def outer_interceptor(request, handler):
    print("outer: before")
    result = await handler(request)
    print("outer: after")
    return result
```

```
async def inner_interceptor(request, handler):
    print("inner: before")
    result = await handler(request)
    print("inner: after")
    return result
```

```
client = MultiServerMCPClient(
    {...},
    tool_interceptors=[outer_interceptor, inner_interceptor],
)
```

执行顺序：

outer: before -> inner: before -> 工具执行 -> inner: after -> outer: after

错误处理 (Error handling)

拦截器可以捕获工具执行错误并实现重试逻辑：

```
import asyncio

async def retry_interceptor(
    request: MCPToolCallRequest,
    handler,
    max_retries: int = 3,
    delay: float = 1.0,
):
    """在错误时按指数退避重试工具调用。"""
```



```

last_error = None
for attempt in range(max_retries):
    try:
        return await handler(request)
    except Exception as e:
        last_error = e
        if attempt < max_retries - 1:
            wait_time = delay * (2 ** attempt)
            print(f"Tool {request.name} failed (attempt {attempt + 1}). Retrying in {wait_time} seconds.")
            await asyncio.sleep(wait_time)
        raise last_error

client = MultiServerMCPClient(
    {...},
    tool_interceptors=[retry_interceptor],
)

```

也可以对特定错误做回退处理：

```

async def fallback_interceptor(
    request: MCPToolCallRequest,
    handler,
):
    """在工具执行失败时返回回退结果。"""
    try:
        return await handler(request)
    except TimeoutError:
        return f"Tool {request.name} timed out. Please try again later."
    except ConnectionError:
        return f"Could not connect to {request.name} service. Using cached results."

```

进度通知 (Progress notifications)

可以订阅长时间运行工具的进度更新：

```

from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain_mcp_adapters.callbacks import Callbacks, CallbackContext

async def on_progress(
    progress: float,
    total: float | None,
    message: str | None,
    context: CallbackContext,
):
    """处理来自 MCP 服务器的进度更新。"""
    percent = (progress / total * 100) if total else progress
    tool_info = f" ({context.tool_name})" if context.tool_name else ""
    print(f"[{context.server_name}{tool_info}] Progress: {percent:.1f}%")

client = MultiServerMCPClient(
    {...},
    callbacks=Callbacks(on_progress=on_progress),
)

```

`CallbackContext` 提供： – `server_name`：MCP 服务器名称 – `tool_name`：正在执行的工具名称（仅在工具调用期间可用）

日志 (Logging)

MCP 协议支持服务器端日志通知。可以通过 `Callbacks` 订阅这些事件：

```

from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain_mcp_adapters.callbacks import Callbacks, CallbackContext
from mcp.types import LoggingMessageNotificationParams

async def on_logging_message(
    params: LoggingMessageNotificationParams,
    context: CallbackContext,
)

```

```

):
    """处理来自 MCP 服务器的日志消息。"""
    print(f"[{context.server_name}] {params.level}: {params.data}")

client = MultiServerMCPClient(
    {...},
    callbacks=Callbacks(on_logging_message=on_logging_message),
)

```

引导式补充输入 (Elicitation)

Elicitation 允许 MCP 服务器在工具执行期间向用户请求额外输入，而不是一开始就要求所有参数。服务器可以按需交互式询问信息。

服务器端示例 (Server setup)

```

from pydantic import BaseModel
from mcp.server.fastmcp import Context, FastMCP

server = FastMCP("Profile")

class UserDetails(BaseModel):
    email: str
    age: int

@server.tool()
async def create_profile(name: str, ctx: Context) -> str:
    """创建用户档案，并通过引导式提问获取详情。"""
    result = await ctx.elicit(
        message=f"Please provide details for {name}'s profile:",
        schema=UserDetails,
    )

```

```

    if result.action == "accept" and result.data:
        return f"Created profile for {name}: email={result.data.email}, "
    if result.action == "decline":
        return f"User declined. Created minimal profile for {name}."
    return "Profile creation cancelled."

if __name__ == "__main__":
    server.run(transport="http")

```

客户端示例 (Client setup)

```

from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain_mcp_adapters.callbacks import Callbacks, CallbackContext
from mcp.shared.context import RequestContext
from mcp.types import ElicitRequestParams, ElicitResult

async def on_elicitation(
    mcp_context: RequestContext,
    params: ElicitRequestParams,
    context: CallbackContext,
) -> ElicitResult:
    """处理来自 MCP 服务器的引导式输入请求。"""
    # 在真实应用中，你会根据 params.message 和 params.requestedSchema
    # 提示真实用户输入。这里用固定值示例。
    return ElicitResult(
        action="accept",
        content={"email": "[email protected]", "age": 25},
    )

client = MultiServerMCPClient(
    {
        "profile": {
            "url": "http://localhost:8000/mcp",

```

```
        "transport": "http",
    },
},
callbacks=Callbacks(on_elicitation=on_elicitation),
)
```

响应动作 (Response actions)

引导式回调可以返回三种动作 (action)：

Action	描述
accept	用户提供了有效输入，数据放在 <code>content</code> 字段中
decline	用户选择不提供请求的信息
cancel	用户取消整个操作

示例：

```
# 接受并带有数据
ElicitResult(action="accept", content={"email": "[email protected]", "age": 30})

# 拒绝（用户不想提供信息）
ElicitResult(action="decline")

# 取消（终止操作）
ElicitResult(action="cancel")
```

更多资源 (Additional resources)

- [MCP documentation](#) — MCP 协议完整文档
 - [MCP Transport documentation](#) — MCP 传输方式详解
 - [langchain-mcp-adapters](#) — LangChain MCP 适配器库文档
-

\newpage

20. 人在回路 (Human-in-the-loop, HITL)

原文链接: <https://docs.langchain.com/oss/python/langchain/human-in-the-loop>

Human-in-the-Loop (HITL) 中间件允许你在智能体的工具调用上加入人工监督。当模型提出的动作可能需要人工复核时（例如写文件、执行 SQL），中间件可以暂停执行并等待决策。

它的工作方式是：

- 对每次工具调用，根据可配置的策略进行检查；
- 如果需要干预，则发出一次“中断 (interrupt)”并暂停执行；
- 使用 LangGraph 的持久化层保存图状态，确保执行可以安全暂停并在之后恢复；
- 人工给出决策后，再决定下一步是：
- 原样执行（`approve`），
- 修改后执行（`edit`），
- 或拒绝并给出反馈（`reject`）。

中断决策类型 (Interrupt decision types)

中间件内置三种决策类型来响应一次中断：

决策类型	描述	示例用法
 <code>approve</code>	按原样批准该动作并直接执行	将邮件草稿原样发送
 <code>edit</code>	对工具调用进行修改后再执行	修改收件人后再发送邮件
 <code>reject</code>	拒绝执行该工具调用，并在对话中附加解释性反馈	拒绝邮件草稿，并说明应该如何重写

每个工具可允许的决策类型由 `interrupt_on` 配置决定。当多个工具调用同时被暂停时，每个动作都需要单独决策，且决策顺序必须与中断请求中的动作顺序一致。

在 编辑 (`edit`) 工具参数时，建议尽量“保守修改”。如果修改过大，模型可能会重新评估整个方案，从而多次执行工具或采取意外操作。

配置中断 (Configuring interrupts)

在创建智能体时，将 HITL 中间件加入 `middleware` 列表即可。通过 `interrupt_on` 指定哪些工具需要人工干预，以及允许的决策类型。

```
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model="gpt-4o",
    tools=[write_file_tool, execute_sql_tool, read_data_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "write_file": True, # 允许 approve / edit / reject
                "execute_sql": {"allowed_decisions": ["approve", "reject"]},
                # 安全操作，无需审批
                "read_data": False,
            },
            # 中断消息前缀，会与工具名和参数组合成完整描述
            # 例如："Tool execution pending approval: execute_sql with query ..."
            # 单个工具可以通过在配置中设置 "description" 覆盖默认描述
            description_prefix="Tool execution pending approval",
        ),
    ],
    # 人在回路依赖检查点来处理中断。
    # 生产环境推荐使用持久化 checkpointer (如 AsyncPostgresSaver) ,
    # 这里示例使用内存版本 InMemorySaver。
    checkpointer=InMemorySaver(),
)
```

你必须配置一个 checkpointer 才能在中断间持久化图状态。生产环境应使用持久化 checkpointer (例如 AsyncPostgresSaver) ，而在测试或原型阶段可以使用 `InMemorySaver` 。

调用智能体时，需要在 `config` 中提供 `thread_id`，用来将执行与某个对话线程绑定。更多细节参见 `LangGraph interrupts` 文档。

配置选项：

- `interrupt_on`（必填）
 - 工具名到审批配置的映射。
- 值可以是：
 - `True`：使用默认配置中断；
 - `False`：自动批准；
 - `InterruptOnConfig` 对象：精细化配置。
- `description_prefix`（字符串，默认 `"Tool execution requires approval"`）
 - 用作中断请求描述的前缀。

`InterruptOnConfig` 选项：

- `allowed_decisions`（字符串列表）
 - 允许的决策类型：`"approve"`、`"edit"`、`"reject"`。
- `description`（字符串或可调用）
 - 静态描述或返回描述的函数，用于自定义某个工具的中断说明。

响应中断（Responding to interrupts）

调用智能体时，它会运行直到： – 正常完成； – 或触发中断。

当某个工具调用匹配 `interrupt_on` 策略时，本次调用结果中会包含一个 `__interrupt__` 字段，描述所有待审核的动作。你可以： – 将这些动作展示给审核人； – 根据人工决策，再调用智能体恢复执行。

```
from langgraph.types import Command
```

```
# 人在回路依赖 LangGraph 的持久化层。
```



```

# 必须提供 thread_id, 将执行与某个对话线程关联,
# 以便在人工审核时可以暂停 / 恢复。
config = {"configurable": {"thread_id": "some_id"}}

# 运行图直到遇到中断
result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "Delete old records from the database",
            }
        ]
    },
    config=config,
)

# 中断对象包含完整的 HITL 请求, 包括 action_requests 和 review_configs
print(result["__interrupt__"])
# > [
# >     Interrupt(
# >         value={
# >             'action_requests': [
# >                 {
# >                     'name': 'execute_sql',
# >                     'arguments': {'query': "DELETE FROM records WHERE cre
# >                     'description': 'Tool execution pending approval\n\nTo
# >                 }
# >             ],
# >             'review_configs': [
# >                 {
# >                     'action_name': 'execute_sql',
# >                     'allowed_decisions': ['approve', 'reject']
# >                 }
# >             ]
# >         }
# >     )

```

```
# > ]
```

```
# 通过审批决策恢复执行
```

```
agent.invoke(  
    Command(  
        resume={"decisions": [{"type": "approve"}]}, # 或 "reject"  
    ),  
    config=config, # 使用相同 thread_id 恢复暂停的对话  
)
```

决策类型 (Decision types)

-  approve
-  edit
-  reject

approve : 直接批准

使用 `approve` 按原样批准工具调用并执行：

```
agent.invoke(  
    Command(  
        # 决策以列表形式提供，每个待审核动作对应一个决策。  
        # 决策顺序必须与 `__interrupt__` 请求中的 action 顺序一致。  
        resume={  
            "decisions": [  
                {  
                    "type": "approve",  
                }  
            ],  
        },  
    ),
```

```
        config=config, # 使用相同 thread_id 恢复
    )
```

edit：修改后执行

使用 `edit` 在执行前修改工具调用。需要在 `edited_action` 字段中提供新的工具名与参数：

```
agent.invoke(
    Command(
        resume={
            "decisions": [
                {
                    "type": "edit",
                    # 修改后的动作（工具名与参数）
                    "edited_action": {
                        # 要调用的工具名（通常与原动作一致）
                        "name": "new_tool_name",
                        # 传递给工具的新参数
                        "args": {"key1": "new_value", "key2": "original_value"}
                    },
                },
            ],
        },
    ),
    config=config,
)
```

再次提醒：编辑参数时务必保守，过大改动可能让模型重新规划，从而多次调用工具或产生不可预期行为。

reject：拒绝并提供反馈

使用 `reject` 拒绝工具调用，并给出一段解释：

```

agent.invoke(
  Command(
    resume={
      "decisions": [
        {
          "type": "reject",
          # 说明为什么拒绝, 以及应当如何改写
          "message": "No, this is wrong because ..., instead do ..."
        }
      ]
    },
    config=config,
  )
)

```

message 内容会作为反馈加入对话, 帮助智能体理解该动作为何被拒绝, 以及之后应如何调整行为。

多个决策 (Multiple decisions)

当一次中断中包含多个待审核动作时, 需要按顺序为每个动作提供决策:

```

{
  "decisions": [
    {"type": "approve"},
    {
      "type": "edit",
      "edited_action": {
        "name": "tool_name",
        "args": {"param": "new_value"}
      }
    },
    {
      "type": "reject",
      "message": "This action is not allowed"
    }
  ]
}

```

```
]
}
```

与流式处理结合使用 (Streaming with human-in-the-loop)

你可以使用 `stream()` 替代 `invoke()`，在智能体运行和处理中断的同时获得实时更新。通过 `stream_mode=["updates", "messages"]` 同时流式传输： – 智能体进度 (updates) – LLM 令牌 (messages)

```
from langgraph.types import Command

config = {"configurable": {"thread_id": "some_id"}}

# 流式输出智能体进度与 LLM 令牌，直到遇到中断
for mode, chunk in agent.stream(
    {"messages": [{"role": "user", "content": "Delete old records from the database"}]},
    config=config,
    stream_mode=["updates", "messages"],
):
    if mode == "messages":
        # LLM token
        token, metadata = chunk
        if token.content:
            print(token.content, end="", flush=True)
    elif mode == "updates":
        # 检查是否有中断
        if "__interrupt__" in chunk:
            print(f"\n\nInterrupt: {chunk['__interrupt__']}")

# 在人工决策后继续以流式方式恢复执行
for mode, chunk in agent.stream(
    Command(resume={"decisions": [{"type": "approve"}]}),
    config=config,
```

```
stream_mode=["updates", "messages"],
):
    if mode == "messages":
        token, metadata = chunk
        if token.content:
            print(token.content, end="", flush=True)
```

更多关于流式模式的细节，参见 Streaming 指南。

执行生命周期 (Execution lifecycle)

HITL 中间件在模型生成响应后、工具调用执行前，通过一个 `after_model` 钩子介入：

1. 智能体调用模型生成响应；
 2. 中间件检查响应中的工具调用；
 3. 对于需要人工输入的工具调用，中间件构造 `HITLRequest`（包含 `action_requests` 与 `review_configs`），并触发中断；
 4. 智能体等待人工决策；
 5. 根据 `HITLResponse` 的决策；
 6. 执行被批准或编辑后的工具调用；
 7. 为被拒绝的调用合成对应的 `ToolMessage`；
 8. 然后继续恢复执行流程。
-

自定义 HITL 逻辑 (Custom HITL logic)

对于更复杂、定制化的工作流，你可以直接使用“中断原语 (interrupt primitive)”和中间件抽象构建自己的 HITL 逻辑。

建议先理解上文的执行生命周期，再根据业务需求在： – 模型前 / 后； – 工具前 / 后； – 或更高层级的控制流

中植入你自己的审核与决策流程。

21. 多智能体 (Multi-agent)

原文链接: <https://docs.langchain.com/oss/python/langchain/multi-agent/index>

多智能体系统通过协调多个“专长组件”来处理复杂工作流。但并不是所有复杂任务都需要多智能体——一个配置合理（有时是动态工具 + 精心设计 prompt）的单智能体，往往也能达到类似效果。

为什么需要多智能体？ (Why multi-agent?)

当开发者说“需要多智能体”时，通常是在追求以下一种或多种能力：

- **上下文管理 (Context management)：**
 - 在不“撑爆”模型上下文窗口的前提下提供专业知识；
 - 如果上下文无限大且延迟为 0，那么可以直接把所有知识都塞进一个 prompt；
 - 现实中做不到，因此需要模式化的方式，按需暴露相关信息。
- **分布式开发 (Distributed development)：**
 - 允许不同团队独立开发与维护各自的能力模块；
 - 再将它们组合成一个边界清晰的大系统。
- **并行化 (Parallelization)：**
 - 为子任务启动“专门工人”，并发执行以更快得到结果。

多智能体模式在以下场景特别有价值：

- 单一智能体拥有**过多工具**，导致工具选择不佳；
- 任务需要专业知识和大量上下文（长 prompt + 领域特定工具）；
- 需要强约束的顺序流程，某些能力只有在满足条件后才被“解锁”。

多智能体设计的核心始终是 **上下文工程 (context engineering)** —— 决定**每个智能体能看到什么信息**。系统质量取决于： – 是否保证每个智能体只获取完成其任务所需的“正确数据”。

模式（Patterns）

构建多智能体系统时，常用的模式主要有以下几种，各自适用不同场景：

模式	工作方式说明
Subagents	主智能体作为协调者，将子智能体当作“工具”调用。所有路由都通过主智能体，由它决定何时、如何调用各个子智能体。
Handoffs	行为会根据状态动态变化。工具调用可以更新状态变量，从而触发路由或配置变化，切换智能体或调整当前智能体的工具 / prompt。
Skills	按需加载“技能”：专门的 prompt 与知识。始终只有“一个智能体”在控制全局，只是根据需要加载不同技能作为上下文。
Router	先有一个“路由步骤”，对输入做分类，再将其分发给一个或多个专门智能体。结果最后被汇总成一个综合响应。
Custom workflow	使用 LangGraph 构建自定义执行流，将确定性逻辑和智能体行为混合在一起；可以将其他模式嵌为节点。

如何选择模式（Choosing a pattern）

下面的表格可以帮你根据需求选模式：

模式	分布式开发	并行化	多跳调用	直接与用户交互
Subagents	★★★★★	★★★★★	★★★★★	★
Handoffs	—	—	★★★★★	★★★★★
Skills	★★★★★	★★★	★★★★★	★★★★★
Router	★★★	★★★★★	—	★★★

- 分布式开发：不同团队是否能独立维护各组件？
- 并行化：是否支持多个智能体并发执行？
- 多跳调用（Multi-hop）：是否支持串联调用多个子智能体？
- 直接与用户交互：子智能体是否能直接与用户对话？

可以混用模式！

例如： – 一个 **subagents** 架构中的工具，可以再调用自定义 workflow 或 router 智能体； – 子智能体自身也可以使用 **skills** 模式按需加载上下文。

组合空间非常灵活。

可视化概览 (Visual overview)

Subagents

- 主智能体协调子智能体，并将它们当作工具调用；
- 所有路由都经过主智能体。

Subagents 模式：主智能体协调若干子智能体作为工具。

Handoffs

- 智能体之间通过工具调用“移交控制权”；
- 每个智能体可以：
- 把控制权交给其他智能体，
- 或直接给用户回复。

Handoffs 模式：智能体之间通过工具调用进行控制权交接。

Skills

- 始终只有一个“控制智能体”；
- 它会在需要时加载专门的 prompt / 文档 / 能力作为“技能”，并纳入上下文；
- 不需要多个智能体实体。

Skills 模式：单智能体按需加载专门上下文。

Router

- 先经过“路由步骤”对输入进行分类；
- 再将输入分发给一个或多个专门智能体；
- 最后将结果综合为一个统一响应。

Router 模式：路由步骤负责把输入分发给专门智能体。

性能对比（Performance comparison）

不同模式在性能上的表现不同。理解这些差异有助于根据延迟与成本要求选择合适的模式。

关键指标：

- **Model calls**：LLM 调用次数。 – 调用越多 → 延迟越高（尤其是串行调用时） → API 成本越高；
- **Tokens processed**：所有调用中处理的 token 总数。 – tokens 越多 → 成本越高，更容易触发上下文上限。

单次请求（One-shot request）

用户：“Buy coffee”

一个专门的“咖啡智能体 / 技能”可以调用 `buy_coffee` 工具。

模式	模型调用数	最佳选择
Subagents	4	
Handoffs	3	✓
Skills	3	✓
Router	3	✓

要点：

- Handoffs / Skills / Router：单任务时都只需 **3 次调用**；
- Subagents：多 1 次调用，因为结果要回流到主智能体做统一协调；
- 这增加了开销，但也提供了更强的集中控制能力。

重复请求（Repeat request）

第 1 轮：“Buy coffee”

第 2 轮：“Buy coffee again”

用户在同一个对话里重复相同请求。

模式	第 2 轮调用数	总调用数（两轮）	最佳选择
Subagents	4	8	
Handoffs	2	5	✓
Skills	2	5	✓
Router	3	6	

直观理解：

- **Subagents（8 调用）：**
 - 设计上是“无状态”的：每次调用都走完整流程；
 - 主智能体保持对话上下文，但子智能体每次从头开始；
 - 提供了很强的上下文隔离，但会重复消耗调用次数。
- **Handoffs（5 调用）：**
 - 第 1 轮完成后，“咖啡智能体”的状态仍然存在；
 - 第 2 轮不再需要 handoff，直接调用 `buy_coffee` 工具 + 回复用户 → 2 次调用；
 - 相比每轮都走完整流程，节省了大约 40% 的调用。
- **Skills（5 调用）：**
 - 第 1 轮中已加载过 coffee skill 的上下文；
 - 第 2 轮直接复用已加载的 skill，上下文仍在对话中；
 - 与 Handoffs 类似，也节省了调用次数。
- **Router（6 调用）：**
 - Router 自身是“无状态”的，每次请求都需 LLM 做一次路由；
 - 可通过在上层再包一层 stateful agent 来优化。

多领域任务（Multi-domain）

用户：“Compare Python, JavaScript, and Rust for web development”

每种语言智能体 / 技能都带有约 2000 tokens 的文档。假设所有模式都可以并行调用工具。

模式	调用数	总 tokens	最佳选择
Subagents	5	~9K	✓
Handoffs	7+	~14K+	
Skills	3	~15K	
Router	5	~9K	✓

简要对比：

- **Subagents (5 调用, ~9K tokens) :**
 - 每个子智能体在自己的上下文中独立工作；
 - 只加载与该子任务相关的文档；
 - 总 token 使用量较低。
- **Handoffs (7+ 调用, ~14K+ tokens) :**
 - 执行是串行的，无法并行查询三种语言；
 - 随着对话历史增长，token 消耗也增加。
- **Skills (3 调用, ~15K tokens) :**
 - 调用次数少，但上下文会不断“累积”：
 - 一旦加载了 3 个技能，对后续每次调用都要处理所有 6K tokens 文档；
 - 总 token 使用反而更高。
- **Router (5 调用, ~9K tokens) :**
 - 通过 LLM 路由，再并行调用各领域智能体；
 - token 使用与 Subagents 类似，但多了显式路由步骤。

关键结论：

- 多领域任务更适合能并行执行的模式 (Subagents、Router) ；
- Skills 调用次数少，但因上下文累计导致 token 成本偏高；

- Handoffs 在这里相对低效：不能并行，只能顺序调用。

总结 (Summary)

三种场景下各模式的整体对比：

模式	单次请求	重复请求总调用	多领域场景
Subagents	4 调用	8 调用 (4+4)	5 调用，约 9K tokens
Handoffs	3 调用	5 调用 (3+2)	7+ 调用，约 14K+ tokens
Skills	3 调用	5 调用 (3+2)	3 调用，约 15K tokens
Router	3 调用	6 调用 (3+3)	5 调用，约 9K tokens

如何按目标选择模式 (Choosing a pattern)

优化目标	Subagents	Handoffs	Skills	Router
单次请求	✓	✓	✓	
重复请求	✓ (成本稳定、强隔离)	✓ (充分复用已有上下文)		
并行执行	✓	✓ (在某些设计下可混用)		✓
大上下文领域	✓ (上下文隔离，每个子智能体只看相关内容)	✓		✓
简单且聚焦的任务	✓			

实战建议：

- 如果你一开始不确定是否需要多智能体，先从单智能体 + 良好上下文工程开始；
- 当出现工具太多、领域太多、需要并行或强隔离时，再逐步引入相应的多智能体模式；

- 混合使用 Subagents / Handoffs / Skills / Router 往往能在灵活性、成本和可维护性之间取得最好平衡。

本文档由 LangChain 官方文档翻译而来

\newpage

22. 多智能体模式：子智能体（Subagents）

原文链接: <https://docs.langchain.com/oss/python/langchain/multi-agent/subagents>

在 **Subagents** 架构中，一个中心的主智能体（通常称为 **supervisor / 协调者**）通过将其他智能体当作“工具”来进行协调：

- 主智能体决定调用哪个子智能体、传入什么输入、以及如何组合子智能体的结果；
- 子智能体本身是**无状态的**——它们不记住过去的交互；
- 所有对话记忆都由主智能体维护；
- 每次子智能体调用都在一个“干净的”上下文窗口中执行，从而避免主对话上下文膨胀（context bloat）。

关键特性（Key characteristics）

- **集中控制**：所有路由都经过主智能体；
- **不直接与用户交互**：子智能体的结果会返回给主智能体，而不是直接给用户（不过在子智能体内部也可以通过中断（interrupt）与用户交互）；
- **子智能体通过工具暴露**：子智能体以“工具”的形式被调用；
- **支持并行执行**：主智能体在单轮中可以调用多个子智能体。

Supervisor vs. Router： – Supervisor（本模式）是一个完整的智能体，会维护对话上下文，并在多轮对话中动态决定要调用哪些子智能体； – Router 通常只是一个“单次分类步骤”，只负责把输入分发给某个智能体，而不会维持持续的对话状态。

何时使用（When to use）

在以下场景适合使用 Subagents 模式：

- 存在多个**清晰不同的领域**（如日历、邮箱、CRM、数据库）；
- 子智能体**不需要**直接与用户对话；
- 你希望在一个中心化的地方把控整个 workflow。

若只是少量工具、逻辑较简单，通常一个智能体即可，不必上多智能体模式。

子智能体内部需要与用户交互？

虽然默认子智能体只把结果返回主智能体，而不直接与用户对话，但你可以在子智能体内部使用“中断（interrupt）”机制来暂停执行、向用户提问或请求审批。此时：

- 主智能体依然是整体协调者；
- 子智能体在执行任务中途也可以向用户收集额外信息。

基本实现（Basic implementation）

核心机制是：把子智能体包装为一个工具，供主智能体调用：

```
from langchain.tools import tool
from langchain.agents import create_agent

# 创建一个子智能体
subagent = create_agent(model="anthropic:claude-sonnet-4-20250514", tools=[])

# 将子智能体包装为工具
@tool("research", description="Research a topic and return findings")
def call_research_agent(query: str):
    result = subagent.invoke({"messages": [{"role": "user", "content": query}]}).content
    return result

# 主智能体，将子智能体工具纳入 tools 列表
main_agent = create_agent(model="anthropic:claude-sonnet-4-20250514", tools=[call_research_agent])
```

设计决策（Design decisions）

实现 Subagents 模式时，你需要做一些关键设计选择。下表对这些选项做了概览，后续小节会分别展开：

决策点	选项
同步 vs. 异步	同步（阻塞） vs. 异步（后台任务）
工具模式	每个子智能体一个工具 vs. 单一调度工具
子智能体输入	仅 query vs. 完整上下文
子智能体输出	只返回结果 vs. 返回完整对话历史

同步 vs. 异步 (Sync vs. async)

子智能体执行可以是： – **同步 (Sync)**：主智能体会等待子智能体完成； – **异步 (Async)**：子智能体在后台运行，主智能体继续与用户交互。

这与 Python 的 `async/await` 概念不同：这里的“异步”指的是在**架构层面**不阻塞主对话。

模式	主智能体行为	适用场景	取舍
Sync	等待子智能体完成	主智能体接下来需要子智能体结果	简单直观，但会阻塞对话
Async	在子智能体后台运行时继续对话	子任务独立、用户不必等待任务完成	更灵活，但实现更复杂

同步（默认） (Synchronous, default)

默认情况下，子智能体调用是同步的： – 主智能体在每次子调用完成前不会继续下一步； – 适合： – 主智能体需要子结果来生成回复； – 任务存在强顺序依赖（如“取数 → 分析 → 回复”）； – 子智能体失败时，需要阻止主智能体继续回复。

权衡： – 实现简单，调用即等待结果； – 用户要等所有子任务完成后才能看到结果； – 长耗时子任务会“冻结对话”。

异步 (Asynchronous)

当子任务与主对话相对独立时，可以采用异步执行模式： – 主智能体只负责启动后台任务，然后可以继续与用户对话； – 适合： – 子任务与当前对话流程关系较弱； – 希望用户在后台任务执行时仍能继续聊天； – 需要对多个独立任务进行并行处理。

典型“三工具模式”： 1. **Start job**：启动后台任务，返回 job ID； 2. **Check status**：根据 job ID 查询状态 (pending / running / completed / failed) ； 3. **Get result**：在任务完成后拉取结果。

任务完成后，你的应用通常需要向用户发出通知，例如： – UI 中弹出通知； – 用户点击后发送一条 `HumanMessage` ： `"Check job_123 and summarize the results"` 。

工具模式 (Tool patterns)

将子智能体作为工具暴露时，常见有两种方式：

模式	适用场景	权衡
每个智能体一个工具	需要对每个子智能体的输入 / 输出做细粒度控制	设置较多，但可高度定制
单一调度工具	智能体数量多、团队分布式开发、偏好“约定优于配置”	组合简单，但对每个子智能体的上下文工程控制较弱

每个智能体一个工具 (Tool per agent)

核心思路：将每个子智能体分别包装为一个工具，主智能体按需调用：

```
from langchain.tools import tool
from langchain.agents import create_agent

# 创建一个子智能体
subagent = create_agent(model="...", tools=[...])

# 包装为工具
```

```
@tool("subagent_name", description="subagent_description")
def call_subagent(query: str):
    result = subagent.invoke({"messages": [{"role": "user", "content": q
    return result["messages"][-1].content

# 主智能体，接入该工具
main_agent = create_agent(model="...", tools=[call_subagent])
```

主智能体： – 判断任务是否匹配子智能体能力； – 决定何时调用该工具； – 接收返回结果，继续进行编排。

更高级的上下文控制手法可以配合“Context engineering”一起使用。

单一调度工具 (Single dispatch tool)

另一种方式：使用一个参数化工具来调度多个“短生命周期子智能体” (ephemeral sub-agents)：

- 与“每个子智能体一个工具”不同，这里只定义一个 `task` 工具；
- 通过参数 `agent_name` 指定要调用哪个子智能体；
- 子智能体接收到的内容是一个用户消息 (description)，最终消息作为工具结果返回。

适用场景： – 希望多个团队独立开发 / 部署各自的智能体能力； – 需要把复杂任务放到独立的上下文窗口中执行； – 希望在不修改协调者代码的情况下新增智能体； – 更偏好“约定优于配置”的模式。

这种模式牺牲了部分上下文工程的灵活性，但带来了： – 更简单的组合方式； – 更强的上下文隔离（每个任务在自己的干净窗口中执行）。

一个有意思的点： – 子智能体的能力可以与主智能体完全相同； – 此时调用子智能体的主要目的是 **上下文隔离**： – 让复杂、多步任务在独立窗口中完成； – 最终只返回精简总结，保持主对话历史的干净与高效。

示例：使用任务调度工具与智能体注册表 (Agent registry + task dispatcher)

```
from langchain.tools import tool
from langchain.agents import create_agent
```

```

# 由不同团队开发的子智能体
research_agent = create_agent(
    model="gpt-4o",
    prompt="You are a research specialist...",
)

writer_agent = create_agent(
    model="gpt-4o",
    prompt="You are a writing specialist...",
)

# 子智能体注册表
SUBAGENTS = {
    "research": research_agent,
    "writer": writer_agent,
}

@tool
def task(
    agent_name: str,
    description: str,
) -> str:
    """为任务启动临时子智能体。

    Available agents:
    - research: Research and fact-finding
    - writer: Content creation and editing
    """
    agent = SUBAGENTS[agent_name]
    result = agent.invoke({
        "messages": [
            {"role": "user", "content": description},
        ]
    })
    return result["messages"][-1].content

```

```
# 主协调智能体
main_agent = create_agent(
    model="gpt-4o",
    tools=[task],
    system_prompt=(
        "You coordinate specialized sub-agents. "
        "Available: research (fact-finding), "
        "writer (content creation). "
        "Use the task tool to delegate work."
    ),
)
```

上下文工程（Context engineering）

Subagents 模式中，需要控制的上下文流主要有三类：

类别	目的	影响对象
Subagent specs	确保在正确场景下调用正确子智能体	主智能体的路由决策
Subagent inputs	确保子智能体拿到足够且适当的上下文来执行任务	子智能体的执行效果
Subagent outputs	确保主智能体能基于子智能体结果做出合理决策	主智能体的编排与表现

子智能体规格（Subagent specs）

子智能体的 **名称（name）** 与 **描述（description）** 是主智能体认知与调用它的主要依据：

- 可以视作**提示工程（prompting levers）**，需要仔细设计；

建议：

- 名称应清晰且偏“动词化”，如：`research_agent`、`code_reviewer`；
- 描述应具体说明：
 - 子智能体负责什么任务；
 - 在什么情况下应当调用它。

对于“单一调度工具”模式： – 主智能体通过调用 `task` 工具，并传入 `agent_name` 来指定子智能体； – 可通过以下方式让主智能体了解可用子智能体： – 在 `system prompt` 中枚举所有可用智能体； – 如果数量较少，可给 `agent_name` 字段加上枚举约束； – 如果数量庞大或动态变化，可提供专门的“发现工具”（如 `list_agents` / `search_agents` ），供主智能体按需查询。

子智能体输入（Subagent inputs）

你可以自定义子智能体接收的上下文，以便其更好地执行任务： – 除了 `query` 文本，还可以从 `state` 中拿到： – 完整消息历史； – 之前的中间结果； – 任务相关元数据等。

示例：从 `state` 中构建子智能体输入

```
from langchain.agents import AgentState
from langchain.tools import tool, ToolRuntime

class CustomState(AgentState):
    example_state_key: str

@tool(
    "subagent1_name",
    description="subagent1_description",
)
def call_subagent1(query: str, runtime: ToolRuntime[None, CustomState]):
    # 根据当前 query 与对话历史构造子智能体输入
    subagent_input = some_logic(query, runtime.state["messages"])
    result = subagent1.invoke({
        "messages": subagent_input,
        # 也可以传递其他 state 字段
        # 需要在主智能体与子智能体的 state schema 中都定义这些键
        "example_state_key": runtime.state["example_state_key"],
    })
    return result["messages"][-1].content
```

子智能体输出 (Subagent outputs)

同样，你也可以自定义主智能体接收的内容，以便更好地使用子智能体结果。

两类常见策略：

1. 通过 prompt 约束子智能体：
2. 在子智能体的 system prompt 中明确说明：
 - 需要在最终消息中包含哪些关键信息；
 - 主智能体只看到“最终输出”，中间推理与工具调用不会直接可见。
3. 在代码中格式化输出：
4. 在子智能体工具包装层对返回结果进行处理或增强；
5. 例如通过 `Command` 将特定 state 字段与最终文本一并返回。

示例：通过 `Command` 返回结构化结果与 `ToolMessage`

```
from typing import Annotated
from langchain.agents import AgentState
from langchain.tools import InjectedToolCallId
from langgraph.types import Command

@tool(
    "subagent1_name",
    description="subagent1_description",
)
def call_subagent1(
    query: str,
    tool_call_id: Annotated[str, InjectedToolCallId],
) -> Command:
    result = subagent1.invoke({
        "messages": [{"role": "user", "content": query}],
    })
    return Command(update={
        # 从子智能体返回并写入主智能体 state
        "example_state_key": result["example_state_key"],
```

```
    "messages": [  
        ToolMessage(  
            content=result["messages"][-1].content,  
            tool_call_id=tool_call_id,  
        ),  
    ],  
})
```

本文档由 LangChain 官方文档翻译而来

\newpage

23. 多智能体模式：技能（Skills）

原文链接: <https://docs.langchain.com/oss/python/langchain/multi-agent/skills>

在 **Skills** 架构中，各种“专门能力”被打包为可调用的 **技能（skills）**，用来增强一个智能体的行为。技能本质上是“按需调用的、以提示词为主的特化能力”。

这个模式在概念上与 Jeremy Howard 提出的 **llms.txt** 十分类似： – llms.txt 使用“工具调用”来渐进式暴露文档； – Skills 模式则把同样的方法应用到“专门的 prompt 与领域知识”上，而不只限于文档页面。

关键特性（Key characteristics）

- 基于 prompt 的专门化（Prompt-driven specialization）：
 - 技能主要由专门化的系统提示 / 上下文定义；
 - 渐进式暴露（Progressive disclosure）：
 - 技能会根据上下文或用户需求“按需出现”；
 - 团队分工（Team distribution）：
 - 不同团队可以独立开发和维护各自的技能；
 - 轻量组合（Lightweight composition）：
 - 技能比完整的子智能体更轻量，只是“给同一个智能体提供不同的专门化上下文”。
-

何时使用（When to use）

在以下场景适合使用 Skills 模式： – 希望只有 **一个智能体**，但拥有许多可能的“专门化模式”； – 不需要在不同技能之间强制执行严格的约束顺序； – 不同团队需要独立开发各自的能力模块。

典型示例： – **编程助手**：针对不同语言或任务的技能（如 `python_debugger`、`refactor_code`）； – **知识库问答**：针对不同业务域的技能（如 `hr_policies`、`product_docs`）； – **创意助手**：针对不同内容形式的技能（如 `blog_post_writer`、`social_copy`、`legal_review`）。

基本实现（Basic implementation）

最基础的 Skills 实现通常是一个“加载技能”的工具：

```
from langchain.tools import tool
from langchain.agents import create_agent

@tool
def load_skill(skill_name: str) -> str:
    """加载一个专门化技能的提示词。

    Available skills:
    - write_sql: SQL query writing expert
    - review_legal_doc: Legal document reviewer

    Returns the skill's prompt and context.
    """
    # 从文件 / 数据库加载技能内容
    ...

agent = create_agent(
    model="gpt-4o",
```



```
tools=[load_skill],
system_prompt=(
    "You are a helpful assistant. "
    "You have access to two skills: "
    "write_sql and review_legal_doc. "
    "Use load_skill to access them."
),
)
```

更完整的实现可以参考官方教程： – [Tutorial: Build a SQL assistant with on-demand skills](#) – 展示如何通过渐进式暴露（progressive disclosure）实现技能： – 智能体不是一开始就加载所有专门 prompt / schema； – 而是按需加载需要的技能。

扩展模式（Extending the pattern）

在自定义实现中，你可以在基础 Skills 模式上做进一步扩展：

1. 动态工具注册（Dynamic tool registration）

将“渐进式暴露”与 state 管理结合起来： – 当加载某个技能时，**动态注册新工具**； – 例如：加载 `database_admin` 技能时，同时： – 添加该技能的专门上下文（prompt / 文档）； – 注册一组数据库专用工具（如 backup、restore、migrate）。

这利用的是多智能体模式中常见的“工具 + 状态”机制： – 工具更新 state； – state 决定当前智能体具备哪些能力。

2. 分层技能（Hierarchical skills）

技能本身也可以定义“子技能”，形成树状结构： – 例如：加载 `data_science` 技能后，使以下子技能可用： – `pandas_expert` – `visualization` – `statistical_analysis` – 每个子技能都可以按需独立加载。

这种层级化方法的好处： – 适合管理大型知识体系； – 能将能力组织成逻辑分组； – 按需发现并加载，从而实现更细粒度的渐进式上下文暴露。

24. 多智能体模式：路由（Router）

原文链接: <https://docs.langchain.com/oss/python/langchain/multi-agent/router>

在 **Router** 架构中，会先有一个“路由步骤”对输入进行分类，然后将其分发给一个或多个专门智能体。这在你有多个**垂直领域（verticals）**——彼此独立的知识域、各自需要单独智能体——时尤其有用。

关键特性（Key characteristics）

- **Router 负责分解查询**：可以把用户请求拆分成多个子查询；
 - **0 个或多个专门智能体可被并行调用**；
 - **结果最终被综合为一个统一响应**。
-

何时使用（When to use）

在以下场景适合使用 Router 模式：

- 存在多个彼此独立的知识域（每个域需要自己的智能体）；
- 需要并行查询多个来源（如 GitHub / Notion / Slack）；
- 最终希望将多源结果综合为一个统一回答。

基本实现（Basic implementation）

Router 的职责是：

- 对查询进行分类；
- 将其路由到相应智能体。

可以使用：

- **Command**：路由到单个专门智能体；
- **Send**：同时向多个智能体“扇出”（fan-out）并行调用。

单智能体路由（Single agent）

使用 **Command** 路由到一个专门智能体：

```

from langgraph.types import Command

def classify_query(query: str) -> str:
    """使用 LLM 对查询进行分类，确定应该路由到哪个智能体。"""
    # 在此实现分类逻辑
    ...

def route_query(state: State) -> Command:
    """根据分类结果路由到合适的智能体。"""
    active_agent = classify_query(state["query"])

    # 路由到选中的智能体
    return Command(goto=active_agent)

```

多智能体并行 (Multiple agents, parallel)

使用 `Send` 将查询“扇出”给多个专门智能体并行处理：

```

from typing import TypedDict
from langgraph.types import Send

class ClassificationResult(TypedDict):
    query: str
    agent: str

def classify_query(query: str) -> list[ClassificationResult]:
    """使用 LLM 对查询进行分类，确定需要调用哪些智能体。"""
    # 在此实现分类逻辑
    ...

```

```
def route_query(state: State):
    """根据分类结果路由到相关智能体。"""
    classifications = classify_query(state["query"])

    # 并行扇出到所选智能体
    return [
        Send(c["agent"], {"query": c["query"]})
        for c in classifications
    ]
```

完整实现可参考官方教程： – **Build a multi-source knowledge base with routing**： – 构建一个 Router，同时查询 GitHub / Notion / Slack 等源，并综合结果； – 覆盖状态定义、专门智能体、`Send` 并行执行与结果综合等内容。

无状态 vs. 有状态 (Stateless vs. stateful)

Router 有两种使用方式： – **无状态 (Stateless router)**：每个请求独立处理； – **有状态 (Stateful router)**：在多轮对话中维护上下文。

无状态 (Stateless)

无状态 router 每个请求都**独立路由**，不会记住之前的调用。如果需要多轮对话能力，见下节有状态 router。

Router vs. Subagents:

二者都可以分发任务给多个智能体，但路由决策方式不同： – **Router**： – 由一个专门的“路由步骤”（通常是一次 LLM 调用或规则逻辑）对输入进行分类并派发； – router 本身一般不维护对话历史，也不做多轮编排，只是预处理步骤； – **Subagents**： – 由一个“主协调智能体 (supervisor)”在多轮对话中动态决定调用哪些子智能体； – 主智能体维护对话上下文，可多轮调用子智能体并执行复杂编排。

总结： – 当你拥有清晰的“输入类别”，并希望某种**确定性 / 轻量级分类**时，使用 **Router**； – 当你需要基于不断演化的上下文做灵活编排时，使用 **supervisor + subagents** 模式更合适。

有状态 (Stateful)

在多轮对话中，你需要在多次调用之间维护上下文。

工具包装法 (Tool wrapper)

最简单的方式是： – 将“无状态 router”包装为一个工具； – 由一个“带记忆的会话型智能体”来调用 router 工具； – 记忆与上下文由会话型智能体负责，router 保持无状态。

这样可以： – 避免在多个并行智能体间自己管理对话历史的复杂性。

```
@tool
def search_docs(query: str) -> str:
    """在多种文档源中进行搜索。"""
    result = workflow.invoke({"query": query}) # 调用无状态 router workflow
    return result["final_answer"]

# 会话型智能体，把 router 当作一个工具使用
conversational_agent = create_agent(
    model,
    tools=[search_docs],
    prompt="You are a helpful assistant. Use search_docs to answer questions."
)
```

完整持久化 (Full persistence)

如果你希望 router 本身也维护状态，可以： – 使用持久化机制存储消息历史； – 在路由到某个智能体时，从 state 中取出历史消息，**选择性地**纳入该智能体的上下文； – 这也是一种上下文工程的手段。

有状态 router 需要你自定义历史管理逻辑： – 如果 router 在不同轮次中切换不同智能体， – 对用户来说，对话的“语气 / 角色”可能感觉不连贯； – 当同时并行调用多个智能体时， – 你需要在 router 层维护“输入 + 综合输出”的历史， – 再在后续路由逻辑中使用这些历史。

在许多需要多轮对话的场景中，可以优先考虑 **Handoffs** 或 **Subagents** 模式： – 它们在设计上更适合表达“多轮、多状态的对话语义”。

25. 多智能体模式：自定义工作流（Custom workflow）

原文链接: <https://docs.langchain.com/oss/python/langchain/multi-agent/custom-workflow>

在 **Custom workflow** 架构中，你可以使用 LangGraph 定义“完全自定义”的执行流程：

- 你对图结构拥有完整控制权；
- 可以自由组合：顺序步骤、条件分支、循环、并行执行等；
- 可以在节点中混合“确定性逻辑”和“智能体行为”。

关键特性（Key characteristics）

- **完全掌控图结构**（Complete control over graph structure）；
- **混合确定性逻辑与智能体逻辑**（Mix deterministic logic with agentic behavior）；
- **支持顺序 / 分支 / 循环 / 并行**（Sequential steps, conditional branches, loops, parallel execution）；
- 可以将其他多智能体模式（subagents、router 等）**嵌入为工作流中的节点**。

何时使用（When to use）

在以下情况适合使用自定义工作流：

- 标准模式（Subagents、Skills 等）无法很好满足需求；
- 需要在一个流程中混合“强约束的确定性步骤”和“灵活的智能体推理”；
- 需要复杂路由或多阶段处理（multi-stage processing）。

在自定义工作流中：

- 每个节点可以是：

 - 普通 Python 函数；
 - 一次 LLM 调用；
 - 一个带工具的完整智能体；

- 你甚至可以将“整个多智能体系统”视作一个节点嵌入到更大工作流中。

官方教程中，“多源知识库 + 路由”（multi-source KB with routing）就是一个典型的自定义工作流示例。

基本实现 (Basic implementation)

核心思想： – 你可以在任意 LangGraph 节点内部直接调用一个 LangChain 智能体； – 这样既能享受自定义工作流带来的灵活性，又能复用已有的智能体与工具。

```
from langchain.agents import create_agent
from langgraph.graph import StateGraph, START, END

agent = create_agent(model="openai:gpt-4o", tools=[...])

def agent_node(state: State) -> dict:
    """在 LangGraph 节点中调用 LangChain 智能体。"""
    result = agent.invoke({
        "messages": [{"role": "user", "content": state["query"]}],
    })
    return {"answer": result["messages"][-1].content}

# 构建一个简单的工作流
workflow = (
    StateGraph(State)
    .add_node("agent", agent_node)
    .add_edge(START, "agent")
    .add_edge("agent", END)
    .compile()
)
```

示例：RAG 工作流（Example: RAG pipeline）

一个常见用例是把“检索（Retrieval）”与智能体结合起来。下面的示例构建了一个 WNBA 数据助手： – 从知识库中检索数据； – 还可以通过工具获取最新新闻； – 使用一个多步骤的 LangGraph 工作流串联整条 RAG 管线。

自定义 RAG 工作流包含三类节点：

1. **Model 节点（Rewrite）**：
 2. 使用结构化输出重写用户问题，使其更适合检索；
3. **确定性节点（Retrieve）**：
 4. 执行向量相似度检索，不涉及 LLM；
5. **Agent 节点（Agent）**：
 6. 在检索到的上下文基础上进行推理；
 7. 可通过工具（如 `get_latest_news`）获取额外信息。

通过 LangGraph 的 State，你可以在工作流各步骤间传递结构化信息： – 每个节点都可以读写 `State` 中的字段； – 方便在不同节点之间共享数据与上下文。

```
from typing import TypedDict
from pydantic import BaseModel
from langgraph.graph import StateGraph, START, END
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore

class State(TypedDict):
    question: str
    rewritten_query: str
    documents: list[str]
    answer: str

# WNBA 知识库：包含阵容、比赛结果和球员数据
embeddings = OpenAIEmbeddings()
vector_store = InMemoryVectorStore(embeddings)
```



```

vector_store.add_texts([
    # 阵容
    "New York Liberty 2024 roster: Breanna Stewart, Sabrina Ionescu, Jonquel Jones,
    "Las Vegas Aces 2024 roster: A'ja Wilson, Kelsey Plum, Jackie Young,
    "Indiana Fever 2024 roster: Caitlin Clark, Aliyah Boston, Kelsey Mitchell",
    # 比赛结果
    "2024 WNBA Finals: New York Liberty defeated Minnesota Lynx 3-2 to win the championship",
    "June 15, 2024: Indiana Fever 85, Chicago Sky 79. Caitlin Clark had 28 points and 11 rebounds.",
    "August 20, 2024: Las Vegas Aces 92, Phoenix Mercury 84. A'ja Wilson had 32 points and 12 rebounds.",
    # 球员数据
    "A'ja Wilson 2024 season stats: 26.9 PPG, 11.9 RPG, 2.6 BPG. Won MVP.",
    "Caitlin Clark 2024 rookie stats: 19.2 PPG, 8.4 APG, 5.7 RPG. Won Rookie of the Year.",
    "Breanna Stewart 2024 stats: 20.4 PPG, 8.5 RPG, 3.5 APG.",
])

retriever = vector_store.as_retriever(search_kwargs={"k": 5})

```

@tool

```

def get_latest_news(query: str) -> str:
    """获取最新的 WNBA 新闻和更新。"""
    # 这里应调用真实新闻 API
    return "Latest: The WNBA announced expanded playoff format for 2025."

```

```

agent = create_agent(
    model="openai:gpt-4o",
    tools=[get_latest_news],
)

```

```

model = ChatOpenAI(model="gpt-4o")

```

```

class RewrittenQuery(BaseModel):
    query: str

```

```

def rewrite_query(state: State) -> dict:

```

```

    """重写用户查询, 使其更利于检索。"""
    system_prompt = """Rewrite this query to retrieve relevant WNBA info
The knowledge base contains: team rosters, game results with scores, and
Focus on specific player names, team names, or stat categories mentioned
    response = model.with_structured_output(RewrittenQuery).invoke([
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": state["question"]},
    ])
    return {"rewritten_query": response.query}

```

```

def retrieve(state: State) -> dict:
    """基于重写后的查询进行检索。"""
    docs = retriever.invoke(state["rewritten_query"])
    return {"documents": [doc.page_content for doc in docs]}

```

```

def call_agent(state: State) -> dict:
    """在检索到的上下文上生成最终回答。"""
    context = "\n\n".join(state["documents"])
    prompt = f"Context:\n{context}\n\nQuestion: {state['question']}"
    response = agent.invoke({"messages": [{"role": "user", "content": prompt}]}
    return {"answer": response["messages"][-1].content_blocks}

```

```

workflow = (
    StateGraph(State)
        .add_node("rewrite", rewrite_query)
        .add_node("retrieve", retrieve)
        .add_node("agent", call_agent)
        .add_edge(START, "rewrite")
        .add_edge("rewrite", "retrieve")
        .add_edge("retrieve", "agent")
        .add_edge("agent", END)
        .compile()
)

```

```
result = workflow.invoke({"question": "Who won the 2024 WNBA Championship?"})
print(result["answer"])
```

本文档由 LangChain 官方文档翻译而来

\newpage

26. 检索 (Retrieval)

原文链接: <https://docs.langchain.com/oss/python/langchain/retrieval>

大型语言模型 (LLM) 很强大，但有两个关键限制：

- **上下文有限 (Finite context)**：一次无法吞下整个语料库；
- **知识静态 (Static knowledge)**：训练数据冻结在某个时间点。

检索 (Retrieval) 通过在查询时获取相关的外部知识来解决这些问题，这是 **检索增强生成 (Retrieval-Augmented Generation, RAG)** 的基础： – 在 LLM 的回答中加入与当前问题紧密相关的外部信息，使结果更加“有据可依”。

构建知识库 (Building a knowledge base)

知识库 (knowledge base) 是在检索阶段使用的一组文档或结构化数据。

如果你需要自定义知识库，可以使用 LangChain 的： – 文档加载器 (Document loaders)； – 向量存储 (Vector stores)；

从你的数据中构建一个可检索的库。

如果你已经有了知识库（如 SQL 数据库、CRM、内部文档系统），**不必重建**： – 在 **Agentic RAG** 中，将它作为智能体的一个 **工具 (tool)**； – 或在 **2-Step RAG** 中，先查询知识库，再将检索结果作为上下文提供给 LLM。

教程示例：**Semantic search**

学习如何使用 LangChain 的文档加载器、embedding 和向量存储，从你自己的数据构建一个可检索的知识库，并在其上实现一个最小 RAG 工作流（如 PDF 上的语义搜索 + 基于检索结果的回答）。

从检索到 RAG (From retrieval to RAG)

检索让 LLM 能在运行时访问相关上下文。但大多数真实应用会更进一步： – 将“检索 + 生成”结合起来，输出**基于检索证据的、上下文感知的答案**；

这就是 **RAG** 的核心思想： – 检索管线成为更大系统的基础，将“搜索”和“生成”组合到一起。

检索管线 (Retrieval pipeline)

一个典型的检索 workflow 大致如下（原文有图示）：

1. 加载与预处理数据 (load & split) ；
2. 生成向量 / 索引 (embeddings + vector store) ；
3. 根据查询检索相关文档 (retriever) ；
4. 使用检索到的文档作为上下文，让 LLM 生成答案；

每个组件都是 **模块化的**： – 你可以在不改业务逻辑的前提下，替换加载器、切分器、embedding 模型或向量存储。

构建模块 (Building blocks)

Document loaders (文档加载器)

- 从外部源 (Google Drive、Slack、Notion 等) 导入数据；
- 返回标准化的 `Document` 对象。

Text splitters (文本切分器)

- 将大文档切分成较小的 chunk；
- 这些 chunk 可以单独被检索，且能适配模型的上下文窗口。

Embedding models (Embedding 模型)

- 将文本映射到向量空间；

- 含义相近的文本在向量空间中彼此接近。

Vector stores（向量存储）

- 用于存储和检索 embedding 的专用数据库；
- 支持相似度搜索、过滤等。

Retrievers（检索器）

- 一个统一接口：给定“非结构化查询”，返回相关 Document 列表；
- 可以基于向量存储、关键词搜索、混合检索等实现。

RAG 架构（RAG architectures）

根据系统需求，RAG 可以用不同方式实现。主要有三种：

架构	描述	控制力	灵活性	延迟	典型用例
2-Step RAG	检索总是先于生成执行，流程简单且可预测	✓ 高	✗ 低	⚡ 快	FAQ、文档问答机器人
Agentic RAG	由智能体（LLM）在推理过程中决定“何时 / 如何”检索	✗ 低	✓ 高	⌚ 可变	拥有多工具的研究助手
Hybrid RAG	结合 2-Step 与 Agentic 的特点，加入校验等中间步骤	⚖️ 中	⚖️ 中	⌚ 可变	需要质量校验的领域问答

延迟说明： – 在 2-Step RAG 中，LLM 调用次数是固定且上限已知的，延迟更可预测； – 现实场景还会受： – 检索步骤的 API 响应时间、网络延迟、数据库性能等影响。

2-Step RAG

在 2-Step RAG 中： – 检索步骤总是在生成步骤之前执行； – 流程相对固定、可预测； – 很适合“先取文档、再基于文档回答”的经典场景。

可参见教程：**Retrieval-Augmented Generation (RAG)**

– 展示如何构建一个基于你数据的 Q&A 聊天机器人； – 包含两种方案： – 使用“RAG 智能体”，通过灵活工具执行搜索； – 使用“2-step RAG 链”，每次查询只需一次 LLM 调用，速度快且简单。

Agentic RAG

Agentic RAG 将 RAG 与智能体推理结合： – 不再“先固定地检索、再回答”； – 而是由一个智能体在多步推理过程中，动态决定： – 何时检索； – 使用哪种检索工具； – 是否再次检索或组合多个来源。

要实现 Agentic RAG，智能体只需要能访问一个或多个**检索工具 (tools)**： – 文档加载器 / 向量检索接口； – Web API； – 数据库查询接口等。

简单示例：使用 `fetch_url` 检索网页

```
import requests
from langchain.tools import tool
from langchain.chat_models import init_chat_model
from langchain.agents import create_agent

@tool
def fetch_url(url: str) -> str:
    """Fetch text content from a URL"""
    response = requests.get(url, timeout=10.0)
    response.raise_for_status()
    return response.text

system_prompt = """\
Use fetch_url when you need to fetch information from a web-page; quote
"""

agent = create_agent(
```

```
model="claude-sonnet-4-5-20250929",
tools=[fetch_url], # 一个用于检索的工具
system_prompt=system_prompt,
)
```

扩展示例：为 LangGraph 文档构建 Agentic RAG (llms.txt)

下面示例实现了一个 **Agentic RAG 系统**，帮助用户查询 LangGraph 文档： – 智能体首先加载 `llms.txt`（列出可用文档 URL）； – 再通过 `fetch_documentation` 工具按需抓取 / 解析文档内容； – 基于这些内容回答用户问题。

```
import requests
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain.tools import tool
from markdownify import markdownify

ALLOWED_DOMAINS = ["https://langchain-ai.github.io/"]
LLMS_TXT = "https://langchain-ai.github.io/langgraph/llms.txt"

@tool
def fetch_documentation(url: str) -> str:
    """Fetch and convert documentation from a URL"""
    if not any(url.startswith(domain) for domain in ALLOWED_DOMAINS):
        return (
            "Error: URL not allowed. "
            f"Must start with one of: {'', ' '.join(ALLOWED_DOMAINS)}"
        )
    response = requests.get(url, timeout=10.0)
    response.raise_for_status()
    return markdownify(response.text)

# 预先获取 llms.txt 内容，这一步无需 LLM 调用
```

```
llms_txt_content = requests.get(LLMS_TXT).text
```

```
# 智能体的系统提示
```

```
system_prompt = f"""
```

```
You are an expert Python developer and technical assistant.
```

```
Your primary role is to help users with questions about LangGraph and re
```

```
Instructions:
```

1. If a user asks a question you're unsure about – or one that likely involves behavior, or configuration – you MUST use the `fetch\_documentation` tool.
2. When citing documentation, summarize clearly and include relevant context.
3. Do not use any URLs outside of the allowed domain.
4. If a documentation fetch fails, tell the user and proceed with your best knowledge.

```
You can access official documentation from the following approved sources:
```

```
{llms_txt_content}
```

```
You MUST consult the documentation to get up to date documentation before answering a user's question about LangGraph.
```

```
Your answers should be clear, concise, and technically accurate.  
"""
```

```
tools = [fetch_documentation]
```

```
model = init_chat_model("claude-sonnet-4-0", max_tokens=32_000)
```

```
agent = create_agent(  
    model=model,  
    tools=tools,  
    system_prompt=system_prompt,  
    name="Agentic RAG",
```



```

)

response = agent.invoke({
    "messages": [
        HumanMessage(
            content=(
                "Write a short example of a langgraph agent using the "
                "prebuilt create react agent. the agent should be able "
                "to look up stock pricing information."
            )
        )
    ]
})

print(response["messages"][-1].content)

```

更多完整示例参见 RAG 教程。

Hybrid RAG

Hybrid RAG 结合了 2-Step 与 Agentic RAG 的特点，通常会加入若干“中间步骤”作为质量控制：

常见组件包括：

- **Query enhancement（查询增强）：**
 - 修改输入问题以提升检索质量；
 - 如重写模糊问题、生成多个变体、注入更多上下文等。
- **Retrieval validation（检索验证）：**
 - 评估检索到的文档是否“足够相关 / 足够充分”；
 - 如果不满意，可以重写查询并重新检索。
- **Answer validation（答案验证）：**

- 检查生成答案的准确性、完整性以及是否与源文档一致；
- 不满足要求时，可触发“再生成 / 修订”。

这类架构通常允许在上述步骤之间多次往返迭代，适用于： – 查询本身模糊或信息不足； – 对答案质量与可追溯性有较高要求； – 需要结合多来源、多轮 refinement 的复杂工作流。

教程示例：Agentic RAG with Self-Correction

– 展示了如何构建带有自我纠错能力的 Agentic RAG 系统，是典型的 Hybrid RAG 架构。

本文档由 LangChain 官方文档翻译而来

\newpage

27. 长期记忆 (Long-term memory)

原文链接: <https://docs.langchain.com/oss/python/langchain/long-term-memory>

概述 (Overview)

LangChain 智能体通过 LangGraph 的持久化能力来实现**长期记忆**。这是一个相对高级的主题，需要对 LangGraph 有一定了解。

记忆存储 (Memory storage)

LangGraph 使用一个 **store** 将长期记忆以 JSON 文档形式持久化： – 每条记忆存放在一个自定义的 **namespace**（类似“文件夹”）下； – 在该命名空间下用一个唯一的 **key**（类似“文件名”）标识； – 命名空间通常会包含用户 ID、组织 ID 或其他标签，便于按层级组织信息； – 支持跨命名空间的内容过滤与相似度搜索。

```
from langgraph.store.memory import InMemoryStore
```

```
def embed(texts: list[str]) -> list[list[float]]:
```

```

# 这里应替换为真实的 embedding 函数或 LangChain 的 embeddings 对象
return [[1.0, 2.0] * len(texts)]

# InMemoryStore 将数据保存在内存字典中，生产环境应使用 DB 驱动的 store
store = InMemoryStore(index={"embed": embed, "dims": 2})
user_id = "my-user"
application_context = "chitchat"
namespace = (user_id, application_context)

store.put(
    namespace,
    "a-memory", # key
    {
        "rules": [
            "User likes short, direct language",
            "User only speaks English & python",
        ],
        "my-key": "my-value",
    },
)

# 通过 ID 读取这条“记忆”
item = store.get(namespace, "a-memory")

# 在该命名空间内搜索“记忆”：
# 按内容过滤（filter）并按向量相似度排序
items = store.search(
    namespace, filter={"my-key": "my-value"}, query="language preference
)

```

关于 memory store 的更多信息可参见 Persistence 指南。

在工具中读取长期记忆（Read long-term memory in tools）

下面示例展示一个“查询用户信息”的工具，供智能体调用：

```
from dataclasses import dataclass

from langchain_core.runnables import RunnableConfig
from langchain.agents import create_agent
from langchain.tools import tool, ToolRuntime
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:
    user_id: str

# InMemoryStore 将数据保存在内存字典中，生产环境应使用 DB 驱动的 store
store = InMemoryStore()

# 使用 put 方法向 store 写入一些示例数据
store.put(
    ("users",),          # 命名空间，用于归类相关数据（这里是用户数据）
    "user_123",          # 命名空间内的 key（这里用 user ID 作为 key）
    {
        "name": "John Smith",
        "language": "English",
    },                  # 为该用户存储的数据
)

@tool
def get_user_info(runtime: ToolRuntime[Context]) -> str:
    """Look up user info."""
    # 访问与 create_agent 传入的同一个 store
```

```

store = runtime.store
user_id = runtime.context.user_id
# 从 store 获取数据, 返回的是带有 value 与 metadata 的 StoreValue 对象
user_info = store.get("users",), user_id)
return str(user_info.value) if user_info else "Unknown user"

agent = create_agent(
    model="claude-sonnet-4-5-20250929",
    tools=[get_user_info],
    # 将 store 传给智能体, 使其在运行工具时可访问该 store
    store=store,
    context_schema=Context,
)

# 运行智能体
agent.invoke(
    {"messages": [{"role": "user", "content": "look up user information"}],
    context=Context(user_id="user_123"),
)

```

在工具中写入长期记忆 (Write long-term memory from tools)

下面示例展示一个“更新用户信息”的工具：

```

from dataclasses import dataclass
from typing_extensions import TypedDict

from langchain.agents import create_agent
from langchain.tools import tool, ToolRuntime
from langgraph.store.memory import InMemoryStore

```

```
# InMemoryStore 将数据保存在内存字典中，生产环境应使用 DB 驱动的 store
store = InMemoryStore()
```

```
@dataclass
class Context:
    user_id: str
```

```
# TypedDict 定义了用户信息的结构，方便 LLM 理解与校验
class UserInfo(TypedDict):
    name: str
```

```
# 允许智能体更新用户信息的工具（常用于聊天应用）
```

```
@tool
```

```
def save_user_info(user_info: UserInfo, runtime: ToolRuntime[Context]) -> str:
    """Save user info."""
    # 访问与 create_agent 传入的同一个 store
    store = runtime.store
    user_id = runtime.context.user_id
    # 将数据写入 store (命名空间、key、data)
    store.put(("users",), user_id, user_info)
    return "Successfully saved user info."
```

```
agent = create_agent(
    model="claude-sonnet-4-5-20250929",
    tools=[save_user_info],
    store=store,
    context_schema=Context,
)
```

```
# 运行智能体
```

```
agent.invoke(
    {"messages": [{"role": "user", "content": "My name is John Smith"}]}
    # 通过 context 中的 user_id 指明要更新哪位用户的信息
```

```
context=Context(user_id="user_123"),
)

# 你也可以直接从 store 读取刚刚写入的数据
store.get(("users",), "user_123").value
```

本文档由 LangChain 官方文档翻译而来

\newpage

28. LangSmith Studio

原文链接: <https://docs.langchain.com/oss/python/langchain/studio>

在本地使用 LangChain 构建智能体时，可视化智能体内部执行过程、实时交互并调试问题会很有帮助。**LangSmith Studio** 是一个免费的视觉界面，用于在本地开发和测试 LangChain 智能体。

Studio 会连接到你在本地运行的智能体，展示：

- 智能体执行的每一步；
- 发送给模型的提示词；
- 工具调用及其结果；
- 最终输出。

你可以：

- 测试不同输入；
- 检查中间状态；
- 迭代优化智能体行为；

而无需额外代码或部署。本页说明如何将 Studio 与本地 LangChain 智能体连接。

前置条件（Prerequisites）

开始前，请确保：

- **LangSmith 账户**：在 smith.langchain.com 注册（免费）或登录。
 - **LangSmith API Key**：按照“创建 API Key 指南”获取。
 - 如果不想将数据追踪到 LangSmith，可在应用的 `.env` 文件中设置 `LANGSMITH_TRACING=false`。禁用追踪后，不会有数据离开你的本地服务器。
-

设置本地智能体服务器 (Set up local Agent server)

1. 安装 LangGraph CLI

LangGraph CLI 提供一个本地开发服务器（也称为 Agent Server），用于将智能体连接到 Studio。

```
# 需要 Python >= 3.11
pip install --upgrade "langgraph-cli[inmem]"
```

2. 准备你的智能体

如果你已有 LangChain 智能体，可以直接使用。下面示例使用一个简单的邮件智能体：

agent.py

```
from langchain.agents import create_agent

def send_email(to: str, subject: str, body: str):
    """Send an email"""
    email = {
        "to": to,
        "subject": subject,
        "body": body,
    }
    # ... email sending logic

    return f"Email sent to {to}"

agent = create_agent(
    "gpt-4o",
    tools=[send_email],
```



```
        system_prompt="You are an email assistant. Always use the send_email  
    )
```

3. 环境变量 (Environment variables)

Studio 需要 LangSmith API Key 来连接本地智能体。在项目根目录创建 `.env` 文件，并添加从 LangSmith 获取的 API Key。

确保 `.env` 文件不会被提交到版本控制系统（如 Git）。

`.env`

```
LANGSMITH_API_KEY=lsv2...
```

4. 创建 LangGraph 配置文件

LangGraph CLI 使用配置文件来定位智能体并管理依赖。在应用目录创建 `langgraph.json` 文件：

`langgraph.json`

```
{  
  "dependencies": ["."],  
  "graphs": {  
    "agent": "./src/agent.py:agent"  
  },  
  "env": ".env"  
}
```

`create_agent` 函数会自动返回一个已编译的 LangGraph 图，这正是配置文件中 `graphs` 键所期望的。

关于配置文件中每个键的详细说明，可参考 [LangGraph 配置文件参考文档](#)。

此时，项目结构应如下：

```
my-app/  
├─ src  
│   └─ agent.py  
├─ .env  
└─ langgraph.json
```

5. 安装依赖 (Install dependencies)

从根目录安装项目依赖：

```
pip install langchain langchain-openai
```

6. 在 Studio 中查看智能体 (View your agent in Studio)

启动开发服务器以将智能体连接到 Studio：

```
langgraph dev
```

Safari 会阻止 `localhost` 连接到 Studio。要解决此问题，可在上述命令中添加 `--tunnel` 参数，通过安全隧道访问 Studio。

服务器运行后，你的智能体可通过以下方式访问： – API：

`http://127.0.0.1:2024` – Studio UI: `https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024`

Studio UI 中的智能体视图：

(原文包含截图，此处略)

连接 Studio 到本地智能体后，你可以快速迭代智能体行为： – 运行测试输入，检查完整执行轨迹，包括： – 提示词 – 工具参数 – 返回值 – token / 延迟指标 – 出现问题时，Studio 会捕获异常及周围状态，帮助理解发生了什么。 – 开发服务器支持热重载：修改代码中的提示词或工具签名，Studio 会立即反映这些更改。 – 可以从任意步骤重新运行对话线程，测试更改而无需从头开始。

该工作流适用于从简单的单工具智能体到复杂的多节点图。

关于如何运行 Studio 的更多信息，可参考 LangSmith 文档中的以下指南： – Run application（运行应用） – Manage assistants（管理助手） – Manage threads（管理线程） – Iterate on prompts（迭代提示词） – Debug LangSmith traces（调试 LangSmith 轨迹） – Add node to dataset（将节点添加到数据集）

视频指南（Video guide）

关于本地和已部署智能体的更多信息，可参见： – Set up local Agent Server（设置本地智能体服务器） – Deploy（部署）

本文档由 LangChain 官方文档翻译而来

\newpage

29. 测试（Test）

原文链接: <https://docs.langchain.com/oss/python/langchain/test>

智能体应用让 LLM 自行决定下一步以解决问题。这种灵活性很强，但模型的“黑盒”特性使得很难预测对智能体某一部分的修改会如何影响其他部分。要构建生产就绪的智能体，充分的测试至关重要。

测试智能体主要有两种方式：

- **单元测试（Unit tests）**：
 - 使用内存假对象（in-memory fakes）隔离测试智能体的小型、确定性片段；
 - 可以快速、确定性地断言精确行为。
- **集成测试（Integration tests）**：
 - 使用真实网络调用测试智能体，确认组件协同工作、凭证与 schema 匹配、延迟可接受。

智能体应用往往更依赖集成测试，因为它们： – 将多个组件串联在一起； – 必须处理因 LLM 的非确定性特性带来的“不稳定性（flakiness）”。

单元测试（Unit testing）

模拟聊天模型（Mocking chat model）

对于不需要 API 调用的逻辑，可以使用内存存根（in-memory stub）来模拟响应。

LangChain 提供 `GenericFakeChatModel` 用于模拟文本响应：

- 它接受一个响应迭代器（`AIMessage` 或字符串）；
- 每次调用返回一个响应；
- 支持常规调用和流式调用。

```
from langchain_core.language_models.fake_chat_models import GenericFakeChatModel

model = GenericFakeChatModel(messages=iter([
    AIMessage(content="", tool_calls=[ToolCall(name="foo", args={"bar":
        "bar"
    ])
]))

model.invoke("hello")
# AIMessage(content='', ..., tool_calls=[{'name': 'foo', 'args': {'bar':
```

如果再次调用模型，它会返回迭代器中的下一个项：

```
model.invoke("hello, again!")
# AIMessage(content='bar', ...)
```

InMemorySaver checkpointer

要在测试中启用持久化，可以使用 `InMemorySaver` checkpointer。这允许你模拟多轮对话，以测试依赖状态的行为：

```
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model,
```

```
tools=[],
    checkpointer=InMemorySaver()
)

# 第一次调用
agent.invoke(HumanMessage(content="I live in Sydney, Australia."))

# 第二次调用：第一条消息已持久化（悉尼位置），因此模型返回 GMT+10 时间
agent.invoke(HumanMessage(content="What's my local time?"))
```

集成测试（Integration testing）

许多智能体行为只有在使用真实 LLM 时才会出现，例如：– 智能体决定调用哪个工具；– 如何格式化响应；– 提示词修改是否影响整个执行轨迹。

LangChain 的 `agentevals` 包提供了专门用于测试智能体轨迹（trajectory）的评估器（evaluators）。AgentEvals 让你可以轻松评估智能体的轨迹（消息的精确序列，包括工具调用），通过以下两种方式：

- **Trajectory match（轨迹匹配）：**
 - 为给定输入硬编码一个参考轨迹，并通过逐步比较验证运行结果。
 - 适合测试明确定义的工作流，你已知预期行为。
 - 当你对应应该调用哪些工具以及调用顺序有特定期望时使用。
- 这种方法是确定性的、快速且成本低，因为它不需要额外的 LLM 调用。
- **LLM-as-judge（LLM 作为评判者）：**
 - 使用 LLM 定性验证智能体的执行轨迹。
 - “评判者”LLM 根据提示词标准（可包含参考轨迹）审查智能体的决策。
 - 更灵活，可以评估效率、适当性等细微方面，但需要 LLM 调用且确定性较低。
 - 当你想评估智能体轨迹的整体质量和合理性，而不需要严格的工具调用或顺序要求时使用。

安装 AgentEvals (Installing AgentEvals)

```
pip install agentevals
```

或者，直接克隆 AgentEvals 仓库。

轨迹匹配评估器 (Trajectory match evaluator)

AgentEvals 提供 `create_trajectory_match_evaluator` 函数，将智能体的轨迹与参考轨迹进行匹配。有四种模式可选：

模式	描述	用例
strict	消息和工具调用按相同顺序精确匹配	测试特定序列（例如，授权前先查找策略）
unordered	允许相同工具调用以任意顺序出现	验证信息检索（当顺序不重要时）
subset	智能体只调用参考中的工具（不允许额外工具）	确保智能体不超过预期范围
superset	智能体至少调用参考中的工具（允许额外工具）	验证至少执行了所需的最小操作

严格匹配 (Strict match)

`strict` 模式确保轨迹包含相同顺序的相同消息和工具调用，但允许消息内容有差异。这在需要强制执行特定操作序列时很有用，例如要求在授权操作前先查找策略。

```
from langchain.agents import create_agent
from langchain.tools import tool
from langchain.messages import HumanMessage, AIMessage, ToolMessage
from agentevals.trajectory.match import create_trajectory_match_evaluator

@tool
def get_weather(city: str):
```

```

    """Get weather information for a city."""
    return f"It's 75 degrees and sunny in {city}."

agent = create_agent("gpt-4o", tools=[get_weather])

evaluator = create_trajectory_match_evaluator(
    trajectory_match_mode="strict",
)

def test_weather_tool_called_strict():
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather in San Francisco?")]
    })

    reference_trajectory = [
        HumanMessage(content="What's the weather in San Francisco?"),
        AIMessage(content="", tool_calls=[
            {"id": "call_1", "name": "get_weather", "args": {"city": "San Francisco"}}
        ]),
        ToolMessage(content="It's 75 degrees and sunny in San Francisco."),
        AIMessage(content="The weather in San Francisco is 75 degrees and sunny.")
    ]

    evaluation = evaluator(
        outputs=result["messages"],
        reference_outputs=reference_trajectory
    )
    # {
    #     'key': 'trajectory_strict_match',
    #     'score': True,
    #     'comment': None,
    # }
    assert evaluation["score"] is True

```

无序匹配 (Unordered match)

`unordered` 模式允许相同工具调用以任意顺序出现，这在你想验证是否检索了特定信息但不关心顺序时很有用。例如，智能体可能需要同时检查城市的天气和活动，但顺序不重要。

```
from langchain.agents import create_agent
from langchain.tools import tool
from langchain.messages import HumanMessage, AIMessage, ToolMessage
from agentevals.trajectory.match import create_trajectory_match_evaluator

@tool
def get_weather(city: str):
    """Get weather information for a city."""
    return f"It's 75 degrees and sunny in {city}."

@tool
def get_events(city: str):
    """Get events happening in a city."""
    return f"Concert at the park in {city} tonight."

agent = create_agent("gpt-4o", tools=[get_weather, get_events])

evaluator = create_trajectory_match_evaluator(
    trajectory_match_mode="unordered",
)

def test_weather_and_events_unordered():
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather and events
    )

    reference_trajectory = [
```



```

HumanMessage(content="What's the weather and events in Boston?")
AIMessage(content="", tool_calls=[
    {"id": "call_1", "name": "get_weather", "args": {"city": "Boston"}},
    {"id": "call_2", "name": "get_events", "args": {"city": "Boston"}},
]),
ToolMessage(content="It's 75 degrees and sunny in Boston.", tool_call_id="call_1"),
ToolMessage(content="Concert at the park in Boston tonight.", tool_call_id="call_2"),
AIMessage(content="The weather in Boston is 75 degrees and sunny")
]

evaluation = evaluator(
    outputs=result["messages"],
    reference_outputs=reference_trajectory,
)
# {
#     'key': 'trajectory_unordered_match',
#     'score': True,
#     'comment': None,
# }
assert evaluation["score"] is True

```

子集匹配 (Subset match)

subset 模式确保智能体只调用参考轨迹中的工具，不允许额外工具。这在你想确保智能体不超过预期范围时很有用。

```

evaluator = create_trajectory_match_evaluator(
    trajectory_match_mode="subset",
)

```

超集匹配 (Superset match)

superset 模式允许智能体调用参考轨迹中的工具以及额外工具。这在你想验证至少执行了所需的最小操作时很有用。

```

evaluator = create_trajectory_match_evaluator(
    trajectory_match_mode="superset",
)

def test_weather_superset():
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather in Boston?")
    ])

    reference_trajectory = [
        HumanMessage(content="What's the weather in Boston?"),
        AIMessage(content="", tool_calls=[
            {"id": "call_1", "name": "get_weather", "args": {"city": "Boston"}
        ]),
        ToolMessage(content="It's 75 degrees and sunny in Boston.", tool_call_id="call_1"),
        AIMessage(content="The weather in Boston is 75 degrees and sunny")
    ]

    evaluation = evaluator(
        outputs=result["messages"],
        reference_outputs=reference_trajectory,
    )
    # {
    #     'key': 'trajectory_superset_match',
    #     'score': True,
    #     'comment': None,
    # }
    assert evaluation["score"] is True

```

你还可以设置 `tool_args_match_mode` 属性和/或 `tool_args_match_overrides` 来自定义评估器如何考虑实际轨迹与参考轨迹中工具调用之间的相等性。默认情况下，只有对同一工具使用相同参数的工具调用才被视为相等。更多细节请访问仓库。

LLM-as-Judge 评估器 (LLM-as-Judge evaluator)

你也可以使用 LLM 通过 `create_trajectory_llm_as_judge` 函数评估智能体的执行路径。与轨迹匹配评估器不同，它不需要参考轨迹，但如果可用也可以提供。

无参考轨迹 (Without reference trajectory)

```
from langchain.agents import create_agent
from langchain.tools import tool
from langchain.messages import HumanMessage, AIMessage, ToolMessage
from agentevals.trajectory.llm import create_trajectory_llm_as_judge, TRAJECTORY_ACCURACY_PROMPT

@tool
def get_weather(city: str):
    """Get weather information for a city."""
    return f"It's 75 degrees and sunny in {city}."

agent = create_agent("gpt-4o", tools=[get_weather])

evaluator = create_trajectory_llm_as_judge(
    model="openai:o3-mini",
    prompt=TRAJECTORY_ACCURACY_PROMPT,
)

def test_trajectory_quality():
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather in Seattle")]
    })

    evaluation = evaluator(
        outputs=result["messages"],
    )
    # {
    #     'key': 'trajectory_accuracy',
```

```
#     'score': True,
#     'comment': 'The provided agent trajectory is reasonable...'
# }
assert evaluation["score"] is True
```

带参考轨迹 (With reference trajectory)

如果你有参考轨迹，可以在提示词中添加额外变量并传入参考轨迹。下面我们使用预构建的 `TRAJECTORY_ACCURACY_PROMPT_WITH_REFERENCE` 提示词，并配置

`reference_outputs` 变量：

```
evaluator = create_trajectory_llm_as_judge(
    model="openai:o3-mini",
    prompt=TRAJECTORY_ACCURACY_PROMPT_WITH_REFERENCE,
)
evaluation = evaluator(
    outputs=result["messages"],
    reference_outputs=reference_trajectory,
)
```

关于如何让 LLM 评估轨迹的更多可配置性，请访问仓库。

异步支持 (Async support)

所有 `agentevals` 评估器都支持 Python `asyncio`。对于使用工厂函数的评估器，异步版本通过在函数名中的 `create_` 后添加 `async` 来提供。

```
from agentevals.trajectory.llm import create_async_trajectory_llm_as_judge
from agentevals.trajectory.match import create_async_trajectory_match_evaluator

async_judge = create_async_trajectory_llm_as_judge(
    model="openai:o3-mini",
    prompt=TRAJECTORY_ACCURACY_PROMPT,
)

async_evaluator = create_async_trajectory_match_evaluator(
```

```
trajectory_match_mode="strict",
)

async def test_async_evaluation():
    result = await agent.ainvoke({
        "messages": [HumanMessage(content="What's the weather?")]
    })

    evaluation = await async_judge(outputs=result["messages"])
    assert evaluation["score"] is True
```

LangSmith 集成 (LangSmith integration)

为了跟踪实验随时间的变化，你可以将评估器结果记录到 LangSmith，这是一个用于构建生产级 LLM 应用的平台，包括追踪、评估和实验工具。

首先，通过设置所需的环境变量来设置 LangSmith：

```
export LANGSMITH_API_KEY="your_langsmith_api_key"
export LANGSMITH_TRACING="true"
```

LangSmith 提供两种主要方法来运行评估：pytest 集成和 `evaluate` 函数。

使用 pytest 集成 (Using pytest integration)

```
import pytest
from langsmith import testing as t
from agentevals.trajectory.llm import create_trajectory_llm_as_judge, TR

trajectory_evaluator = create_trajectory_llm_as_judge(
    model="openai:o3-mini",
    prompt=TRAJECTORY_ACCURACY_PROMPT,
)
```

```

@pytest.mark.langsmith
def test_trajectory_accuracy():
    result = agent.invoke({
        "messages": [HumanMessage(content="What's the weather in SF?")]
    })

    reference_trajectory = [
        HumanMessage(content="What's the weather in SF?"),
        AIMessage(content="", tool_calls=[
            {"id": "call_1", "name": "get_weather", "args": {"city": "SF"}
        ]),
        ToolMessage(content="It's 75 degrees and sunny in SF.", tool_call_id="call_1"),
        AIMessage(content="The weather in SF is 75 degrees and sunny."),
    ]

    # 将输入、输出和参考输出记录到 LangSmith
    t.log_inputs({})
    t.log_outputs({"messages": result["messages"]})
    t.log_reference_outputs({"messages": reference_trajectory})

    trajectory_evaluator(
        outputs=result["messages"],
        reference_outputs=reference_trajectory
    )

```

使用 pytest 运行评估：

```
pytest test_trajectory.py --langsmith-output
```

结果将自动记录到 LangSmith。

使用 evaluate 函数 (Using the evaluate function)

或者，你可以在 LangSmith 中创建数据集并使用 `evaluate` 函数：

```
from langsmith import Client
from agentevals.trajectory.llm import create_trajectory_llm_as_judge, TRAJECTORY_ACCURACY_PROMPT

client = Client()

trajectory_evaluator = create_trajectory_llm_as_judge(
    model="openai:o3-mini",
    prompt=TRAJECTORY_ACCURACY_PROMPT,
)

def run_agent(inputs):
    """返回轨迹消息的智能体函数。"""
    return agent.invoke(inputs)["messages"]

experiment_results = client.evaluate(
    run_agent,
    data="your_dataset_name",
    evaluators=[trajectory_evaluator]
)
```

结果将自动记录到 LangSmith。

要了解更多关于评估智能体的信息，请参阅 [LangSmith 文档](#)。

记录与重放 HTTP 调用 (Recording & replaying HTTP calls)

调用真实 LLM API 的集成测试可能很慢且昂贵，尤其是在 CI/CD 管道中频繁运行时。建议使用库来记录 HTTP 请求和响应，然后在后续运行中重放它们，而无需进行实际网络调用。

你可以使用 `vcrpy` 来实现这一点。如果使用 `pytest`，`pytest-recording` 插件提供了一种简单的方法，只需最少的配置即可启用此功能。

请求/响应记录在 `cassettes` 中，然后在后续运行中用于模拟真实网络调用。

设置 `conftest.py` 文件以从 `cassettes` 中过滤敏感信息：

`conftest.py`

```
import pytest

@pytest.fixture(scope="session")
def vcr_config():
    return {
        "filter_headers": [
            ("authorization", "XXXX"),
            ("x-api-key", "XXXX"),
            # ... 其他你想屏蔽的请求头
        ],
        "filter_query_parameters": [
            ("api_key", "XXXX"),
            ("key", "XXXX"),
        ],
    }
```

然后配置项目以识别 `vcr` 标记：

`pytest.ini` 或 `pyproject.toml`

```
[pytest]
markers =
    vcr: record/replay HTTP via VCR
addopts = --record-mode=once
```

`--record-mode=once` 选项在第一次运行时记录 HTTP 交互，在后续运行中重放它们。

现在，只需用 `vcr` 标记装饰你的测试：


```
@pytest.mark.vcr()
def test_agent_trajectory():
    # ...
```

第一次运行此测试时，你的智能体将进行真实网络调用，pytest 将在 `tests/cassettes` 目录中生成一个 cassette 文件 `test_agent_trajectory.yaml`。后续运行将使用该 cassette 来模拟真实网络调用，前提是智能体的请求与之前的运行相比没有变化。如果它们发生变化，测试将失败，你需要删除 cassette 并重新运行测试以记录新的交互。

当你修改提示词、添加新工具或更改预期轨迹时，保存的 cassettes 将过时，现有测试将失败。你应该删除相应的 cassette 文件并重新运行测试以记录新的交互。

本文档由 LangChain 官方文档翻译而来

\newpage

30. 智能体聊天界面 (Agent Chat UI)

原文链接: <https://docs.langchain.com/oss/python/langchain/ui>

Agent Chat UI 是一个 Next.js 应用，为与任何 LangChain 智能体交互提供对话式界面。它支持：– 实时聊天 – 工具可视化 – 高级功能，如时间旅行调试 (time-travel debugging) 和状态分叉 (state forking)

Agent Chat UI 与使用 `create_agent` 创建的智能体无缝协作，只需最少的设置即可为你的智能体提供交互体验，无论你是在本地运行还是在已部署环境（如 LangSmith）中运行。

Agent Chat UI 是开源的，可以根据你的应用需求进行定制。

你可以在 Agent Chat UI 中使用生成式 UI (generative UI)。更多信息请参见“使用 LangGraph 实现生成式用户界面”。

快速开始 (Quick start)

最快的方式是使用托管版本：

1. 访问 Agent Chat UI
 2. 连接你的智能体：输入你的部署 URL 或本地服务器地址
 3. 开始聊天：UI 会自动检测并渲染工具调用和中断 (interrupts)
-

本地开发 (Local development)

要进行定制或本地开发，你可以在本地运行 Agent Chat UI：

使用 npx

```
# 创建一个新的 Agent Chat UI 项目
npx create-agent-chat-app --project-name my-chat-ui
cd my-chat-ui

# 安装依赖并启动
pnpm install
pnpm dev
```

克隆仓库

你也可以直接克隆 Agent Chat UI 仓库进行开发。

连接到你的智能体 (Connect to your agent)

Agent Chat UI 可以连接到本地和已部署的智能体。启动 Agent Chat UI 后，需要配置它以连接到你的智能体：

1. Graph ID:
2. 输入你的图名称（可在 `langgraph.json` 文件的 `graphs` 下找到）

3. Deployment URL:

4. 你的智能体服务器端点

5. 例如：本地开发使用 `http://localhost:2024`

6. 或已部署智能体的 URL

7. LangSmith API key (可选) :

8. 添加你的 LangSmith API key

9. 如果使用本地智能体服务器，则不需要

配置完成后，Agent Chat UI 会自动获取并显示来自你智能体的任何中断线程 (interrupted threads) 。

Agent Chat UI 开箱即用支持渲染工具调用和工具结果消息。要自定义显示哪些消息，请参见“在聊天中隐藏消息 (Hiding Messages in the Chat) ”。

本文档由 LangChain 官方文档翻译而来

\newpage

31. LangSmith 部署 (LangSmith Deployment)

原文链接: <https://docs.langchain.com/oss/python/langchain/deploy>

当你准备将 LangChain 智能体部署到生产环境时，LangSmith 提供了一个专为智能体工作负载设计的托管平台。传统托管平台是为无状态、短生命周期的 Web 应用构建的，而 LangGraph 专为有状态、长时间运行的智能体设计，需要持久化状态和后台执行。

LangSmith 处理基础设施、扩展和运维问题，让你可以直接从仓库部署。

前置条件 (Prerequisites)

开始前，请确保：

- GitHub 账户
 - LangSmith 账户 (免费注册)
-

部署你的智能体（Deploy your agent）

1. 在 GitHub 上创建仓库

你的应用代码必须位于 GitHub 仓库中才能部署到 LangSmith。支持公共和私有仓库。

对于此快速开始，首先确保你的应用与 LangGraph 兼容（按照本地服务器设置指南），然后将代码推送到仓库。

2. 部署到 LangSmith

步骤 1：导航到 LangSmith Deployment

登录 LangSmith。在左侧边栏，选择 **Deployments**。

步骤 2：创建新部署

点击 **+ New Deployment** 按钮。将打开一个面板，你可以在其中填写必填字段。

步骤 3：链接仓库

如果你是首次使用，或要添加之前未连接的私有仓库，请点击 **Add new account** 按钮，按照说明连接你的 GitHub 账户。

步骤 4：部署仓库

选择你的应用仓库。点击 **Submit** 进行部署。这可能需要大约 15 分钟完成。你可以在 **Deployment details** 视图中查看状态。

3. 在 Studio 中测试你的应用

部署完成后：

1. 选择你刚创建的部署以查看详细信息。
2. 点击右上角的 **Studio** 按钮。Studio 将打开并显示你的图。

4. 获取部署的 API URL

1. 在 LangGraph 的 **Deployment details** 视图中，点击 **API URL** 将其复制到剪贴板。
2. 点击 **URL** 将其复制到剪贴板。

5. 测试 API

现在你可以测试 API：

Python

1. 安装 LangGraph Python：

```
pip install langgraph-sdk
```

1. 向智能体发送消息：

```
from langgraph_sdk import get_sync_client # 或使用 get_client 用于异步

client = get_sync_client(url="your-deployment-url", api_key="your-langsm

for chunk in client.runs.stream(
    None,      # 无线程运行
    "agent",   # 智能体名称，在 langgraph.json 中定义
    input={
        "messages": [{
            "role": "human",
            "content": "What is LangGraph?",
        }],
    },
    stream_mode="updates",
):
    print(f"Receiving new event of type: {chunk.event}...")
    print(chunk.data)
    print("\n\n")
```

REST API

```
curl -s --request POST \  
  --url <DEPLOYMENT_URL>/runs/stream \  
  --header 'Content-Type: application/json' \  
  --header "X-API-Key: <LANGSMITH_API_KEY>" \  
  --data "{  
    \"assistant_id\": \"agent\", # 智能体名称, 在 langgraph.json 中定义  
    \"input\": {  
      \"messages\": [  
        {  
          \"role\": \"human\",  
          \"content\": \"What is LangGraph?\"  
        }  
      ]  
    },  
    \"stream_mode\": \"updates\"  
  }"
```

LangSmith 还提供其他托管选项，包括自托管和混合部署。更多信息请参见 Platform setup overview。

本文档由 LangChain 官方文档翻译而来

\newpage

32. LangSmith 可观测性 (LangSmith Observability)

原文链接: <https://docs.langchain.com/oss/python/langchain/observability>

在使用 LangChain 构建和运行智能体时，你需要了解它们的行为： – 调用了哪些工具 – 生成了什么提示词 – 如何做决策

使用 `create_agent` 构建的 LangChain 智能体自动支持通过 LangSmith 进行追踪 (tracing)。LangSmith 是一个用于捕获、调试、评估和监控 LLM 应用行为的平台。

Traces (轨迹) 记录智能体执行的每一步： – 从初始用户输入到最终响应 – 包括所有工具调用、模型交互和决策点

这些执行数据帮助你： – 调试问题 – 评估不同输入下的性能 – 监控生产环境中的使用模式

本指南说明如何为 LangChain 智能体启用追踪，并使用 LangSmith 分析其执行。

前置条件 (Prerequisites)

开始前，请确保：

- **LangSmith 账户**：在 smith.langchain.com 注册（免费）或登录
 - **LangSmith API Key**：按照“创建 API Key 指南”获取
-

启用追踪 (Enable tracing)

所有 LangChain 智能体自动支持 LangSmith 追踪。要启用它，请设置以下环境变量：

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=<your-api-key>
```

快速开始 (Quickstart)

无需额外代码即可将轨迹记录到 LangSmith。只需像往常一样运行智能体代码：

```
from langchain.agents import create_agent

def send_email(to: str, subject: str, body: str):
    """Send an email to a recipient."""
    # ... email sending logic
    return f"Email sent to {to}"
```

```
def search_web(query: str):
    """Search the web for information."""
    # ... web search logic
    return f"Search results for: {query}"

agent = create_agent(
    model="gpt-4o",
    tools=[send_email, search_web],
    system_prompt="You are a helpful assistant that can send emails and ..."
)

# 运行智能体 - 所有步骤将自动被追踪
response = agent.invoke({
    "messages": [{"role": "user", "content": "Search for the latest AI news"}]
})
```

默认情况下，轨迹将记录到名为 `default` 的项目中。要配置自定义项目名称，请参见“记录到项目（Log to a project）”。

选择性追踪（Trace selectively）

你可以选择使用 LangSmith 的 `tracing_context` 上下文管理器来追踪特定调用或应用的部分：

```
import langsmith as ls

# 这将被追踪
with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "Send a test email"}]})
```



```
# 这不会被追踪（如果未设置 LANGSMITH_TRACING）
agent.invoke({"messages": [{"role": "user", "content": "Send another email"}]})
```

记录到项目（Log to a project）

静态设置（Statically）

你可以通过设置 `LANGSMITH_PROJECT` 环境变量为整个应用设置自定义项目名称：

```
export LANGSMITH_PROJECT=my-agent-project
```

动态设置（Dynamically）

你可以为特定操作以编程方式设置项目名称：

```
import langsmith as ls

with ls.tracing_context(project_name="email-agent-test", enabled=True):
    response = agent.invoke({
        "messages": [{"role": "user", "content": "Send a welcome email"}]
    })
```

向轨迹添加元数据（Add metadata to traces）

你可以使用自定义元数据和标签来注释轨迹：

```
response = agent.invoke(
    {"messages": [{"role": "user", "content": "Send a welcome email"}]},
    config={
        "tags": ["production", "email-assistant", "v1.0"],
        "metadata": {
```

```
        "user_id": "user_123",
        "session_id": "session_456",
        "environment": "production"
    }
}
```

`tracing_context` 也接受标签和元数据，用于细粒度控制：

```
with ls.tracing_context(
    project_name="email-agent-test",
    enabled=True,
    tags=["production", "email-assistant", "v1.0"],
    metadata={"user_id": "user_123", "session_id": "session_456", "environment": "production"},
):
    response = agent.invoke(
        {"messages": [{"role": "user", "content": "Send a welcome email"}]}
```

这些自定义元数据和标签将附加到 LangSmith 中的轨迹。

要了解更多关于如何使用轨迹来调试、评估和监控智能体的信息，请参阅 [LangSmith 文档](#)。

本文档由 LangChain 官方文档翻译而来

\newpage

LangChain 中文教程（Python 版）

本教程由 LangChain 官方 Python 文档翻译整理而成

\newpage